Tugas Besar 1 IF3070 Dasar Inteligensi Artifisial

Pencarian Solusi Kapal Kargo Pengangkut dengan Local Search



Disusun oleh Kelompok:

31 - Claudia

Keisha Sakura Widjaja	18224002
Claudia Melati Krid	18224032
Zahra Zahidah Putri Ruchwijaya	18224058

PROGRAM STUDI SISTEM DAN TEKNOLOGI INFORMASI

SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA

INSTITUT TEKNOLOGI BANDUNG

2026

Daftar Isi

Daftar Isi **2**
Daftar Tabel **4**
BAB 1: Deskripsi Persoalan **6**
1.1 Informasi Status 7
1.2 Solusi Valid 9
1.3 Solusi Tidak Valid 10
BAB 2: Pembahasan **11**
2.1 Pemilihan Fungsi Initial State 11
2.2 Pemilihan Objective Function 12
2.2.1 Fungsi Objektif Dasar 12
2.2.2 Fungsi Objektif untuk Constraint Handling 13
2.2.3 Contoh Perhitungan Skor Objective Function 14
2.2.4 Rumusan Akhir Fungsi Objektif 16
2.3 Evaluation Functions 17
2.4 Main File 25
2.5 Data.json 28
Bab 3: Implementasi Algoritma **29**
3.1 Algoritma Hill Climbing: Steepest Ascent 29
Source Code 30
3.2 Algoritma Hill Climbing: Sideways Move 35
Source Code 36
3.3 Algoritma Hill Climbing: Random Restart 39
Source Code 40
3.4 Algoritma Hill Climbing: Stochastic 46
3.5 Algoritma Simulated Annealing 52
3.6 Algoritma Genetic Algorithm (GA) 56
Bab 4: Hasil Eksperimen **61**
4.1 Hasil Eksperimen Hill Climbing Steepest Ascent 61
4.2 Hasil Eksperimen Hill Climbing Sideways Move 64
4.3 Hasil Eksperimen Hill Climbing Random Restart 67
4.4 Hasil Eksperimen Hill Climbing Stochastic 70
4.5 Hasil Eksperimen Algoritma Simulated Annealing 73
4.6 Hasil Eksperimen Genetic Algorithm 76
Skenario 1: Jumlah Populasi Tetap, Iterasi Bervariasi 76
Skenario 2: Jumlah Iterasi Tetap, Populasi Bervariasi 86
BAB 5: Analisis Eksperimen **95**
5.1 Pertanyaan Pertama 95

5.2 Pertanyaan Kedua 96
5.3 Pertanyaan Ketiga 97
5.4 Pertanyaan Keempat 98
5.5 Pertanyaan Kelima 99
BAB 6: Kesimpulan dan Saran **100**
6.1 Kesimpulan 100
6.2 Saran 101
Pembagian Tugas **102**
Referensi **104**

Daftar Tabel

Tabel 1.1 Informasi Contoh Kasus 6

Tabel 1.2 Daftar Kendaraan Contoh 7

Tabel 1.3 Solusi Valid Contoh Kasus 8

Tabel 1.4 Solusi Tidak Valid Contoh Kasus 9

Tabel 2.1 Contoh Perhitungan Skor Objective Function (1) 13

Tabel 2.2 Contoh Perhitungan Skor Objective Function (2) 14

Tabel 2.3 Deskripsi Evaluation Functions 16

Tabel 2.4 Deskripsi Main File 25

Tabel 3.1 Deskripsi Hill Climbing: Steepest Ascent 28

Tabel 3.2 Deskripsi Hill Climbing: Sideways Move 34

Tabel 3.3 Deskripsi Hill Climbing: Random Restart 38

Tabel 3.4 Deskripsi Hill Climbing: Stochastic 45

Tabel 3.5 Deskripsi Simulated Annealing 51

Tabel 3.6 Deskripsi Genetic Algorithm 55

Tabel 4.1.1 Tabel Hasil Eksperimen Hill Climbing Steepest Ascent 60

Tabel 4.1.2 Plot Nilai Objective Function Algoritma Hill Climbing Steepest Ascent 61

Tabel 4.2.1 Tabel Hasil Eksperimen Hill Climbing Sideways Move 63

Tabel 4.2.2 Plot Nilai Objective Function Algoritma Hill Climbing Sideways Move 64

Tabel 4.3.1 Tabel Hasil Eksperimen Hill Climbing Random Restart 66

Tabel 4.3.2 Plot Nilai Objective Function Algoritma Hill Climbing Random Restart 67

Tabel 4.4.1 Tabel Hasil Eksperimen Hill Climbing Stochastic 69

Tabel 4.4.2 Plot Nilai Objective Function Algoritma Hill Climbing Stochastic 70

Tabel 4.5.1 Tabel Hasil Eksperimen Algoritma Simulated Annealing 72

Tabel 4.5.2 Plot Nilai Objective Function Algoritma Simulated Annealing 73

Tabel 4.6.1 Tabel Hasil Eksperimen Genetic Algorithm Populasi=50 Iterasi=100 75

Tabel 4.6.2 Plot Nilai Objective Function Algoritma Genetic Algorithm Populasi=50 Iterasi=100 77

Tabel 4.6.3 Tabel Hasil Eksperimen Genetic Algorithm Populasi=50 Iterasi=300 79

Tabel 4.6.4 Plot Nilai Objective Function Algoritma Genetic Algorithm Populasi=50 Iterasi=300 80

Tabel 4.6.5 Tabel Hasil Eksperimen Genetic Algorithm Populasi=50 Iterasi=600 82

Tabel 4.6.6 Plot Nilai Objective Function Algoritma Genetic Algorithm Populasi=50 Iterasi=600 83

Tabel 4.6.7 Tabel Hasil Eksperimen Genetic Algorithm Iterasi=300 Populasi=20 85

Tabel 4.6.8 Plot Nilai Objective Function Algoritma Genetic Algorithm Iterasi=300 Populasi=20 86

Tabel 4.6.9 Tabel Hasil Eksperimen Genetic Algorithm Iterasi=300 Populasi=50 88

Tabel 4.6.10 Plot Nilai Objective Function Algoritma Genetic Algorithm Iterasi=300 Populasi=50 89

Tabel 4.6.11 Tabel Hasil Eksperimen Genetic Algorithm Iterasi=300 Populasi=100 91

Tabel 4.6.12 Plot Nilai Objective Function Algoritma Genetic Algorithm Iterasi=300 Populasi=100 92

Tabel 8.1 Pembagian Tugas Anggota Kelompok 101

BAB 1: Deskripsi Persoalan

Tugas Besar 1 mata kuliah IF3070 Dasar Inteligensi Artifisial ini berfokus pada penyelesaian masalah optimasi pemuatan kendaraan ke dalam kapal (Vehicle Loading Problem) yang merupakan salah satu permasalahan optimasi kombinatorial. Permasalahan ini bertujuan untuk menentukan kendaraan mana saja yang akan dimuat ke dalam kapal serta mengatur posisi dan orientasi setiap kendaraan pada area geladak kapal dengan mempertimbangkan keterbatasan dimensi dan kapasitas berat kapal.

Tujuan utama persoalan ini adalah memaksimalkan total pendapatan dari ShippingFee kendaraan yang berhasil dimuat ke dalam kapal sekaligus memastikan bahwa konfigurasi pemuatan memenuhi seluruh batasan yang telah ditentukan. Setiap kendaraan memiliki dimensi, berat, dan ShippingFee yang berbeda-beda. Oleh karena itu, pemilihan kendaraan tidak hanya mempertimbangkan besarnya pendapatan yang diperoleh, tetapi juga kelayakan posisi kendaraan di dalam geladak serta total berat seluruh kendaraan yang dimuat.

Penyelesaian masalah dilakukan menggunakan algoritma local search, yaitu pendekatan optimasi yang mengeksplorasi solusi-solusi tetangga (neighboring solutions) secara iteratif untuk menemukan konfigurasi pemuatan dengan nilai objektif yang lebih baik. Suatu solusi merepresentasikan status setiap kendaraan, yaitu apakah kendaraan dimuat ke dalam kapal atau tidak, beserta koordinat posisi dan orientasinya apabila dimuat. Kendaraan yang tidak dimuat tidak berkontribusi terhadap total ShippingFee yang diperoleh.

Untuk memperoleh solusi pemuatan yang baik, terdapat tiga batasan utama yang perlu dipenuhi:

1. Batasan dimensi dan batas geladak kapal (boundary constraint)
   Setiap kendaraan yang dimuat harus sepenuhnya berada di dalam area geladak kapal. Posisi kendaraan ditentukan oleh koordinat ((x,y)), sedangkan orientasinya dapat berupa horizontal atau vertikal. Perubahan orientasi akan menukar dimensi kendaraan pada sumbu lebar dan panjang. Kendaraan tidak boleh menempati area di luar batas geladak kapal.

2. Batasan tumpang tindih kendaraan (overlap constraint)
   Setiap kendaraan yang dimuat tidak boleh bertumpang tindih dengan kendaraan lainnya. Area yang ditempati oleh setiap kendaraan harus terpisah dari area kendaraan lain agar konfigurasi pemuatan valid secara geometris.
3. Batasan kapasitas berat kapal (weight capacity constraint)
   Total berat seluruh kendaraan yang dimuat tidak boleh melebihi kapasitas berat maksimum kapal. Dengan demikian, pemilihan kendaraan harus mempertimbangkan akumulasi berat kendaraan yang dimuat, bukan hanya pendapatan dari masing-masing kendaraan.

Ketiga batasan tersebut diterapkan secara langsung dalam fungsi evaluasi solusi. Apabila ditemukan pelanggaran berupa tumpang tindih kendaraan, kendaraan berada di luar batas geladak, atau total berat kendaraan melebihi kapasitas kapal, solusi akan dinyatakan tidak valid dan memperoleh nilai objektif sebesar (-1). Sebaliknya, apabila seluruh batasan terpenuhi, nilai objektif dihitung berdasarkan total ShippingFee dari kendaraan yang berhasil dimuat.

Dengan demikian, persoalan ini tidak sekadar menentukan sebanyak mungkin kendaraan yang dapat dimuat, melainkan mencari konfigurasi pemuatan yang menghasilkan total ShippingFee setinggi mungkin dengan tetap memenuhi seluruh batasan. Hasil penyelesaian diharapkan dapat memberikan konfigurasi pemuatan yang efisien dan valid.

1.1 Informasi Status

Tabel 1.1 Informasi Contoh Kasus

Parameter	Nilai
Dimensi geladak kapal	10 × 20 grid
Kapasitas berat maksimum kapal	500 unit
Tujuan optimasi	Memaksimalkan total ShippingFee kendaraan

	yang dimuat
Batasan geometris	Kendaraan tidak boleh bertumpuk dan seluruh bagiannya harus berada di dalam geladak kapal
Batasan berat	Total berat kendaraan yang dimuat tidak boleh melebihi 500 unit

Untuk memberikan ilustrasi konkret mengenai permasalahan pemuatan kendaraan, digunakan contoh lima kendaraan dengan atribut sebagai berikut.

Tabel 1.2 Daftar Kendaraan Contoh

ID Kendaraan	Dimensi (w × l)	Berat	ShippingFee
V001	3 × 4 grid	80 unit	60 juta
V002	4 × 5 grid	120 unit	80 juta
V003	2 × 3 grid	60 unit	45 juta
V004	3 × 3 grid	100 unit	70 juta
V005	2 × 4 grid	70 unit	50 juta
Total		**430 unit**	**305 juta**

Berdasarkan data ilustratif tersebut, total berat seluruh kendaraan adalah 430 unit, sehingga masih berada di bawah kapasitas maksimum kapal sebesar 500 unit. Total ShippingFee apabila seluruh kendaraan berhasil dimuat adalah 305 juta. Namun, kelayakan solusi tetap bergantung pada penempatan geometris setiap kendaraan di dalam geladak.

1.2 Solusi Valid

Salah satu konfigurasi valid untuk memuat seluruh kendaraan dapat ditunjukkan melalui koordinat posisi dan orientasi setiap kendaraan. Pada contoh berikut, semua kendaraan menggunakan orientasi horizontal.

Tabel 1.3 Solusi Valid Contoh Kasus

ID Kendaraan	Koordinat (x, y)	Dimensi Efektif	ShippingFee
V001	(0, 0)	3 × 4 grid	60 juta
V002	(3, 0)	4 × 5 grid	80 juta
V003	(7, 0)	2 × 3 grid	45 juta
V004	(0, 5)	3 × 3 grid	70 juta
V005	(3, 6)	2 × 4 grid	50 juta
Total		**430 unit berat**	**305 juta**

Konfigurasi tersebut memenuhi seluruh batasan karena:

1. Batas geladak terpenuhi. Seluruh kendaraan berada di dalam area geladak berukuran 10 × 20 grid.
2. Tidak terjadi tumpang tindih. Setiap kendaraan menempati area grid yang berbeda.
3. Kapasitas berat terpenuhi. Total berat sebesar 430 unit tidak melebihi kapasitas kapal sebesar 500 unit.

Dengan demikian, nilai objektif dari solusi valid tersebut adalah 305 juta, sesuai dengan total ShippingFee seluruh kendaraan yang dimuat tanpa pelanggaran constraint.

1.3 Solusi Tidak Valid

Suatu solusi dinyatakan tidak valid apabila setidaknya satu kendaraan bertumpuk dengan kendaraan lain, berada di luar batas geladak, atau menyebabkan total berat melebihi kapasitas kapal.

Tabel 1.4 Solusi Tidak Valid Contoh Kasus

Kasus	Konfigurasi	Pelanggaran
1	V001 dan V002 ditempatkan pada koordinat (0, 0) dengan orientasi horizontal	Overlap: kedua kendaraan menempati sebagian sel grid yang sama.
2	V002 ditempatkan pada koordinat (8, 0) dengan orientasi horizontal	Boundary violation: lebar kendaraan 4 grid menyebabkan batas akhirnya mencapai (x=12), melewati lebar geladak 10 grid.
3	V001, V002, V003, V004, V005, serta kendaraan tambahan V006 dengan berat 100 unit dimuat ke kapal	Weight overflow: total berat mencapai 530 unit, melebihi kapasitas maksimum sebesar 30 unit.

Pada kasus ketiga, nilai ShippingFee V006 tidak diperlukan untuk menunjukkan pelanggaran kapasitas. Dengan asumsi penempatan kendaraan tidak melanggar batasan geometris lainnya, total berat 530 unit sudah cukup untuk menyatakan solusi tidak valid.

Berdasarkan fungsi evaluasi yang digunakan, ketiga kasus tersebut memperoleh nilai objektif sebesar (-1), terlepas dari besarnya total ShippingFee kendaraan yang dimuat.

BAB 2: Pembahasan

2.1 Pemilihan Fungsi Initial State

Pada setiap eksperimen, fungsi pembentuk initial state akan dipanggil untuk menghasilkan solusi awal yang nantinya akan dibandingkan dengan solusi akhir dari algoritma yang ditentukan. Fungsi ini dibuat dengan pendekatan pembangkitan state acak, karena sederhana, efisien, dan memberikan titik awal yang beragam sehingga performa algoritma dapat diamati dari kondisi awal yang berbeda beda. Cara kerjanya adalah mengiterasi seluruh kendaraan, lalu memberi masing masing orientasi acak dan koordinat acak yang dijamin berada di dalam batas geladak, serta menentukan secara acak (peluang 0.5) apakah kendaraan tersebut dimuat atau tidak. Metode ini dipakai pada Steepest Ascent, Stochastic, Random Restart, dan Genetic Algorithm. Konsekuensinya, state awal tidak selalu valid karena kendaraan yang dimuat bisa saling bertumpang tindih atau melebihi kapasitas berat. Kondisi ini terlihat pada Steepest Ascent run 1 yang berhenti pada skor invalid, karena semua solusi invalid bernilai sama sehingga tidak ada tetangga yang lebih baik.

Untuk Hill Climbing Sideways Move dan Simulated Annealing, kami menambahkan pendekatan lain agar pencarian selalu dimulai dari state yang valid. Seluruh kendaraan mula mula ditempatkan di luar kapal, kemudian dimasukkan satu per satu menurut urutan acak dengan koordinat dan orientasi acak. Setiap pemasukan langsung dievaluasi, dan jika menyebabkan overlap atau kelebihan kapasitas, pemasukan dibatalkan dan kendaraan tetap berada di luar kapal. Pendekatan ini dipilih karena Sideways Move berhenti bila tidak ada tetangga valid, sedangkan Simulated Annealing membandingkan selisih nilai objektif yang tidak bermakna bila state awalnya invalid. Pada kedua pendekatan, urutan dan posisi yang diacak membuat initial state berbeda pada setiap run, sehingga hasil akhir algoritma dapat dibandingkan dari berbagai kondisi awal.

2.2 Pemilihan Objective Function

Program optimasi pemuatan kendaraan ke dalam kapal (Vehicle Loading Problem) menggunakan fungsi objektif untuk mengevaluasi kualitas suatu konfigurasi pemuatan. Terdapat dua aspek utama yang diperhatikan dalam evaluasi solusi, yaitu:

1. **Maksimalisasi total ShippingFee**, yaitu memaksimalkan total pendapatan dari seluruh kendaraan yang berhasil dimuat ke dalam kapal.
2. **Pemenuhan constraint**, yaitu memastikan bahwa konfigurasi pemuatan tidak melanggar batasan tumpang tindih kendaraan (overlap constraint), batas geladak kapal (boundary constraint), dan kapasitas berat kapal (weight capacity constraint).

Dalam proses pencarian solusi, algoritma local search berusaha memaksimalkan fungsi objektif (f(s)), dengan (s) merepresentasikan suatu state yang berisi status pemuatan, posisi, dan orientasi setiap kendaraan. Kendaraan dapat berada di dalam kapal atau di luar kapal. Kendaraan yang berada di luar kapal tidak berkontribusi terhadap total ShippingFee.

Berbeda dengan pendekatan yang memberikan penalti berdasarkan tingkat pelanggaran, implementasi ini menggunakan pendekatan hard constraint. Solusi yang melanggar salah satu constraint langsung dianggap tidak valid dan memperoleh nilai objektif (-1). Sementara itu, solusi yang memenuhi seluruh constraint memperoleh nilai objektif berupa total ShippingFee kendaraan yang dimuat.

2.2.1 Fungsi Objektif Dasar

Fungsi objektif dasar berfokus pada maksimalisasi pendapatan pengiriman dari kendaraan yang berhasil dimuat. Fungsi ini dirumuskan sebagai:

$$f(s) = \begin{cases} -1, & \text{jika solusi } s \text{ melanggar constraint} \\ \sum_{v \in V_{\text{placed}}} \text{ShippingFee}(v), & \text{jika solusi } s \text{ valid} \end{cases}$$

Keterangan:

- (s) = state atau konfigurasi pemuatan kendaraan.
- (V_{placed}) = himpunan kendaraan yang berhasil dimuat ke dalam kapal.
- $(\text{ShippingFee}(v))$ = nilai biaya pengiriman kendaraan (v).

Fungsi tersebut menjumlahkan ShippingFee dari seluruh kendaraan yang berada di dalam kapal. Kendaraan yang tidak dimuat tidak memberikan kontribusi terhadap nilai objektif. Dengan demikian, semakin besar total ShippingFee kendaraan yang berhasil dimuat, semakin tinggi nilai objektif yang diperoleh, selama konfigurasi pemuatan tetap memenuhi seluruh constraint.

2.2.2 Fungsi Objektif untuk Constraint Handling

Persoalan ini memiliki tiga hard constraints yang wajib dipenuhi, yaitu:

1. **Overlap constraint:** kendaraan yang dimuat tidak boleh menempati area geladak yang sama dengan kendaraan lainnya.
2. **Boundary constraint:** seluruh bagian kendaraan harus berada di dalam batas dimensi geladak kapal.
3. **Weight capacity constraint:** total berat seluruh kendaraan yang dimuat tidak boleh melebihi kapasitas berat maksimum kapal.

Implementasi fungsi objektif menggunakan pendekatan nilai penalti mutlak melalui pemeriksaan constraint pada fungsi objective_function(). Jika ditemukan pelanggaran overlap, pelanggaran batas geladak, atau kelebihan kapasitas berat, fungsi langsung mengembalikan nilai (-1). Solusi yang melanggar constraint tidak memperoleh skor berdasarkan total ShippingFee, meskipun pendapatan potensialnya tinggi.

Pendekatan ini dipilih karena ketiga constraint merupakan batasan mutlak yang tidak boleh dilanggar. Oleh karena itu, program tidak membedakan tingkat pelanggaran

antara dua solusi yang tidak valid. Solusi yang melanggar satu constraint maupun beberapa constraint sama-sama memperoleh nilai (-1).

Dari sisi komputasi, pendekatan ini juga relatif sederhana dan efisien. Ketika suatu pelanggaran terdeteksi, fungsi dapat langsung mengembalikan nilai (-1) tanpa melanjutkan pemeriksaan constraint berikutnya atau menghitung total ShippingFee. Dengan demikian, proses evaluasi dapat dihentikan lebih awal untuk solusi yang telah dipastikan tidak valid.

2.2.3 Contoh Perhitungan Skor Objective Function

Misalkan digunakan contoh kapal dengan kapasitas berat maksimum 500 unit dan lima kendaraan berikut.

Tabel 2.1 Contoh Perhitungan Skor Objective Function (1)

Kendaraan	Berat	ShippingFee
V001	80 unit	60 juta
V002	120 unit	80 juta
V003	60 unit	45 juta
V004	100 unit	70 juta
V005	70 unit	50 juta
Total	**430 unit**	**305 juta**

Berdasarkan data tersebut, berikut ilustrasi perhitungan nilai objektif untuk beberapa state pemuatan.

Tabel 2.2 Contoh Perhitungan Skor Objective Function (2)

State	Konfigurasi	Total Berat	Total Shipping Fee	Validitas	Skor Akhir
A	V001, V002, V003	260 unit	185 juta	Valid	185
B	V001, V002, V003, V004, V005	430 unit	305 juta	Valid	305
C	V001, V002, V003, dengan V001 dan V002 bertumpang tindih	260 unit	185 juta	Tidak valid: overlap	-1
D	V001, V002, V003, dengan V002 melewati batas geladak	260 unit	185 juta	Tidak valid: boundary	-1
E	V001, V002, V003, V004, V005, dan V006 dengan berat 100 unit	530 unit	Tidak perlu dihitung	Tidak valid: kelebihan kapasitas	-1

Pada State A, ketiga kendaraan memiliki total berat 260 unit, yang masih berada di bawah kapasitas maksimum kapal. Jika seluruh kendaraan ditempatkan tanpa overlap dan tetap berada di dalam geladak, state tersebut valid dan memperoleh skor sebesar:

$$f(A) = 60 + 80 + 45 = 185$$

Pada State B, seluruh lima kendaraan memiliki total berat 430 unit, yang masih memenuhi kapasitas kapal sebesar 500 unit. Dengan asumsi tidak terjadi overlap dan seluruh kendaraan berada di dalam batas geladak, skor yang diperoleh adalah:

$$f(B) = 60 + 80 + 45 + 70 + 50 = 305$$

State B memiliki nilai objektif lebih tinggi daripada State A karena menghasilkan total ShippingFee yang lebih besar, sekaligus tetap memenuhi seluruh constraint.

Sebaliknya, State C dan State D memperoleh skor (-1) karena masing-masing melanggar overlap constraint dan boundary constraint. State E juga memperoleh skor (-1) karena total berat kendaraan mencapai 530 unit, melebihi kapasitas kapal sebesar 30 unit. Pada ketiga state tersebut, nilai ShippingFee tidak digunakan sebagai skor akhir karena solusi telah dinyatakan tidak valid.

Contoh ini menunjukkan bahwa tingginya total ShippingFee saja tidak cukup untuk menghasilkan skor objektif yang tinggi. Solusi harus memenuhi seluruh constraint terlebih dahulu agar nilai pendapatan pengiriman dapat digunakan sebagai nilai objektif.

2.2.4 Rumusan Akhir Fungsi Objektif

Berdasarkan kedua aspek tersebut, fungsi objektif akhir dirumuskan sebagai berikut:

$$f(s) = \begin{cases} -1, & \text{jika terjadi overlap} \\ -1, & \text{jika terjadi pelanggaran boundary} \\ -1, & \text{jika total berat melebihi kapasitas kapal} \\ \sum_{v \in V_{placed}} \text{ShippingFee}(v), & \text{jika solusi valid} \end{cases}$$

Dengan rumusan tersebut, fungsi objektif memprioritaskan validitas konfigurasi pemuatan sebelum mempertimbangkan besarnya pendapatan. Solusi valid dengan total ShippingFee lebih tinggi akan memperoleh skor lebih besar, sedangkan seluruh solusi yang melanggar constraint memperoleh skor (-1).

Fungsi objektif ini digunakan oleh algoritma pencarian untuk membandingkan state saat ini dengan state tetangganya (neighboring states). Pada algoritma Steepest Ascent

Hill Climbing, perpindahan dilakukan apabila ditemukan tetangga dengan nilai objektif yang lebih tinggi. Pencarian berhenti ketika tidak ditemukan tetangga yang mampu meningkatkan nilai objektif atau ketika batas maksimum iterasi tercapai.

2.3 Evaluation Functions

File evaluation.py dibuat untuk membangun beberapa fungsi evaluasi atau utilitas terpusat yang digunakan pada banyak algoritma sehingga output yang dihasilkan lebih seragam. Evaluation functions mencakup metode-metode yang umum digunakan di seluruh algoritma seperti loading data, pengujian hasil iterasi terhadap constraints, penghitungan objective function, manipulasi peletakan kendaraan, serta generate neighbor.

Tabel 2.3 Deskripsi Evaluation Functions

Fungsi / Prosedur	**Deskripsi**
load_data(file_path)	Membaca file JSON berisi data kapal dan daftar kendaraan, lalu memisahkannya menjadi data kapal dan data kendaraan.
check_overlap(placed_vehicles)	Memeriksa constraint no overlapping: menandai setiap sel geladak yang ditempati kendaraan, lalu mengecek apakah ada sel yang ditempati lebih dari satu kendaraan.
check_boundary_violation(placed_vehicles, ship_dim)	Memeriksa constraint batas geladak: mengecek apakah ada kendaraan yang sebagian badannya berada di luar geladak (koordinat negatif atau melewati lebar/panjang kapal).
check_weight_violation(placed_vehicles, max_capacity)	Memeriksa constraint kapasitas: menjumlahkan berat semua kendaraan di kapal dan membandingkannya dengan kapasitas maksimum.
objective_function(state,	Menghitung nilai suatu state. Jika state melanggar salah

ship)	satu constraint (overlap, batas geladak, atau kapasitas), state ditolak dengan nilai −1. Jika memenuhi semua constraint, nilainya adalah total ShippingFee kendaraan di kapal.
ORIENTATIONS	Konstanta berisi dua kemungkinan orientasi kendaraan, "Horizontal" dan "Vertical".
get_size(vehicle, orientation)	Menghitung ukuran kendaraan pada sumbu x (lebar kapal) dan sumbu y (panjang kapal) sesuai orientasinya. Horizontal: lebar kendaraan di sumbu x; Vertical: ukuran ditukar.
fits(vehicle, orientation, ship)	Memeriksa apakah satu kendaraan dengan orientasi tertentu muat di geladak kosong, tanpa memperhitungkan kendaraan lain.
random_position(vehicle, orientation, ship)	Menghasilkan koordinat acak untuk kendaraan dengan jaminan seluruh badan kendaraan berada di dalam geladak.
clamp_placement(placement, vehicle, ship)	Memperbaiki penempatan satu kendaraan yang keluar batas geladak: memutar orientasinya jika tidak muat, lalu menggeser koordinatnya hingga kembali ke dalam geladak. Mengubah placement secara langsung.
generate_random_state(vehicles, ship, p_inside=0.5)	Membuat initial state acak. Setiap kendaraan mendapat orientasi acak, koordinat acak di dalam geladak, dan dimasukkan ke kapal dengan peluang p_inside.
decode_state(state, vehicles)	Mengubah state menjadi format yang dibaca objective_function: data kendaraan asli digabung dengan koordinat dan orientasi dari state, lalu dipisahkan menjadi kendaraan di dalam dan di luar kapal.

evaluate_state(state, vehicles, ship)	Menghitung nilai objective suatu state secara langsung, dengan menggabungkan decode_state dan objective_function.
swap_vehicles(state, i, j, vehicles, ship)	Menukar dua kendaraan, baik di dalam maupun di luar kapal. Kendaraan i dan j bertukar koordinat serta status di dalam/luar kapal, lalu penempatannya diperbaiki dengan clamp_placement jika keluar batas. Mengubah state secara langsung.
move_vehicle(state, i, vehicles, ship, p_out=0.2)	Memindahkan kendaraan i ke koordinat acak lain di geladak. Kendaraan yang berada di kapal dipindah ke luar kapal dengan peluang p_out; kendaraan di luar kapal dimasukkan ke koordinat acak. Mengubah state secara langsung.
rotate_vehicle(state, i, vehicles, ship)	Mengubah orientasi kendaraan i (Horizontal ↔ Vertical) jika kendaraan masih muat dalam orientasi baru, lalu menggeser posisinya jika keluar batas. Mengubah state secara langsung.
random_neighbor(state, vehicles, ship)	Membuat satu neighbor acak: menyalin state, memilih satu kendaraan dan satu langkah secara acak (swap, move, atau rotate), lalu menerapkannya. State asli tidak berubah. Digunakan oleh Stochastic Hill-Climbing dan Simulated Annealing.
all_neighbors(state, vehicles, ship)	Membangkitkan seluruh neighbor dari suatu state: semua pasangan swap (minimal salah satu kendaraan di kapal), semua kemungkinan koordinat baru untuk setiap kendaraan, pemindahan kendaraan ke luar kapal, dan semua rotate yang mungkin.
best_neighbor(state,	Mencari neighbor dengan nilai objective tertinggi di

vehicles, ship)	antara seluruh neighbor. Jika ada beberapa neighbor dengan nilai tertinggi yang sama, salah satunya dipilih secara acak. Digunakan oleh Steepest Ascent, Sideways Move, dan Random Restart Hill-Climbing.

Source Code

```
import copy
import json
import random

def load_data(file_path):
    with open(file_path, 'r') as f:
        data = json.load(f)

    ship = data['Ship']
    vehicles = data['Vehicles']

    return ship, vehicles

def check_overlap(placed_vehicles):

    occupied_cells = set()
    for v in placed_vehicles:
        w = v['Dimensions']['w'] if v['Orientation'] == 'Horizontal' else
v['Dimensions']['l']
        l = v['Dimensions']['l'] if v['Orientation'] == 'Horizontal' else
v['Dimensions']['w']

        x_start, y_start = v['x'], v['y']

        for x in range(x_start, x_start + w):
            for y in range(y_start, y_start + l):
                if (x, y) in occupied_cells:
                    return True
                occupied_cells.add((x, y))
    return False

def check_boundary_violation(placed_vehicles, ship_dim):
```

```
    ship_w, ship_l = ship_dim['w'], ship_dim['l']
    for v in placed_vehicles:
        w = v['Dimensions']['w'] if v['Orientation'] == 'Horizontal' else
v['Dimensions']['l']
        l = v['Dimensions']['l'] if v['Orientation'] == 'Horizontal' else
v['Dimensions']['w']

        if v['x'] < 0 or v['y'] < 0 or (v['x'] + w) > ship_w or (v['y'] + l) >
ship_l:
            return True
    return False

def check_weight_violation(placed_vehicles, max_capacity):
    total_weight = sum(v['Weight'] for v in placed_vehicles)
    return max(0, total_weight - max_capacity)

def objective_function(state, ship):
    placed_vehicles = state['placed_vehicles']

    if check_overlap(placed_vehicles):
        return -1

    if check_boundary_violation(placed_vehicles, ship['Dimensions']):
        return -1

    if check_weight_violation(placed_vehicles, ship['MaxCapacity']) > 0:
        return -1

    return sum(v['ShippingFee'] for v in placed_vehicles)



ORIENTATIONS = ("Horizontal", "Vertical")


def get_size(vehicle, orientation):
    w, l = vehicle["Dimensions"]["w"], vehicle["Dimensions"]["l"]
    return (w, l) if orientation == "Horizontal" else (l, w)
```

```
def fits(vehicle, orientation, ship):
    size_x, size_y = get_size(vehicle, orientation)
    return size_x <= ship["Dimensions"]["w"] and size_y <= ship["Dimensions"]["l"]


def random_position(vehicle, orientation, ship):
    size_x, size_y = get_size(vehicle, orientation)
    x = random.randint(0, ship["Dimensions"]["w"] - size_x)
    y = random.randint(0, ship["Dimensions"]["l"] - size_y)
    return x, y


def clamp_placement(placement, vehicle, ship):
    if not fits(vehicle, placement["Orientation"], ship):
        placement["Orientation"] = "Vertical" if placement["Orientation"] ==
"Horizontal" else "Horizontal"
    size_x, size_y = get_size(vehicle, placement["Orientation"])
    placement["x"] = min(max(0, placement["x"]), ship["Dimensions"]["w"] - size_x)
    placement["y"] = min(max(0, placement["y"]), ship["Dimensions"]["l"] - size_y)


def generate_random_state(vehicles, ship, p_inside=0.5):
    state = []
    for v in vehicles:
        orientation = random.choice(ORIENTATIONS)
        if not fits(v, orientation, ship):
            orientation = "Vertical" if orientation == "Horizontal" else
"Horizontal"
        x, y = random_position(v, orientation, ship)
        state.append({
            "inside": random.random() < p_inside,
            "x": x,
            "y": y,
            "Orientation": orientation,
        })
    return state


def decode_state(state, vehicles):
    placed, outside = [], []
    for placement, v in zip(state, vehicles):
```

```
        if placement["inside"]:
            placed_v = copy.deepcopy(v)
            placed_v["x"] = placement["x"]
            placed_v["y"] = placement["y"]
            placed_v["Orientation"] = placement["Orientation"]
            placed.append(placed_v)
        else:
            outside.append(copy.deepcopy(v))
    return {"placed_vehicles": placed, "outside_vehicles": outside}


def evaluate_state(state, vehicles, ship):
    return objective_function(decode_state(state, vehicles), ship)

def swap_vehicles(state, i, j, vehicles, ship):
    a, b = state[i], state[j]
    a["inside"], b["inside"] = b["inside"], a["inside"]
    a["x"], b["x"] = b["x"], a["x"]
    a["y"], b["y"] = b["y"], a["y"]
    clamp_placement(a, vehicles[i], ship)
    clamp_placement(b, vehicles[j], ship)


def move_vehicle(state, i, vehicles, ship, p_out=0.2):
    placement = state[i]
    if placement["inside"] and random.random() < p_out:
        placement["inside"] = False
        return
    placement["inside"] = True
    placement["x"], placement["y"] = random_position(vehicles[i],
placement["Orientation"], ship)


def rotate_vehicle(state, i, vehicles, ship):
    placement = state[i]
    new_orientation = "Vertical" if placement["Orientation"] == "Horizontal" else
"Horizontal"
    if fits(vehicles[i], new_orientation, ship):
        placement["Orientation"] = new_orientation
        clamp_placement(placement, vehicles[i], ship)
```

```
def random_neighbor(state, vehicles, ship):
    neighbor = copy.deepcopy(state)
    n = len(neighbor)
    i = random.randrange(n)
    move = random.choice(("swap", "move", "rotate"))
    if move == "swap" and n > 1:
        j = random.choice([k for k in range(n) if k != i])
        swap_vehicles(neighbor, i, j, vehicles, ship)
    elif move == "rotate" and neighbor[i]["inside"]:
        rotate_vehicle(neighbor, i, vehicles, ship)
    else:
        move_vehicle(neighbor, i, vehicles, ship)
    return neighbor


def all_neighbors(state, vehicles, ship):
    neighbors = []
    n = len(state)
    ship_w, ship_l = ship["Dimensions"]["w"], ship["Dimensions"]["l"]

    for i in range(n):
        for j in range(i + 1, n):
            if state[i]["inside"] or state[j]["inside"]:
                neighbor = copy.deepcopy(state)
                swap_vehicles(neighbor, i, j, vehicles, ship)
                neighbors.append(neighbor)

    for i in range(n):
        size_x, size_y = get_size(vehicles[i], state[i]["Orientation"])
        for x in range(ship_w - size_x + 1):
            for y in range(ship_l - size_y + 1):
                if state[i]["inside"] and x == state[i]["x"] and y == state[i]["y"]:
                    continue
                neighbor = copy.deepcopy(state)
                neighbor[i].update({"inside": True, "x": x, "y": y})
                neighbors.append(neighbor)
        if state[i]["inside"]:
            neighbor = copy.deepcopy(state)
            neighbor[i]["inside"] = False
            neighbors.append(neighbor)
```

```
    for i in range(n):
        if state[i]["inside"]:
            new_orientation = "Vertical" if state[i]["Orientation"] == "Horizontal"
else "Horizontal"
            if fits(vehicles[i], new_orientation, ship):
                neighbor = copy.deepcopy(state)
                rotate_vehicle(neighbor, i, vehicles, ship)
                neighbors.append(neighbor)

    return neighbors


def best_neighbor(state, vehicles, ship):
    best_value = float("-inf")
    best_candidates = []
    for neighbor in all_neighbors(state, vehicles, ship):
        value = evaluate_state(neighbor, vehicles, ship)
        if value > best_value:
            best_value = value
            best_candidates = [neighbor]
        elif value == best_value:
            best_candidates.append(neighbor)
    return random.choice(best_candidates), best_value
```

2.4 Main File

File main.py dibuat sebagai antarmuka grafis untuk menjalankan eksperimen seluruh algoritma. Fungsinya mencakup pemuatan data, pengaturan parameter, pelaksanaan pengulangan eksperimen, serta penyeragaman format hasil untuk ditampilkan. Hasil disajikan dalam bentuk ringkasan ShippingFee dan statistik pencarian, grafik perkembangan objective function, serta visualisasi susunan kendaraan awal dan akhir. File ini juga mengatur penyimpanan hasil eksperimen ke folder output berdasarkan algoritma, waktu pelaksanaan, konfigurasi, dan nomor pengulangan.

Tabel 2.4 Deskripsi Main File

Fungsi / Prosedur	Deskripsi
read_parameters(algorithm, values)	Membaca nilai parameter dari input GUI dan mengubahnya menjadi int atau float sesuai kebutuhan algoritma. Fungsi memeriksa batas nilai, seperti mutation rate antara 0–1, cooling rate di antara 0 dan 1, serta jumlah iterasi yang harus positif. Mengembalikan dictionary parameter untuk diteruskan ke algoritma.
run_algorithm(algorithm, vehicles, ship, parameters)	Memanggil algoritma yang dipilih menggunakan data kendaraan, kapal, dan parameter input. Hasil berbentuk tuple dari Sideways dan Simulated Annealing diubah menjadi dictionary, sedangkan state internalnya diterjemahkan menggunakan decode_state(). Fungsi juga menyiapkan riwayat fitness dan data tambahan agar hasil berbagai algoritma dapat ditampilkan melalui alur GUI yang sama.
experiment_plan(name, parameters, generation_text, population_text)	Menyusun daftar pelaksanaan eksperimen. Untuk algoritma selain GA, fungsi membuat tiga pengulangan dengan parameter yang sama. Untuk GA, fungsi menyusun 18 run: tiga variasi generasi dengan populasi tetap dan tiga variasi populasi dengan generasi tetap, masing-masing diulang tiga kali. Setiap entri memuat kelompok eksperimen, nomor pengulangan, dan konfigurasi parameter.
variations(text)	Fungsi lokal di dalam experiment_plan() yang mengubah input variasi GA berupa teks dipisahkan koma menjadi daftar bilangan bulat. Fungsi memastikan terdapat tepat tiga nilai yang berbeda dan positif, kemudian mengembalikan daftar tersebut.

create_experiment_folder(name)	Membuat folder eksperimen di dalam output, dikelompokkan berdasarkan nama fungsi algoritma. Nama folder memuat tanggal dan waktu pelaksanaan dalam WIB. Jika nama folder sudah digunakan, fungsi menambahkan nomor pembeda agar hasil sebelumnya tidak tertimpa. Mengembalikan lokasi folder eksperimen.
run_output_folder(batch_path, name, group, repeat, parameters)	Menentukan lokasi penyimpanan untuk satu run. Algoritma selain GA menggunakan subfolder berdasarkan nomor pengulangan. Untuk GA, lokasi juga memuat kelompok eksperimen serta kombinasi jumlah populasi dan generasi. Fungsi mengembalikan path folder tanpa membuatnya secara langsung.
result_figure(result)	Membuat grafik hasil pencarian menggunakan Matplotlib. Hill-Climbing dan SA menampilkan objective function terhadap iterasi, sedangkan Random Restart menampilkan garis terpisah untuk setiap restart. GA menampilkan fitness maksimum dan rata-rata populasi terhadap generasi. Untuk SA, fungsi menambahkan grafik nilai eksponensial terhadap iterasi dengan skala symlog; nilai yang tidak tersedia atau tidak berhingga ditampilkan sebagai celah.
total_shipping_fee(state)	Menjumlahkan atribut ShippingFee seluruh kendaraan pada placed_vehicles. Nilai yang dikembalikan merupakan total tarif pengiriman kendaraan yang dimuat, tanpa menghitung kendaraan di luar kapal dan tanpa melakukan pemeriksaan validitas state.
state_description(state)	Mengubah data state menjadi teks yang mengelompokkan kendaraan di dalam dan di luar kapal. Deskripsi memuat jumlah kendaraan, ID, posisi untuk

	kendaraan yang dimuat, orientasi, dimensi, berat, dan ShippingFee. Teks ini digunakan dalam bagian hasil eksperimen.
state_figure(result, ship)	Membuat gambar Matplotlib berisi susunan awal dan akhir secara berdampingan untuk disimpan sebagai file. Geladak digambarkan sebagai grid sesuai dimensi kapal, sedangkan kendaraan berupa persegi panjang dengan ID di tengah. Gambar juga mencantumkan total ShippingFee, nilai objective, dan daftar ID kendaraan di luar kapal.

2.5 Data.json


```
{
  "Ship": {
    "Dimensions": {
      "w": 10,
      "l": 20
    },
    "MaxCapacity": 500
  },

  "Vehicles": [
    {
      "id": "1",
      "Dimensions": {
        "w": 3,
        "l": 4
      },
      "Orientation": "Horizontal",
      "ShippingFee": 60,
      "Weight": 5,
      "ETA": 3
    },
    {
      "id": "2",
      "Dimensions": {
        "w": 2,
        "l": 3
      },
      "Orientation": "Vertical",
      "ShippingFee": 40,
      "Weight": 3,
      "ETA": 1
    },
    {
      "id": "3",
```

```
    "Dimensions": {
      "w": 4,
      "l": 5
    },
    "Orientation": "Horizontal",
    "ShippingFee": 85,
    "Weight": 8,
    "ETA": 2
  },
  {
    "id": "4",
    "Dimensions": {
      "w": 2,
      "l": 4
    },
    "Orientation": "Vertical",
    "ShippingFee": 50,
    "Weight": 4,
    "ETA": 5
  },
  {
    "id": "5",
    "Dimensions": {
      "w": 3,
      "l": 6
    },
    "Orientation": "Horizontal",
    "ShippingFee": 95,
    "Weight": 10,
    "ETA": 3
  },
  {
    "id": "6",
    "Dimensions": {
      "w": 1,
      "l": 2
    },
    "Orientation": "Vertical",
    "ShippingFee": 20,
    "Weight": 2,
    "ETA": 1
  },
  {
    "id": "7",
    "Dimensions": {
      "w": 3,
      "l": 5
    },
    "Orientation": "Horizontal",
    "ShippingFee": 70,
    "Weight": 6,
    "ETA": 4
  },
  {
    "id": "8",
    "Dimensions": {
      "w": 2,
      "l": 2
    },
    "Orientation": "Horizontal",
    "ShippingFee": 30,
```

```
      "Weight": 3,
      "ETA": 2
    },
    {
      "id": "9",
      "Dimensions": {
        "w": 4,
        "l": 4
      },
      "Orientation": "Vertical",
      "ShippingFee": 75,
      "Weight": 7,
      "ETA": 6
    },
    {
      "id": "10",
      "Dimensions": {
        "w": 2,
        "l": 5
      },
      "Orientation": "Horizontal",
      "ShippingFee": 65,
      "Weight": 5,
      "ETA": 3
    }
  ]
}
```

Bab 3: Implementasi Algoritma

3.1 Algoritma Hill Climbing: Steepest Ascent

Algoritma Hill Climbing Steepest Ascent merupakan algoritma pencarian lokal yang dimulai dari suatu initial state yang dibangkitkan secara acak. Pada setiap iterasi, algoritma mengevaluasi seluruh successor atau solusi tetangga dari current state, kemudian memilih tetangga dengan nilai objektif tertinggi (highest-valued successor) sebagai solusi berikutnya. Proses ini terus berulang selama ditemukan tetangga yang memiliki nilai lebih baik. Algoritma berhenti ketika tidak ada tetangga dengan nilai lebih tinggi daripada solusi saat ini, sehingga menghasilkan local maximum, termasuk kondisi ketika beberapa tetangga memiliki nilai yang sama (plateau).

Algoritma ini digunakan untuk mencari konfigurasi penempatan kendaraan yang menghasilkan total ShippingFee setinggi mungkin. Setiap state merepresentasikan konfigurasi kendaraan yang dimuat, sedangkan neighbor merupakan konfigurasi alternatif yang dihasilkan melalui perubahan penempatan kendaraan. Setiap kandidat tetangga dievaluasi menggunakan objective function dengan mempertimbangkan validitas penempatan dan total ShippingFee. Algoritma kemudian memilih tetangga dengan nilai objektif tertinggi dan berpindah ke konfigurasi tersebut apabila nilainya lebih baik daripada current state, hingga tidak ditemukan konfigurasi tetangga yang dapat meningkatkan nilai objektif. Proses ini memungkinkan pencarian konfigurasi pemuatan yang menguntungkan, meskipun hasil akhirnya belum tentu merupakan solusi optimal secara global.

Tabel 3.1 Deskripsi Hill Climbing: Steepest Ascent

Fungsi	**Deskripsi**
steepest_ascent(vehicles, ship, max_iterations=300)	Steepest Ascent Hill Climbing merupakan algoritma pencarian lokal yang bertujuan untuk memaksimalkan nilai fitness dengan mengevaluasi seluruh neighbor dari current state, kemudian memilih neighbor dengan nilai fitness tertinggi. Algoritma

	dimulai dengan membangkitkan initial state secara acak, kemudian mengevaluasi kualitas solusi tersebut menggunakan fungsi evaluasi. Pada setiap iterasi, algoritma mencari neighbor terbaik melalui fungsi best_neighbor(). Jika nilai fitness neighbor terbaik lebih tinggi daripada current state, algoritma berpindah ke solusi tersebut dan memperbarui nilai fitness. Sebaliknya, jika tidak ditemukan neighbor dengan nilai fitness yang lebih tinggi, pencarian dihentikan. Proses ini berulang hingga tidak ditemukan peningkatan atau jumlah iterasi maksimum tercapai. Algoritma kemudian mengembalikan solusi terbaik yang ditemukan beserta nilai fitness awal dan akhir, jumlah iterasi, durasi eksekusi, serta konfigurasi kendaraan yang berhasil dimuat. Namun, algoritma ini berpotensi berhenti pada local optimum, yaitu solusi terbaik di antara tetangga-tetangganya, tetapi belum tentu merupakan solusi optimal global.

Source Code

```
import time
import copy
import os
import sys
import random

sys.path.insert(
    0,
    os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
)

from evaluation import (
    load_data,
    generate_random_state,
    evaluate_state,
    best_neighbor,
```

```
    decode_state,
)


def steepest_ascent(vehicles, ship, max_iterations=300):

    # 1. inisialisasi random generator
    random.seed()

    start_time = time.time()

    # 2. generate initial state
    initial_state = generate_random_state(
        vehicles,
        ship,
        p_inside=0.5
    )

    initial_fitness = evaluate_state(
        initial_state,
        vehicles,
        ship
    )

    current_state = copy.deepcopy(initial_state)
    current_fitness = initial_fitness

    best_state = copy.deepcopy(current_state)
    best_fitness = current_fitness

    fitness_list = [best_fitness]

    print("=" * 60)
    print("STEEPEST ASCENT HILL CLIMBING")
    print("=" * 60)
    print(f"Initial fitness: {initial_fitness}")
    print(f"Max iterations : {max_iterations}")

    # 3. cari neighbor terbaik pada setiap iterasi
    iterations = 0
```

```
for iteration in range(1, max_iterations + 1):

    neighbor_state, neighbor_fitness = best_neighbor(
        current_state,
        vehicles,
        ship
    )

    iterations = iteration

    print(f"\nIteration       : {iteration}")
    print(f"Current fitness : {current_fitness}")
    print(f"Best neighbor   : {neighbor_fitness}")

    # 4. berhenti jika tidak ada peningkatan
    if neighbor_fitness <= current_fitness:
        print("Decision        : STOP, no improvement")
        break

    # 5. pindah ke neighbor dengan fitness lebih tinggi
    current_state = copy.deepcopy(neighbor_state)
    current_fitness = neighbor_fitness

    if current_fitness > best_fitness:
        best_state = copy.deepcopy(current_state)
        best_fitness = current_fitness

    fitness_list.append(best_fitness)

    print("Decision        : MOVE to best neighbor")

# 6. hitung durasi eksekusi
duration = time.time() - start_time

# 7. decode state awal dan akhir
decoded_initial_state = decode_state(
    initial_state,
    vehicles
)

decoded_final_state = decode_state(
```

```
        best_state,
        vehicles
    )

    print("\n" + "=" * 60)
    print("HASIL AKHIR")
    print("=" * 60)
    print(f"Initial fitness : {initial_fitness}")
    print(f"Final fitness   : {best_fitness}")
    print(f"Iterations      : {iterations}")
    print(f"Duration        : {duration:.6f} seconds")
    print(
        f"Vehicles loaded : "
        f"{len(decoded_final_state['placed_vehicles'])}"
    )
    print(
        f"Vehicles outside: "
        f"{len(decoded_final_state['outside_vehicles'])}"
    )

    print("\nKendaraan yang dimuat:")

    for vehicle in decoded_final_state["placed_vehicles"]:
        vehicle_name = vehicle.get(
            "Name",
            vehicle.get("ID", "Vehicle")
        )

        print(
            f"- {vehicle_name}: "
            f"x={vehicle['x']}, "
            f"y={vehicle['y']}, "
            f"orientation={vehicle['Orientation']}, "
            f"fee={vehicle['ShippingFee']}"
        )

    # 8. return hasil
    return {
        "initial_state": decoded_initial_state,
        "initial_fitness": initial_fitness,
        "final_state": decoded_final_state,
```

```
        "final_fitness": best_fitness,
        "fitness_list": fitness_list,
        "duration": duration,
        "max_iteration": max_iterations,
        "iterations": iterations,
    }
```

3.2 Algoritma Hill Climbing: Sideways Move

Hill-climbing with Sideways Move adalah varian hill-climbing untuk mengatasi masalah area datar. Hill-climbing biasa (steepest ascent) berhenti di tiga kondisi selain global maximum: local maximum, flat local maximum, dan shoulder. Shoulder sebenarnya dataran yang masih bisa naik lagi menuju global maximum. Masalahnya, hill-climbing biasa langsung berhenti begitu tidak ada neighbor yang lebih baik, termasuk saat neighbor terbaiknya bernilai sama. Akibatnya ia terjebak di dataran padahal jalan ke puncak masih ada. Sideways move mengatasinya dengan mengizinkan perpindahan ke neighbor yang nilainya sama (bergerak "menyamping"). Dengan begitu pencarian bisa menyeberangi shoulder dan terus naik.

Perubahan pada algoritmanya sangat kecil, hanya pada kondisi berhenti:

- HC biasa: if neighbor.VALUE ≤ current.VALUE then return current, artinya berhenti di puncak termasuk area datar.
- HC + sideways move: if neighbor.VALUE < current.VALUE then return current, artinya hanya berhenti di puncak sungguhan, dan neighbor yang nilainya sama tetap diambil.
- Batas langkah sideways berturut-turut diperlukan supaya algoritma tidak berputar tanpa henti di flat local maximum.

Tabel 3.2 Deskripsi Hill Climbing: Sideways Move

Fungsi	**Deskripsi**
hc_sideways(vehicles, ship, max_sideways, max_iterasi)	Mencari susunan kendaraan yang memaksimalkan total ShippingFee menggunakan Hill-Climbing dengan perpindahan sideways. Fungsi membentuk state awal yang valid dengan mencoba memasukkan kendaraan dalam urutan acak dan membatalkan pemasukan yang melanggar constraints. Pada setiap iterasi, fungsi membangkitkan kumpulan neighbor

	menggunakan all_neighbors(), mengabaikan kandidat yang identik dengan current atau bernilai -1, lalu memilih kandidat dengan fitness tertinggi. Jika terdapat beberapa kandidat terbaik, salah satunya dipilih secara acak. Neighbor dengan fitness lebih tinggi diterima dan hitungan sideways berturut-turut direset. Neighbor dengan fitness sama diterima selama jumlah sideways berturut-turut belum mencapai max_sideways. Pencarian berhenti ketika tidak ada kandidat valid, neighbor terbaik memiliki fitness lebih rendah, batas sideways tercapai saat akan melakukan perpindahan bernilai sama, atau batas iterasi tercapai. Fungsi mengembalikan state akhir dan fitnessnya, state awal dan fitnessnya, riwayat iterasi, jumlah iterasi, total perpindahan sideways, dan durasi pencarian.

Source Code

```
import copy, random, time

from evaluation import generate_random_state, all_neighbors, evaluate_state

def hc_sideways(vehicles, ship, max_sideways, max_iterasi):

    start_time = time.perf_counter()

    current = generate_random_state(vehicles, ship, p_inside = 0)

    urutan_kendaraan=list(range(len(vehicles)))
    random.shuffle(urutan_kendaraan)

    for i in urutan_kendaraan:
        current[i]["inside"] = True

        if evaluate_state(current, vehicles, ship) == -1:
            current[i]["inside"] = False
```

```
    current_value = evaluate_state(current, vehicles, ship)

    initial = copy.deepcopy(current)
    initial_value = current_value

    sideways_count = 0
    sideways_total = 0
    history_iterasi = []
    iterasi_count = 0

    for iterasi in range(max_iterasi):
        iterasi_count +=1

        neighbors = all_neighbors(current, vehicles, ship)

        best_value = float("-inf")
        best_kandidat = []

        for kandidat in neighbors:
            if kandidat == current:
                continue

            kandidat_value = evaluate_state(kandidat, vehicles, ship)

            if kandidat_value == -1:
                continue

            if kandidat_value > best_value:
                best_value = kandidat_value
                best_kandidat = [kandidat]

            elif kandidat_value == best_value:
                best_kandidat.append(kandidat)

        if not best_kandidat:
            break

        neighbor = random.choice(best_kandidat)
        neighbor_value = evaluate_state(neighbor, vehicles, ship)

        if neighbor_value > current_value:
```

```
            current = neighbor
            current_value = neighbor_value
            sideways_count = 0

        elif neighbor_value == current_value:
            if sideways_count >= max_sideways:
                break

            current = neighbor
            current_value = neighbor_value
            sideways_count += 1
            sideways_total += 1

        if neighbor_value < current_value:
            break

        history_iterasi.append((iterasi + 1, current_value, sideways_count))

    return(current, current_value, initial, initial_value, history_iterasi,
iterasi_count, sideways_total, time.perf_counter() - start_time)
```

3.3 Algoritma Hill Climbing: Random Restart

Algoritma Random Restart Hill Climbing merupakan variasi dari Hill Climbing yang menjalankan serangkaian pencarian dengan initial state yang dibangkitkan secara acak pada setiap pengulangan. Jika suatu pencarian berhenti pada local maximum yang belum memenuhi tujuan, algoritma memulai kembali pencarian dari initial state yang baru. Proses ini dilakukan berulang kali hingga ditemukan solusi yang memenuhi tujuan, sehingga meningkatkan peluang menemukan solusi yang lebih baik dibandingkan hanya menjalankan satu kali pencarian.

Algoritma ini digunakan untuk mencari konfigurasi penempatan kendaraan yang menghasilkan total ShippingFee setinggi mungkin. Setiap pencarian dimulai dari konfigurasi awal kendaraan yang dibangkitkan secara acak, kemudian menggunakan mekanisme Hill Climbing Steepest Ascent untuk mengevaluasi dan memilih konfigurasi tetangga dengan nilai objektif tertinggi. Jika pencarian berhenti tanpa menghasilkan solusi yang lebih baik, algoritma memulai pencarian baru dari konfigurasi awal yang berbeda. Dengan mengulangi proses tersebut dan membandingkan hasil setiap pencarian, algoritma dapat memperoleh konfigurasi pemuatan dengan total ShippingFee yang lebih tinggi, meskipun tidak menjamin ditemukannya solusi optimal secara global.

Tabel 3.3 Deskripsi Hill Climbing: Random Restart

Fungsi	**Deskripsi**
random_restart(vehicles, ship, max_restarts=10, max_iterations=300)	Random Restart Hill Climbing merupakan pengembangan algoritma Hill Climbing yang menjalankan proses pencarian berulang kali dengan membangkitkan initial state baru secara acak pada setiap pengulangan (restart). Pada setiap restart, algoritma menerapkan mekanisme Steepest Ascent Hill Climbing, yaitu mencari neighbor dengan nilai fitness tertinggi dan berpindah ke solusi tersebut apabila nilai fitness-nya lebih tinggi daripada current state. Pencarian pada setiap restart dihentikan

	ketika tidak ditemukan peningkatan nilai fitness atau jumlah iterasi maksimum tercapai. Setelah satu proses pencarian selesai, algoritma menyimpan solusi terbaik dari restart tersebut, kemudian membandingkannya dengan solusi terbaik yang telah ditemukan pada restart sebelumnya. Proses ini diulang hingga jumlah restart maksimum tercapai, yaitu sebanyak 10 kali secara default. Pada akhir proses, algoritma mengembalikan solusi dengan nilai fitness tertinggi dari seluruh restart, beserta informasi nilai fitness awal dan akhir, jumlah restart, total iterasi, durasi eksekusi, serta konfigurasi kendaraan yang berhasil dimuat. Dengan memulai pencarian dari beberapa kondisi awal yang berbeda, Random Restart Hill Climbing dapat meningkatkan peluang menemukan solusi yang lebih baik dan mengurangi risiko terjebak pada local optimum dibandingkan dengan hanya menjalankan Hill Climbing satu kali.

Source Code

```
import time
import copy
import os
import sys
import random

sys.path.insert(
    0,
    os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
)

from evaluation import (
    load_data,
    generate_random_state,
    evaluate_state,
```

```
    best_neighbor,
    decode_state,
)


def random_restart(vehicles, ship, max_restarts=10, max_iterations=300):

    # 1. inisialisasi random generator
    random.seed()

    start_time = time.time()

    best_initial_state = None
    best_initial_fitness = None
    best_state = None
    best_fitness = None

    fitness_list = []
    restart_fitness_list = []
    iterations_per_restart = []
    total_iterations = 0

    print("=" * 60)
    print("RANDOM RESTART HILL CLIMBING")
    print("=" * 60)
    print(f"Max restarts   : {max_restarts}")
    print(f"Max iterations : {max_iterations}")

    # 2. ulangi pencarian dari state awal yang baru
    for restart in range(1, max_restarts + 1):

        print("\n" + "=" * 60)
        print(f"RESTART {restart}")
        print("=" * 60)

        # 3. generate initial state untuk restart ini
        initial_state = generate_random_state(
            vehicles,
            ship,
            p_inside=0.5
        )
```

```
initial_fitness = evaluate_state(
    initial_state,
    vehicles,
    ship
)

current_state = copy.deepcopy(initial_state)
current_fitness = initial_fitness

run_best_state = copy.deepcopy(current_state)
run_best_fitness = current_fitness

print(f"Initial fitness: {initial_fitness}")

# 4. cari neighbor terbaik pada setiap iterasi
iterations = 0

for iteration in range(1, max_iterations + 1):

    neighbor_state, neighbor_fitness = best_neighbor(
        current_state,
        vehicles,
        ship
    )

    iterations = iteration

    print(f"\nIteration       : {iteration}")
    print(f"Current fitness : {current_fitness}")
    print(f"Best neighbor   : {neighbor_fitness}")

    # 5. berhenti jika tidak ada peningkatan
    if neighbor_fitness <= current_fitness:
        print("Decision        : STOP, no improvement")
        break

    # 6. pindah ke neighbor dengan fitness lebih tinggi
    current_state = copy.deepcopy(neighbor_state)
    current_fitness = neighbor_fitness
```

```
            if current_fitness > run_best_fitness:
                run_best_state = copy.deepcopy(current_state)
                run_best_fitness = current_fitness

            print("Decision        : MOVE to best neighbor")

        total_iterations += iterations
        iterations_per_restart.append(iterations)
        restart_fitness_list.append(run_best_fitness)

        print(f"\nRestart {restart} selesai")
        print(f"Final fitness   : {run_best_fitness}")
        print(f"Iterations      : {iterations}")

        # 7. simpan hasil terbaik dari semua restart
        if best_fitness is None or run_best_fitness > best_fitness:
            best_state = copy.deepcopy(run_best_state)
            best_fitness = run_best_fitness
            best_initial_state = copy.deepcopy(initial_state)
            best_initial_fitness = initial_fitness
            print("Decision        : NEW BEST across restarts")

        fitness_list.append(best_fitness)

    # 8. hitung durasi eksekusi
    duration = time.time() - start_time

    # 9. decode state awal dan akhir
    decoded_initial_state = decode_state(
        best_initial_state,
        vehicles
    )

    decoded_final_state = decode_state(
        best_state,
        vehicles
    )

    print("\n" + "=" * 60)
    print("HASIL AKHIR")
    print("=" * 60)
```

```
    print(f"Initial fitness : {best_initial_fitness}")
    print(f"Final fitness   : {best_fitness}")
    print(f"Restarts        : {max_restarts}")
    print(f"Iterations      : {total_iterations}")
    print(f"Duration        : {duration:.6f} seconds")
    print(
        f"Vehicles loaded : "
        f"{len(decoded_final_state['placed_vehicles'])}"
    )
    print(
        f"Vehicles outside: "
        f"{len(decoded_final_state['outside_vehicles'])}"
    )

    print("\nKendaraan yang dimuat:")

    for vehicle in decoded_final_state["placed_vehicles"]:
        vehicle_name = vehicle.get(
            "Name",
            vehicle.get("ID", "Vehicle")
        )

        print(
            f"* {vehicle_name}: "
            f"x={vehicle['x']}, "
            f"y={vehicle['y']}, "
            f"orientation={vehicle['Orientation']}, "
            f"fee={vehicle['ShippingFee']}"
        )

    # 10. return hasil
    return {
        "initial_state": decoded_initial_state,
        "initial_fitness": best_initial_fitness,
        "final_state": decoded_final_state,
        "final_fitness": best_fitness,
        "fitness_list": fitness_list,
        "restart_fitness_list": restart_fitness_list,
        "iterations_per_restart": iterations_per_restart,
        "duration": duration,
        "max_restart": max_restarts,
```

```
        "max_iteration": max_iterations,
        "iterations": total_iterations,
    }
```

3.4 Algoritma Hill Climbing: Stochastic

Algoritma Stochastic Hill Climbing merupakan variasi dari Hill Climbing yang memilih satu successor atau solusi tetangga secara acak pada setiap iterasi, tanpa mengevaluasi seluruh tetangga seperti pada Steepest Ascent. Jika neighbor yang terpilih memiliki nilai objektif lebih tinggi daripada current state, algoritma berpindah ke neighbor tersebut. Proses ini dilakukan berulang kali hingga mencapai batas iterasi maksimum (nmax) atau kondisi penghentian lainnya. Karena hanya mengevaluasi satu tetangga secara acak pada setiap iterasi, algoritma ini cenderung membutuhkan lebih banyak langkah dan waktu pencarian dapat lebih lama.

Algoritma ini digunakan untuk mencari konfigurasi penempatan kendaraan yang memaksimalkan total ShippingFee. Setiap state merepresentasikan konfigurasi kendaraan yang dimuat, sedangkan neighbor merupakan konfigurasi alternatif yang dipilih secara acak. Nilai objektif neighbor dibandingkan dengan current state, dan perpindahan dilakukan hanya jika nilai objektif neighbor lebih tinggi. Proses ini berlanjut hingga mencapai batas iterasi maksimum atau tidak ada lagi perpindahan yang dilakukan sesuai mekanisme algoritma. Pendekatan ini memungkinkan eksplorasi konfigurasi yang lebih bervariasi, tetapi karena pemilihan tetangga dilakukan secara acak dan tidak semua tetangga dievaluasi, solusi yang diperoleh belum tentu merupakan solusi optimal secara global.

Tabel 3.4 Deskripsi Hill Climbing: Stochastic

Fungsi	**Deskripsi**
stochastic(vehicles, ship, max_iterations=300)	Stochastic Hill Climbing merupakan algoritma pencarian lokal yang memilih satu neighbor secara acak pada setiap iterasi untuk dievaluasi, tanpa harus mengevaluasi seluruh tetangga dari current state. Algoritma dimulai dengan membangkitkan initial state secara acak, kemudian mengevaluasi kualitas solusi tersebut menggunakan fungsi evaluasi. Pada setiap

	iterasi, algoritma memilih satu neighbor secara acak melalui fungsi random_neighbor() dan menghitung nilai fitness-nya. Jika nilai fitness neighbor lebih tinggi daripada current state, algoritma berpindah ke solusi tersebut dan memperbarui nilai fitness. Sebaliknya, jika nilai fitness neighbor lebih rendah atau sama dengan current state, algoritma tetap berada pada solusi saat ini dan melanjutkan pencarian ke iterasi berikutnya. Proses ini berulang hingga jumlah iterasi maksimum tercapai, yaitu sebanyak 300 iterasi secara default. Selama pencarian berlangsung, algoritma menyimpan solusi terbaik yang ditemukan beserta nilai fitness-nya. Pada akhir proses, algoritma mengembalikan solusi terbaik tersebut beserta informasi nilai fitness awal dan akhir, jumlah iterasi, durasi eksekusi, serta konfigurasi kendaraan yang berhasil dimuat. Dengan pemilihan neighbor secara acak, Stochastic Hill Climbing memiliki mekanisme pencarian yang lebih sederhana karena hanya mengevaluasi satu tetangga pada setiap iterasi. Namun, karena pemilihan tetangga dilakukan secara acak dan perpindahan hanya diterima jika menghasilkan peningkatan nilai fitness, algoritma ini tetap berpotensi terjebak pada local optimum.

Source Code

```
import time
import copy
import os
import sys
import random
```

```
sys.path.insert(
    0,
    os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
)

from evaluation import (
    load_data,
    generate_random_state,
    evaluate_state,
    random_neighbor,
    decode_state,
)


def stochastic(vehicles, ship, max_iterations=300):

    # 1. inisialisasi random generator
    random.seed()

    start_time = time.time()

    # 2. generate initial state
    initial_state = generate_random_state(
        vehicles,
        ship,
        p_inside=0.5
    )

    initial_fitness = evaluate_state(
        initial_state,
        vehicles,
        ship
    )

    current_state = copy.deepcopy(initial_state)
    current_fitness = initial_fitness

    best_state = copy.deepcopy(current_state)
    best_fitness = current_fitness
```

```
    fitness_list = [best_fitness]

    print("=" * 60)
    print("STOCHASTIC HILL CLIMBING")
    print("=" * 60)
    print(f"Initial fitness: {initial_fitness}")
    print(f"Max iterations : {max_iterations}")

    # 3. ambil satu neighbor acak pada setiap iterasi
    iterations = 0

    for iteration in range(1, max_iterations + 1):

        neighbor_state = random_neighbor(
            current_state,
            vehicles,
            ship
        )

        neighbor_fitness = evaluate_state(
            neighbor_state,
            vehicles,
            ship
        )

        iterations = iteration

        print(f"\nIteration       : {iteration}")
        print(f"Current fitness : {current_fitness}")
        print(f"Random neighbor : {neighbor_fitness}")

        # 4. tetap di state sekarang jika tidak ada peningkatan
        if neighbor_fitness <= current_fitness:
            print("Decision        : STAY, no improvement")
            fitness_list.append(best_fitness)
            continue

        # 5. pindah ke neighbor dengan fitness lebih tinggi
        current_state = copy.deepcopy(neighbor_state)
        current_fitness = neighbor_fitness
```

```
        if current_fitness > best_fitness:
            best_state = copy.deepcopy(current_state)
            best_fitness = current_fitness

        fitness_list.append(best_fitness)

        print("Decision        : MOVE to random neighbor")

    # 6. hitung durasi eksekusi
    duration = time.time() - start_time

    # 7. decode state awal dan akhir
    decoded_initial_state = decode_state(
        initial_state,
        vehicles
    )

    decoded_final_state = decode_state(
        best_state,
        vehicles
    )

    print("\n" + "=" * 60)
    print("HASIL AKHIR")
    print("=" * 60)
    print(f"Initial fitness : {initial_fitness}")
    print(f"Final fitness   : {best_fitness}")
    print(f"Iterations      : {iterations}")
    print(f"Duration        : {duration:.6f} seconds")
    print(
        f"Vehicles loaded : "
        f"{len(decoded_final_state['placed_vehicles'])}"
    )
    print(
        f"Vehicles outside: "
        f"{len(decoded_final_state['outside_vehicles'])}"
    )

    print("\nKendaraan yang dimuat:")

    for vehicle in decoded_final_state["placed_vehicles"]:
```

```
        vehicle_name = vehicle.get(
            "Name",
            vehicle.get("ID", "Vehicle")
        )

        print(
            f"* {vehicle_name}: "
            f"x={vehicle['x']}, "
            f"y={vehicle['y']}, "
            f"orientation={vehicle['Orientation']}, "
            f"fee={vehicle['ShippingFee']}"
        )

    # 8. return hasil
    return {
        "initial_state": decoded_initial_state,
        "initial_fitness": initial_fitness,
        "final_state": decoded_final_state,
        "final_fitness": best_fitness,
        "fitness_list": fitness_list,
        "duration": duration,
        "max_iteration": max_iterations,
        "iterations": iterations,
    }
```

3.5 Algoritma Simulated Annealing

Simulated Annealing (SA) adalah metode local search yang menggabungkan kelebihan hill-climbing dan random walk (metode pencarian solusi di mana langkah berikutnya dipilih secara acak dari solusi tetangga yang tersedia, tanpa memedulikan apakah solusi baru tersebut lebih baik atau lebih buruk dari solusi saat ini. Hill-climbing efisien tetapi tidak lengkap karena tidak pernah mengambil langkah "turun" dan bisa terjebak di local maximum. Random walk sebaliknya, yaitu lengkap tetapi sangat tidak efisien. SA mengambil jalan tengah dengan mengizinkan beberapa langkah "buruk" agar bisa keluar dari local maximum, lalu mengurangi frekuensinya secara bertahap. Karena itu, SA bisa dilihat sebagai versi stochastic hill-climbing yang membolehkan sebagian langkah menurun. Di setiap iterasi, SA memilih satu successor acak dari state saat ini. Untuk 8-queen, ada 56 kemungkinan successor. Lalu dihitung ΔE = neighbor.value – current.value, dan keputusannya bergantung pada nilai itu:

- Jika ΔE > 0 (neighbor lebih baik): langsung pindah ke neighbor, sama seperti stochastic HC.
- Jika ΔE < 0 (neighbor lebih buruk): pindah hanya dengan probabilitas e^(ΔE/T). Probabilitas ini bisa dibandingkan dengan nilai statis (misalnya > 0,5) atau dengan random(0,1).
    - Saat T tinggi, misalnya ΔE = −5 dan T = 100, probabilitasnya ≈ 0,95. Hampir semua langkah buruk diterima, jadi perilakunya mirip random walk.
    - Saat T mendekati 0, misalnya ΔE = −5 dan T = 1, probabilitasnya ≈ 0,007. Langkah buruk hampir tidak pernah diterima, jadi perilakunya mirip stochastic HC.
- Terminasi: algoritma berhenti ketika T = 0 dan mengembalikan state saat ini.

Tabel 3.5 Deskripsi Simulated Annealing

Fungsi	**Deskripsi**
penurunan_suhu (iterasi, T_awal,	Menghitung suhu pada suatu iterasi menggunakan rumus T_awal * cooling_rate ** iterasi. Dengan cooling_rate antara 0

cooling_rate)	dan 1, suhu menurun seiring bertambahnya iterasi. Suhu ini digunakan untuk mengatur peluang penerimaan neighbor yang memiliki fitness lebih rendah.
simulated_annealing(vehicles, ship, T_awal, cooling_rate, max_iterasi, stuck_threshold)	Mencari susunan kendaraan yang memaksimalkan total ShippingFee. Mulai dengan membentuk state awal yang valid dengan mencoba memasukkan kendaraan dalam urutan acak dan membatalkan pemasukan yang melanggar constraints. Pada setiap iterasi, fungsi menghitung suhu, membangkitkan satu neighbor secara acak melalui fungsi random_neighbor(), dan mengevaluasinya menggunakan fungsi evaluate_state(). Neighbor invalid akan ditolak, sedangkan neighbor dengan fitness yang lebih rendah dapat diterima berdasarkan peluang yang dihitung dengan rumus eksponensial. Jika fitness neighbor lebih bagus atau sama akan selalu diterima. Fungsi menyimpan state terbaik yang ditemukan dan mencatat total kejadian tanpa peningkatan fitness terbaik yang mencapai stuck_threshold. Pencarian akan berakhir ketika batas iterasi tercapai. Return dari fungsi ini adalah state terbaik dan fitness-nya, state awal dan fitness-nya, riwayat iterasi, jumlah perpindahan yang diterima, frekuensi stuck, durasi, serta nilai eksponensial terakhir.

Source Code

```
import copy, math, random, time

from evaluation import evaluate_state, generate_random_state, random_neighbor,
objective_function, decode_state



def penurunan_suhu(iterasi, T_awal, cooling_rate):
    return T_awal * (cooling_rate ** iterasi)

def simulated_annealing(vehicles, ship, T_awal, cooling_rate, max_iterasti,
```

```
stuck_threshold):

    start_time = time.perf_counter()

    current = generate_random_state(vehicles, ship, p_inside=0)

    urutan_kendaraan = list(range(len(vehicles)))

    random.shuffle(urutan_kendaraan)

    # periksa apkh overlap
    for i in urutan_kendaraan:
        current[i]["inside"] = True

        if evaluate_state(current, vehicles, ship) == -1:
            current[i]["inside"] = False

    current_value = evaluate_state(current, vehicles, ship)

    initial = copy.deepcopy(current)
    initial_value = current_value

    best = copy.deepcopy(current)
    best_value = current_value

    history_iterasi = []
    no_improvement_iterasi_count = 0
    stuck_count = 0
    accepted_count = 0

    for iterasi in range(max_iterasti):
        T = penurunan_suhu(iterasi, T_awal, cooling_rate)

        if T <= 1e-12: break

        neighbor = random_neighbor(current, vehicles, ship)
        neighbor_value = evaluate_state(neighbor, vehicles, ship)

        accepted = False
        peluang = 0.0
        exp_value = None
```

```
        if neighbor_value != -1:
            try:
                exp_value = math.exp((neighbor_value - current_value) / T)
            except OverflowError:
                exp_value = None

            if neighbor_value < current_value:
                peluang = exp_value
                accepted = random.random() < peluang
            else:
                peluang = 1.0
                accepted = True

        if accepted:
            current = neighbor
            current_value = neighbor_value
            accepted_count += 1

        if current_value > best_value:
            best = copy.deepcopy(current)
            best_value = current_value
            no_improvement_iterasi_count = 0
        else:
            no_improvement_iterasi_count += 1

        if no_improvement_iterasi_count >= stuck_threshold:
            stuck_count += 1
            no_improvement_iterasi_count = 0

        history_iterasi.append((iterasi + 1, current_value, T, peluang, accepted,
exp_value))

    return best, best_value, history_iterasi, initial, initial_value,
accepted_count, stuck_count, time.perf_counter() - start_time, exp_value
```

3.6 Algoritma Genetic Algorithm (GA)

Genetic Algorithm (GA) adalah metode local search yang terinspirasi dari evolusi biologis. GA bisa dilihat sebagai pengembangan dari random restart hill-climbing. Bedanya, successor pada GA dibentuk dengan menggabungkan dua state induk (parent). Setiap state atau individu direpresentasikan sebagai string atas alfabet terbatas, sering berupa 0 dan 1, dengan satu karakter mewakili satu variabel. Kualitas tiap individu diukur dengan fitness function, dan nilai yang lebih tinggi berarti state yang lebih baik. Prosesnya dimulai dari populasi awal berisi k state yang dibangkitkan secara acak. Generasi berikutnya kemudian dibentuk melalui tiga tahap:

- Selection: induk dipilih secara acak dengan peluang sebanding dengan nilai fitness-nya (seperti roulette wheel). Contohnya, individu dengan peluang fitness score 540 akan lebih mungkin untuk dipilih dibandingkan individu dengan fitness sedangkan yang F=11 hanya ≈ 14%.
- Crossover: dua induk “dipotong” di titik acak lalu bagiannya ditukar. Misalnya 327|52411 + 247|48552 menghasilkan 32748552 dan 24752411. Pada tugas besar kali ini, satu perkawinan hanya dipilih satu anak.
- Mutation: dengan probabilitas kecil, satu karakter acak pada anak diubah, misalnya 32748552 menjadi 32741552.

Tabel 3.6 Deskripsi Genetic Algorithm

Nama fungsi	**Deskripsi**
initialize_population(population_size, vehicles, ship)	Membuat populasi awal GA berisi population_size individu. Setiap individu adalah satu state acak yang dibangkitkan dengan generate_random_state.
evaluate_population(population, vehicles, ship)	Menghitung fitness setiap individu di populasi menggunakan evaluate_state (nilai objective function). Urutan hasil sama dengan urutan individu di populasi.

roulette_selection(population, fitness_scores)	Memilih satu individu sebagai parent menggunakan roulette wheel selection: peluang terpilih sebanding dengan fitness. Karena fitness bisa bernilai −1, semua fitness terlebih dahulu digeser menjadi positif dengan rumus fitness − fitness_terkecil + 1, sehingga individu terburuk tetap memiliki peluang kecil untuk terpilih.
crossover(x, y)	Menghasilkan satu anak dari dua parent dengan one-point crossover: memilih satu titik potong secara acak, lalu menggabungkan gen parent x sebelum titik potong dengan gen parent y setelah titik potong. Hasilnya disalin dengan deepcopy agar perubahan pada anak tidak memengaruhi parent.
mutation(child, vehicles, ship)	Memutasi anak pada satu titik mutasi: memilih satu kendaraan secara acak, lalu menerapkan salah satu langkah dari spesifikasi secara acak, yaitu menukar kendaraan tersebut dengan kendaraan lain (swap_vehicles), memutar orientasinya (rotate_vehicle, hanya jika kendaraan berada di kapal), atau memindahkannya ke koordinat lain (move_vehicle). Mengubah anak secara langsung.
genetic_algorithm(vehicles, ship, population_size, max_iteration, mutation_rate)	Fungsi utama Genetic Algorithm sesuai pseudocode materi. Membuat populasi awal, lalu pada setiap iterasi membentuk populasi baru sebanyak population_size anak: dua parent dipilih dengan roulette selection, digabung dengan crossover, dan dimutasi dengan peluang mutation_rate. Populasi lama diganti seluruhnya oleh populasi baru. Proses berhenti setelah max_iteration iterasi. Selama berjalan, nilai fitness maksimum dan rata-rata setiap iterasi dicatat untuk

	plot.

Source code

```
import copy
import random
import time

from evaluation import (decode_state, evaluate_state, generate_random_state,
move_vehicle, rotate_vehicle, swap_vehicles)

GA_CONTROL_POPULATION = 50
GA_ITERATION_VARIATIONS = [100, 300, 600]
GA_CONTROL_ITERATION = 300
GA_POPULATION_VARIATIONS = [20, 50, 100]
GA_MUTATION_RATE = 0.1

def initialize_population(population_size, vehicles, ship):
    population = []
    for i in range(population_size):
        state = generate_random_state(vehicles, ship)
        population.append(state)
    return population

def evaluate_population(population, vehicles, ship):
    evaluations = []
    for individual in (population):
        evaluate = evaluate_state(individual, vehicles, ship)
        evaluations.append(evaluate)
    return evaluations

def roulette_selection(population, fitness_scores):
    min_fitness = min(fitness_scores)
    portions = []
    for f in (fitness_scores):
        portion = f - min_fitness + 1
        portions.append(portion)
    return random.choices(population, weights=portions)[0]
```

```
def crossover(x, y):
    cut_point = random.randint(1, len(x))
    child = x[:cut_point] + y[cut_point:]
    return copy.deepcopy(child)

def mutation(child, vehicles, ship):
    mutating_vehicle = random.randint(0, len(child) - 1)
    mutate = random.choice(['swap', 'rotate', 'move'])
    if mutate == 'swap':
        j = random.choice([k for k in range(len(child)) if k != mutating_vehicle])
        swap_vehicles(child, mutating_vehicle, j, vehicles, ship)
    elif mutate == 'rotate' and child[mutating_vehicle]['inside']:
        rotate_vehicle(child, mutating_vehicle, vehicles, ship)
    else:
        move_vehicle(child, mutating_vehicle, vehicles, ship)
    return child

def    genetic_algorithm(vehicles,    ship,    population_size=GA_CONTROL_POPULATION,
max_iteration=GA_CONTROL_ITERATION, mutation_rate=GA_MUTATION_RATE):
    start_time = time.perf_counter()
    population = initialize_population(population_size, vehicles, ship)
    fitness_scores = evaluate_population(population, vehicles, ship)
    initial_fitness = max(fitness_scores)
    i = fitness_scores.index(initial_fitness)
    initial_state = copy.deepcopy(population[i])
    max_fitness_list = [max(fitness_scores)]
    avg_fitness_list = [sum(fitness_scores) / len(fitness_scores)]

    for i in range(max_iteration):
        new_population = []
        for j in range(population_size):
            x = roulette_selection(population, fitness_scores)
            y = roulette_selection(population, fitness_scores)
            child = crossover(x,y)
            if random.random() < mutation_rate:
                mutation(child, vehicles, ship)
            new_population.append(child)
        population = new_population
        fitness_scores = evaluate_population(population, vehicles, ship)
        max_fitness_list.append(max(fitness_scores))
```

```
        avg_fitness_list.append(sum(fitness_scores) / len(fitness_scores))

    duration = time.perf_counter() - start_time

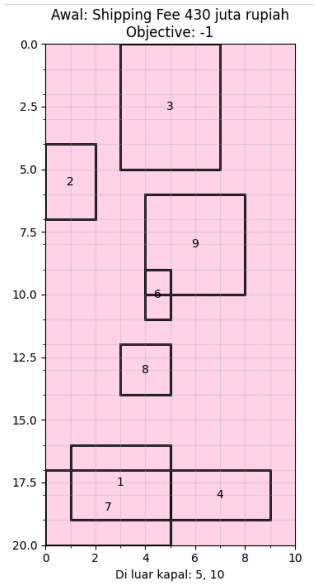
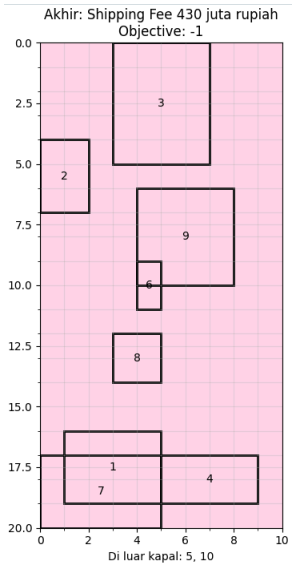
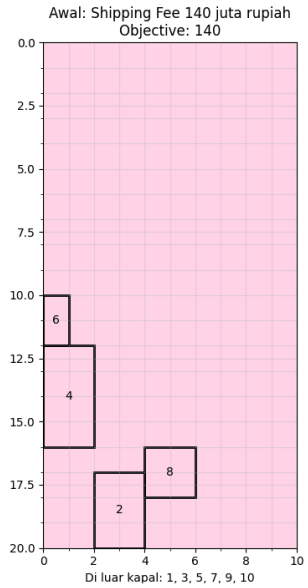
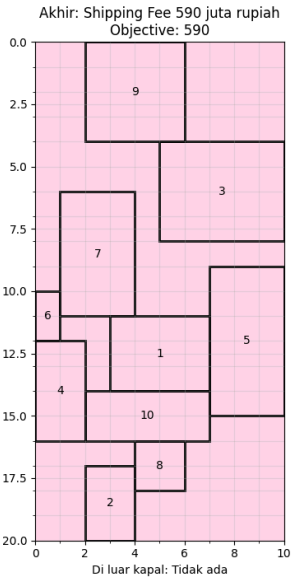
    best_fitness_scores = max(fitness_scores)
    best_index = fitness_scores.index(best_fitness_scores)
    best_individual = population[best_index]

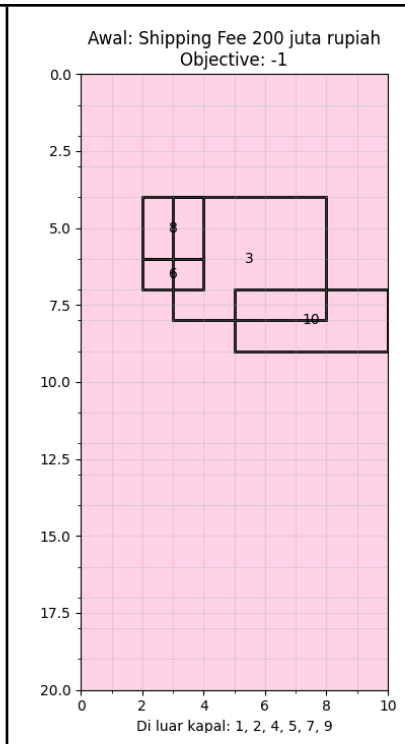
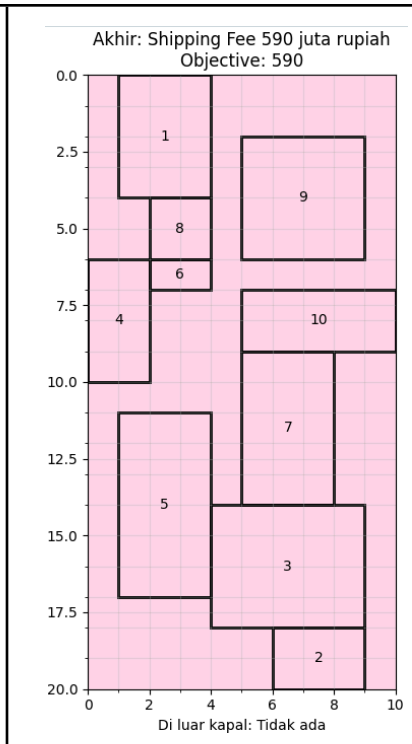
    return {
        "initial_state": decode_state(initial_state, vehicles),
        "initial_fitness": initial_fitness,
        "final_state": decode_state(best_individual, vehicles),
        "final_fitness": best_fitness_scores,
        "fitness_list": max_fitness_list,
        "duration": duration,
        "avg_fitness_list": avg_fitness_list,
        "population_size": population_size,
        "max_iteration": max_iteration,
    }
```

Bab 4: Hasil Eksperimen

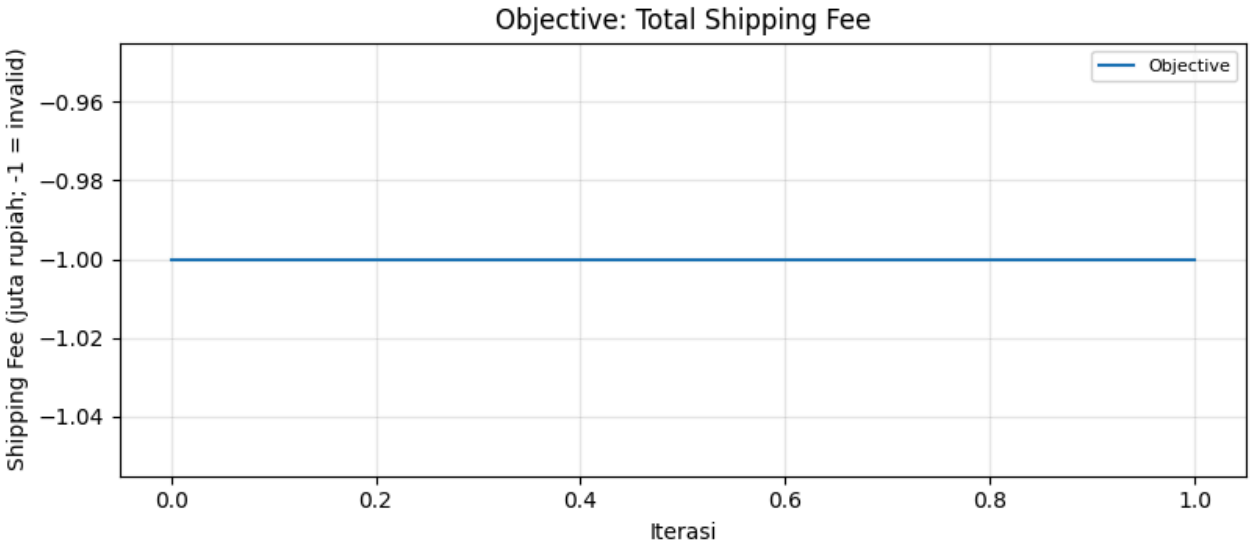
4.1 Hasil Eksperimen Hill Climbing Steepest Ascent

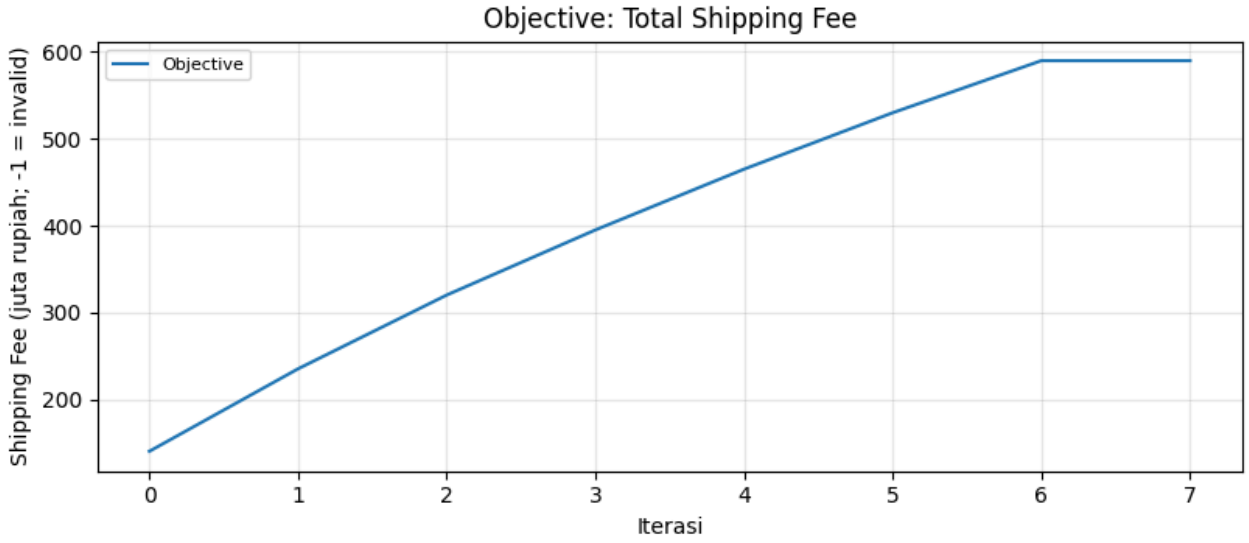
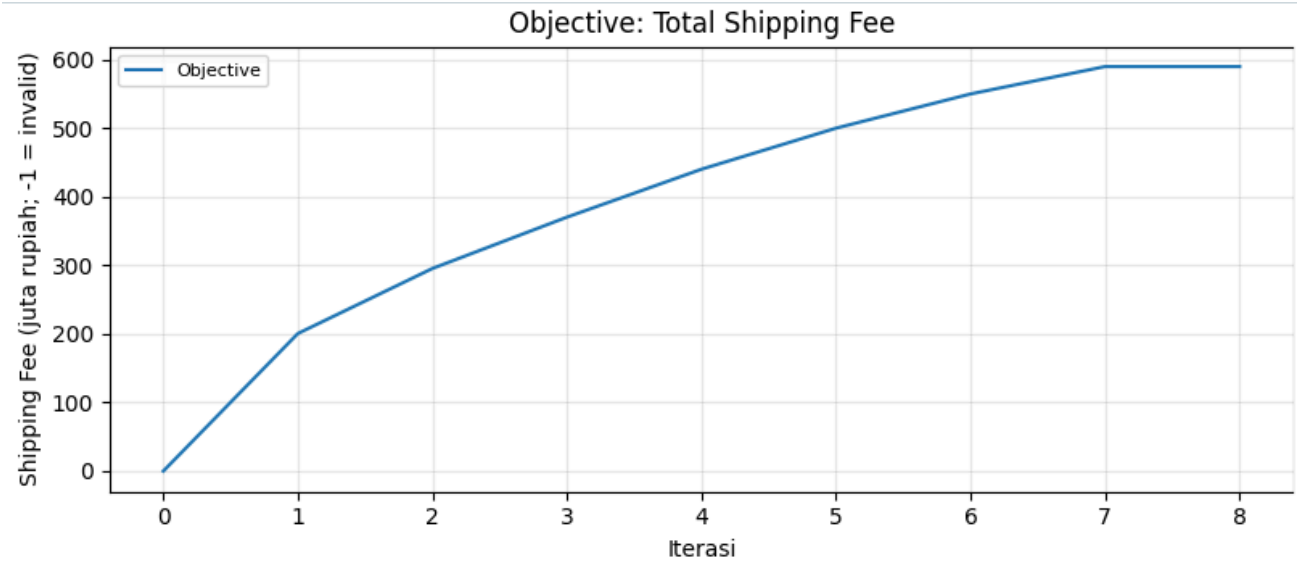
Tabel 4.1.1 Tabel Hasil Eksperimen Hill Climbing Steepest Ascent

Run	State awal	State akhir	Objective Akhir	Durasi	Total Iterasi
1			-1	0.125235s	1
2			590	0.859016s	7

3			590	0.979242s	8

Tabel 4.1.2 Plot Nilai Objective Function Algoritma Hill Climbing Steepest Ascent

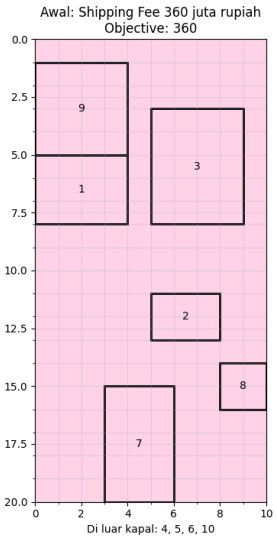
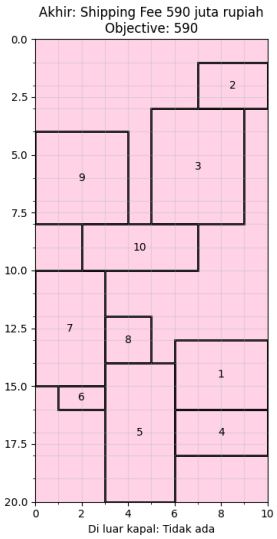
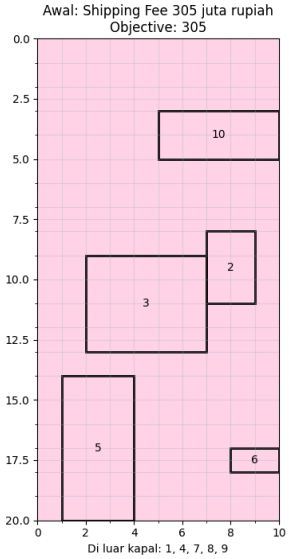
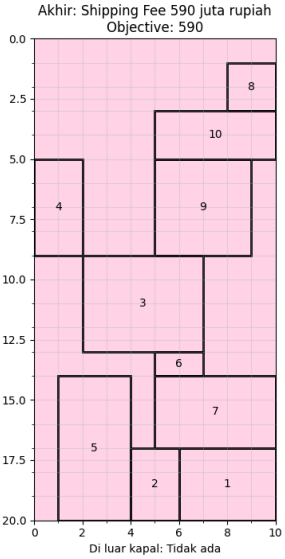
Run	Plot nilai objective function
1	

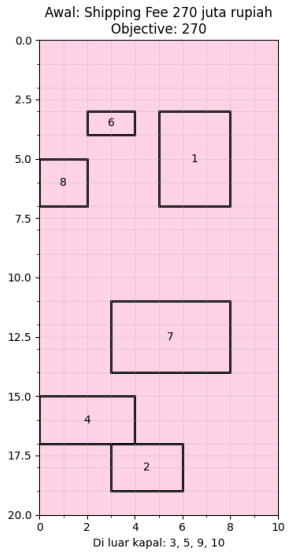
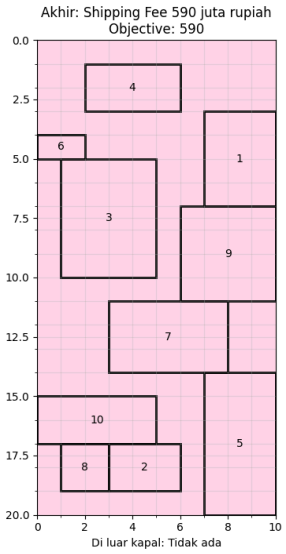
2	 Objective: Total Shipping Fee Objective Shipping Fee (juta rupiah; -1 = invalid) 600 500 400 300 200 0 1 2 3 4 5 6 7 Iterasi
3	 Objective: Total Shipping Fee Objective Shipping Fee (juta rupiah; -1 = invalid) 600 500 400 300 200 100 0 0 1 2 3 4 5 6 7 8 Iterasi

4.2 Hasil Eksperimen Hill Climbing Sideways Move

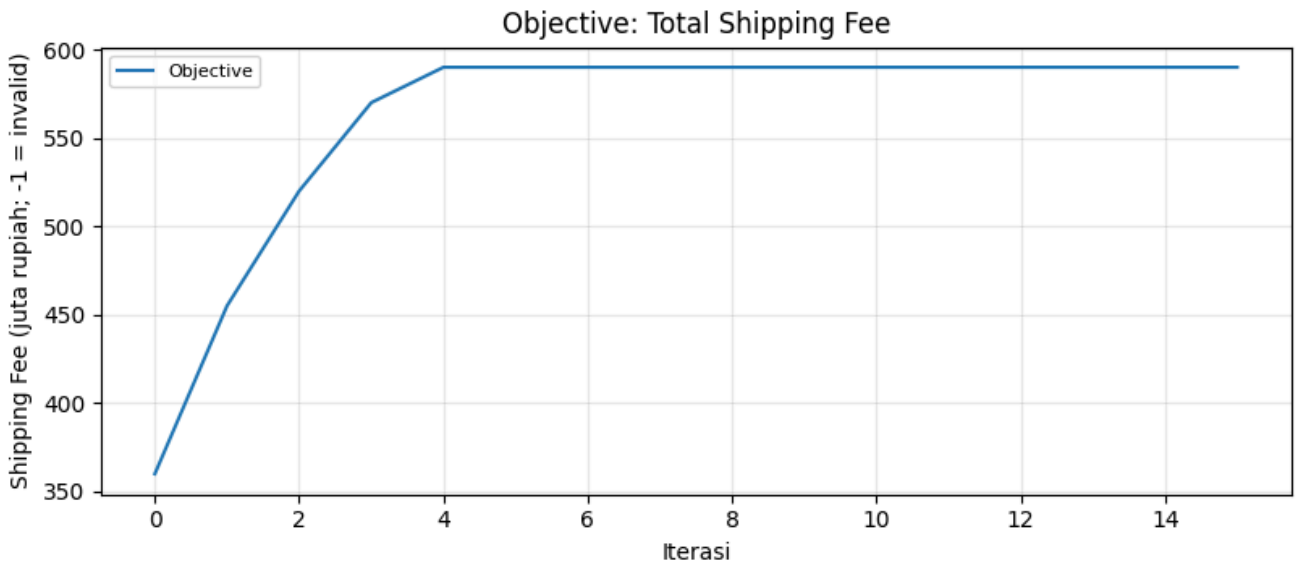
Maximum sideways move = 10; Maximum iterasi = 300

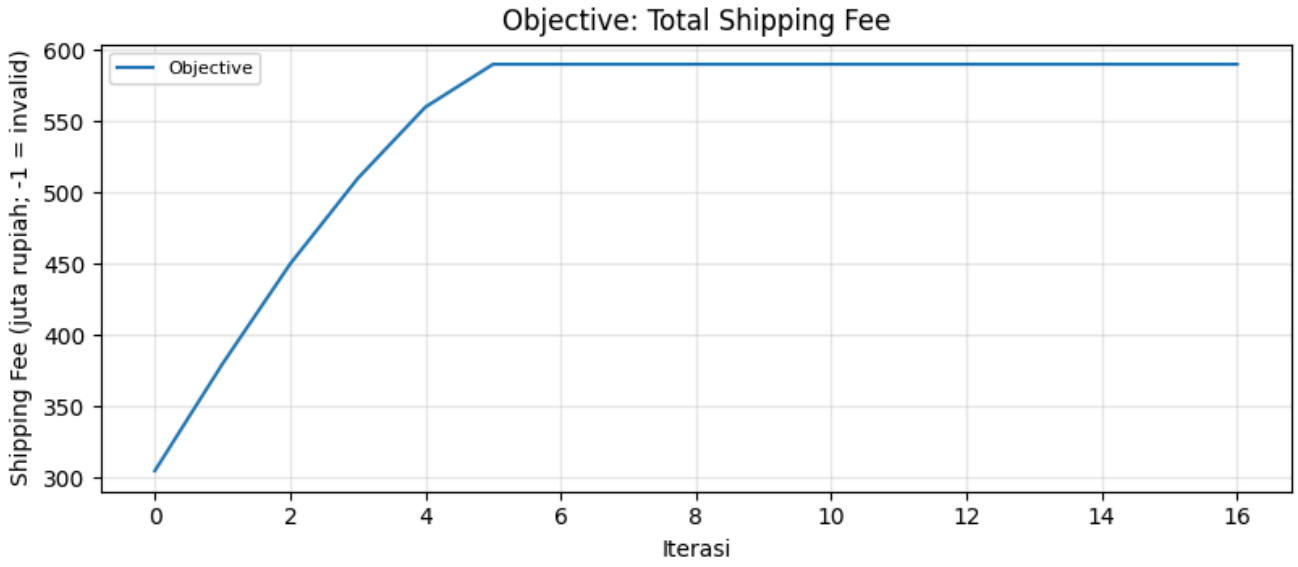
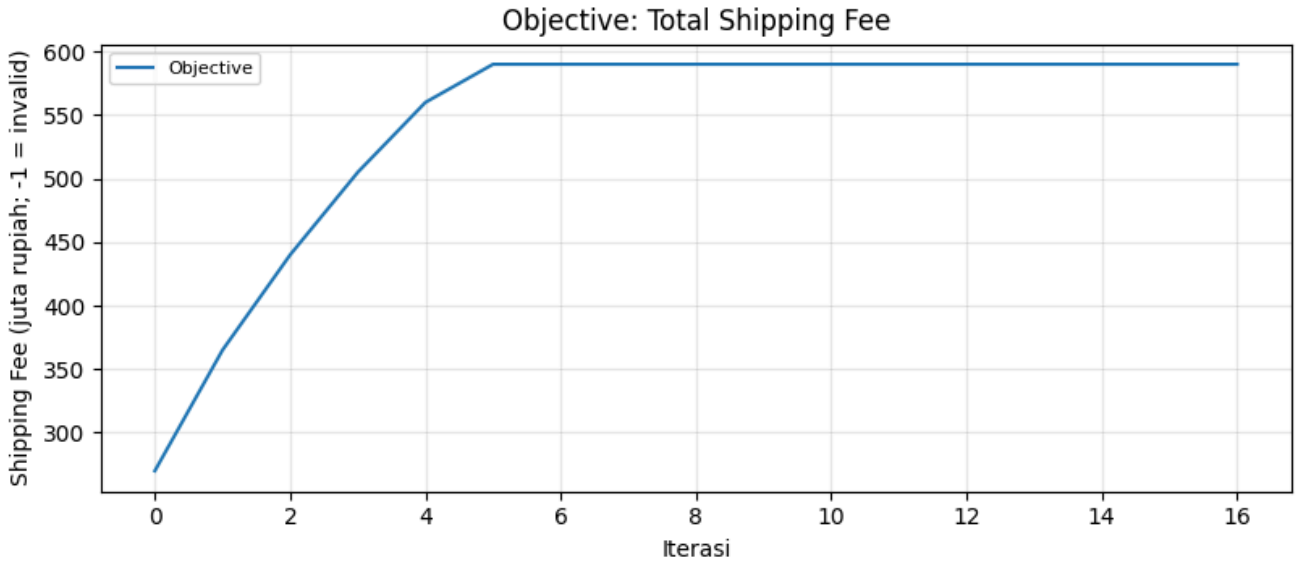
Tabel 4.2.1 Tabel Hasil Eksperimen Hill Climbing Sideways Move

Run	State awal	State akhir	Objective Akhir	Durasi	Total Iterasi
1			590	1.91	15
2			590	2.05	16

3			590	2.06	16

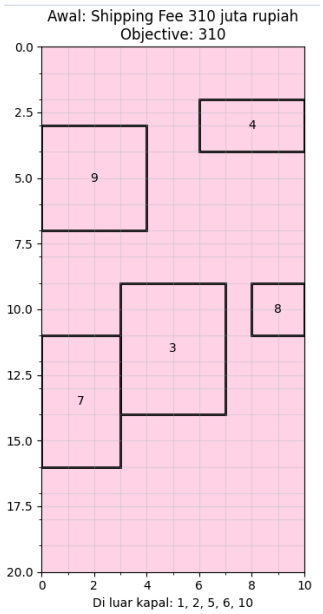
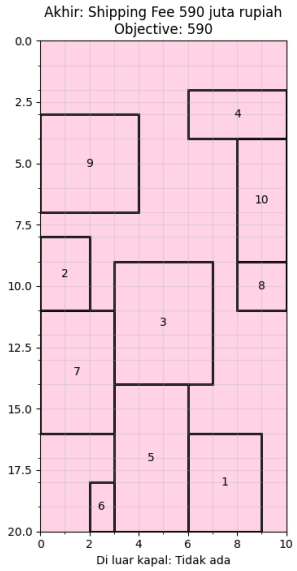
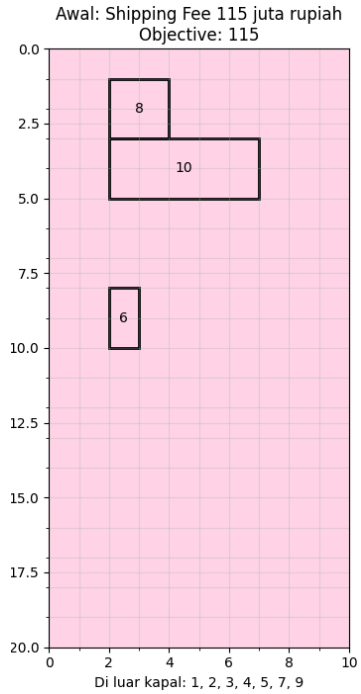
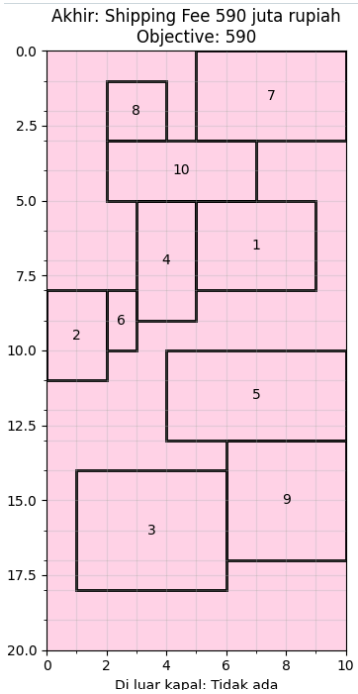
Tabel 4.2.2 Plot Nilai Objective Function Algoritma Hill Climbing Sideways Move

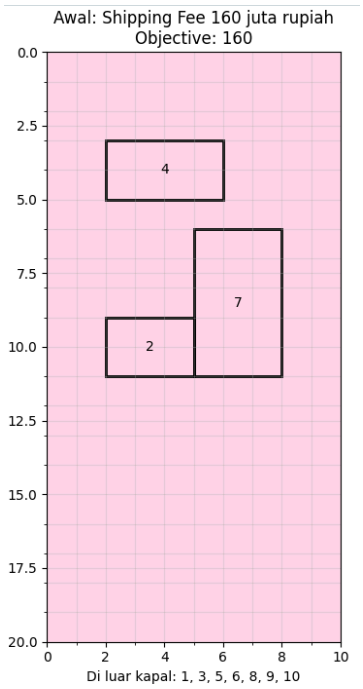
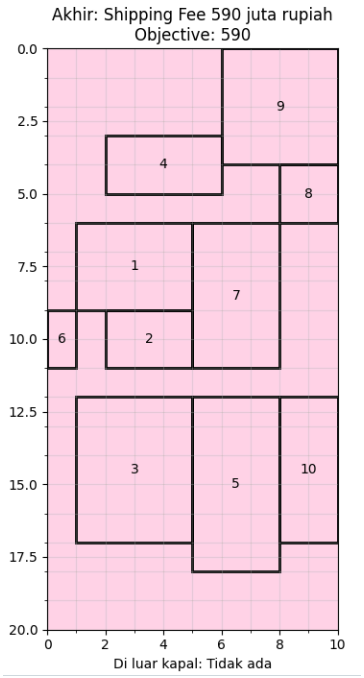
Run	Plot nilai objective function
1	

2	 Objective: Total Shipping Fee Objective Shipping Fee (juta rupiah; -1 = invalid) 600 550 500 450 400 350 300 0 2 4 6 8 10 12 14 16 Iterasi
3	 Objective: Total Shipping Fee Objective Shipping Fee (juta rupiah; -1 = invalid) 600 550 500 450 400 350 300 0 2 4 6 8 10 12 14 16 Iterasi

4.3 Hasil Eksperimen Hill Climbing Random Restart

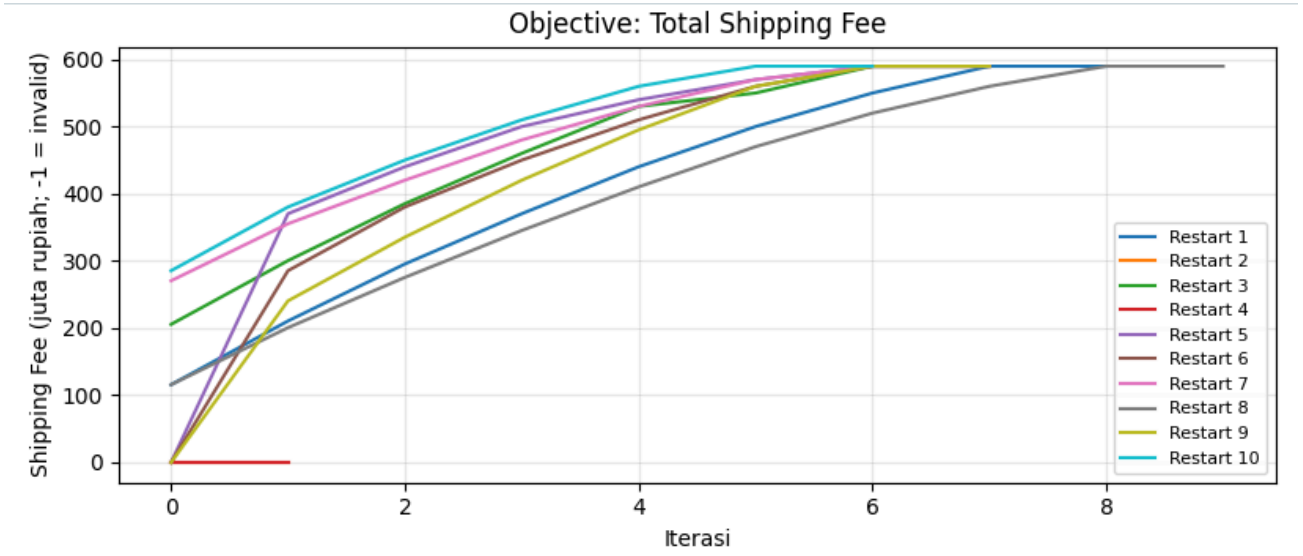
Tabel 4.3.1 Tabel Hasil Eksperimen Hill Climbing Random Restart

Run	State awal	State akhir	Objective Akhir	Durasi	Total Iterasi	Restart
1			590	7.3082 24s	60	10
2			590	7.5314 66s	60	10

3			590	6.0521 35s	49	10

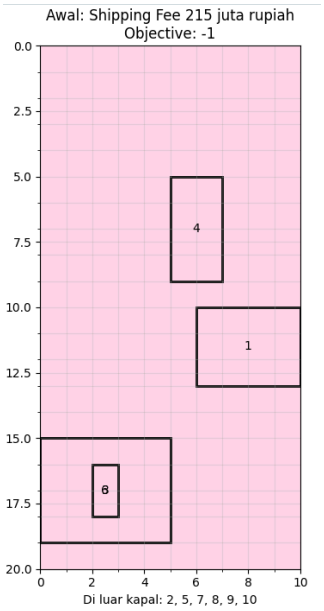
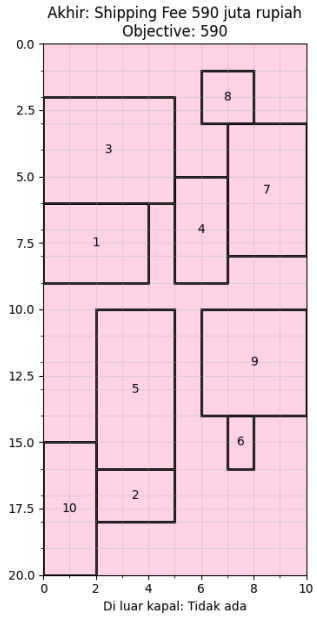
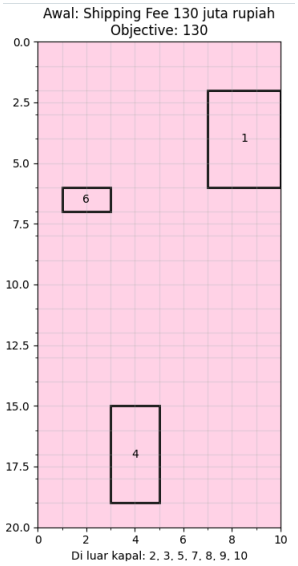
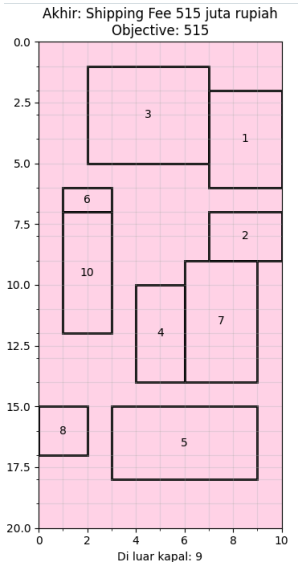
Tabel 4.3.2 Plot Nilai Objective Function Algoritma Hill Climbing Random Restart

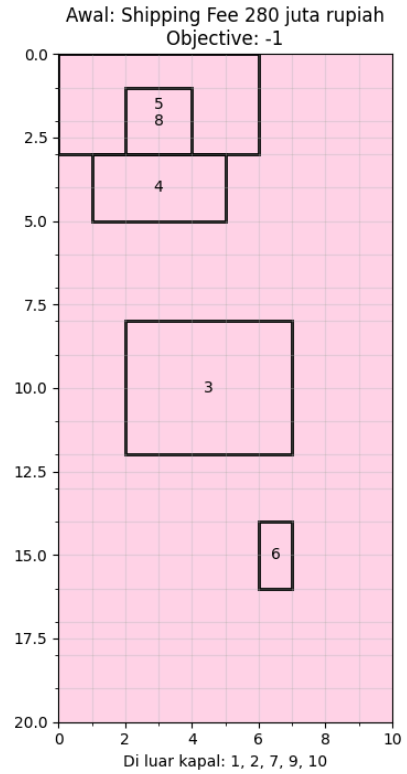
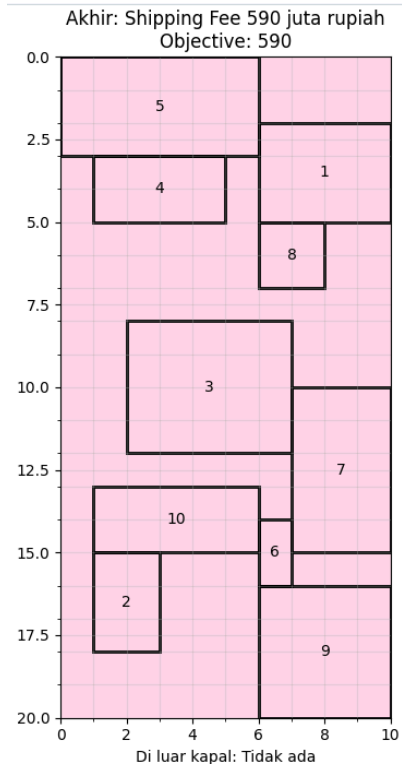
Run	**Plot nilai objective function**
1	

2	 Objective: Total Shipping Fee Shipping Fee (juta rupiah; -1 = invalid) Iterasi Restart 1 Restart 2 Restart 3 Restart 4 Restart 5 Restart 6 Restart 7 Restart 8 Restart 9 Restart 10
3	 Objective: Total Shipping Fee Shipping Fee (juta rupiah; -1 = invalid) Iterasi Restart 1 Restart 2 Restart 3 Restart 4 Restart 5 Restart 6 Restart 7 Restart 8 Restart 9 Restart 10

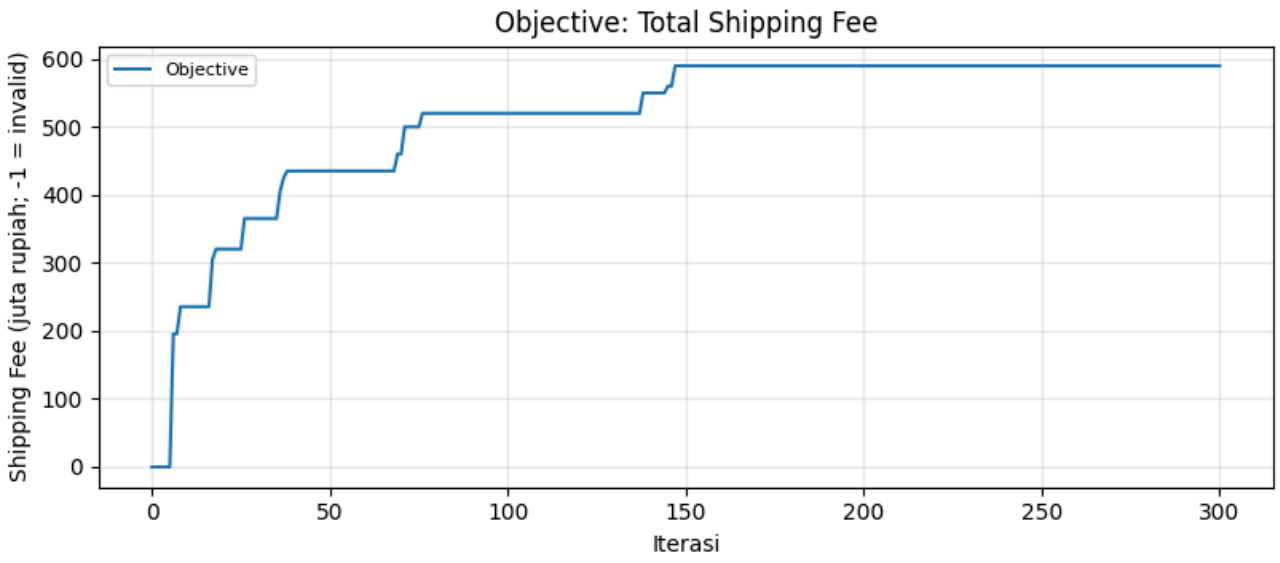
4.4 Hasil Eksperimen Hill Climbing Stochastic

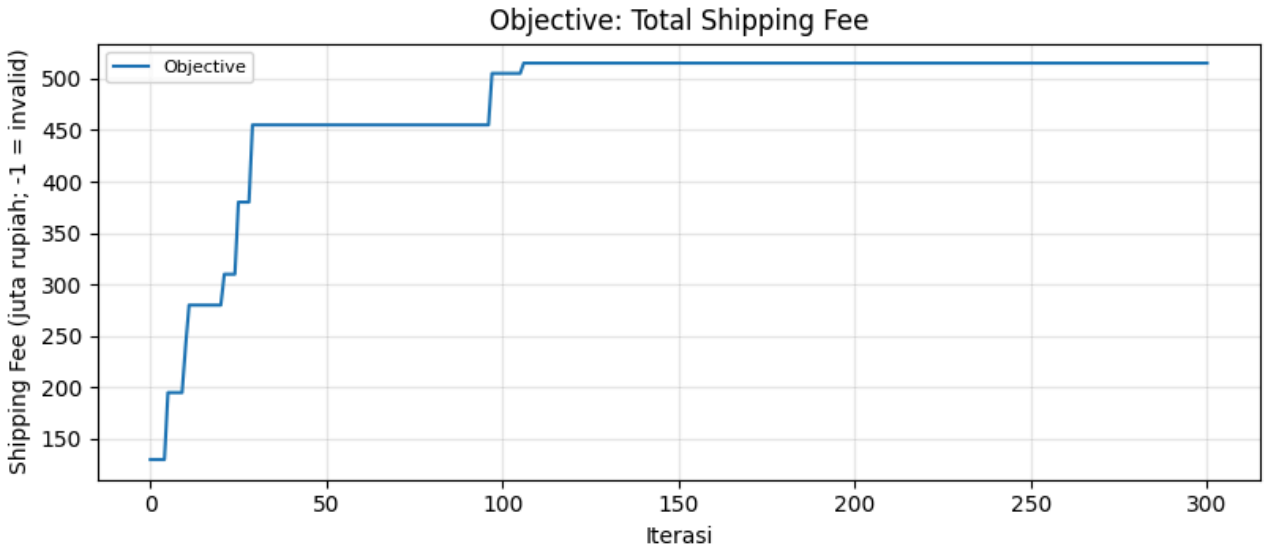
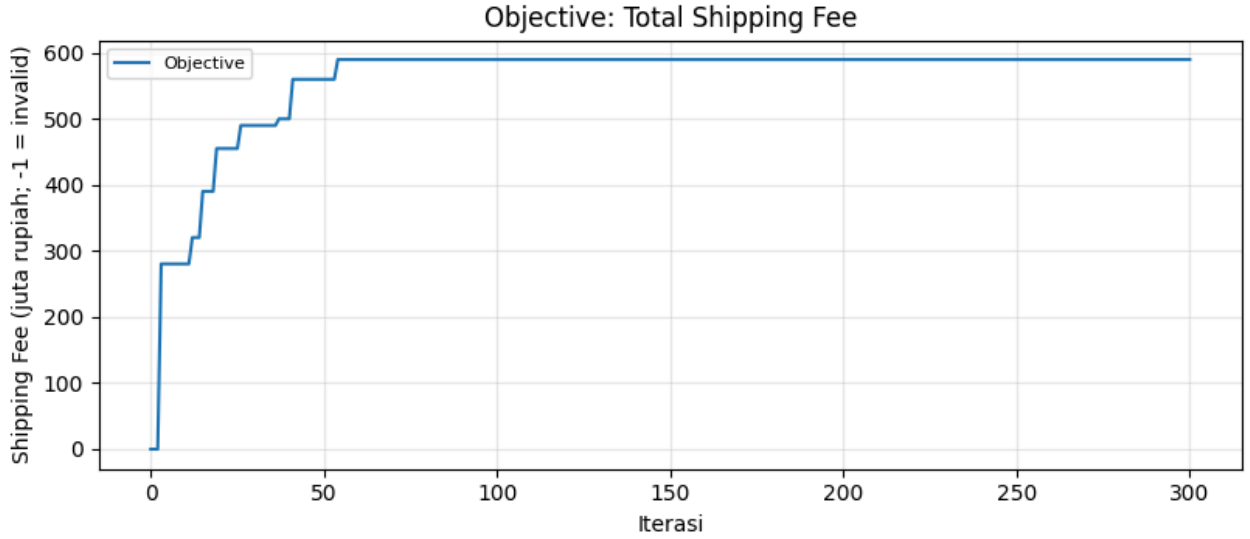
Tabel 4.4.1 Tabel Hasil Eksperimen Hill Climbing Stochastic

Run	State awal	State akhir	Objective Akhir	Durasi	Total Iterasi
1			590	0.028891s	300
2			515	0.029260s	300

3			590	0.030078s	300

Tabel 4.4.2 Plot Nilai Objective Function Algoritma Hill Climbing Stochastic

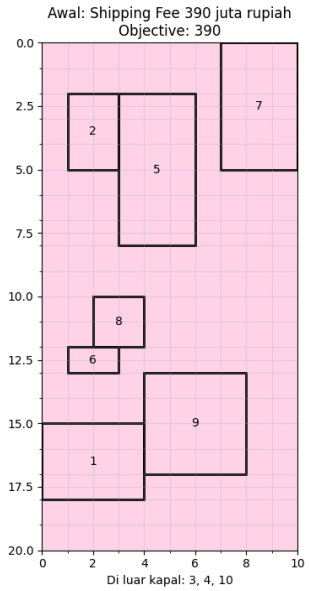
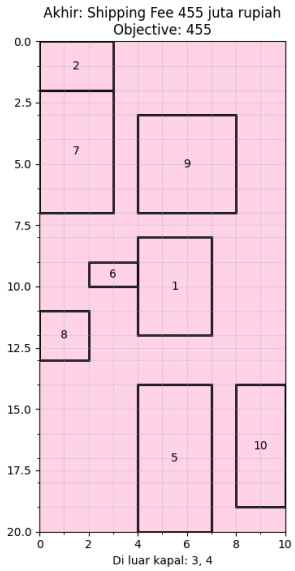
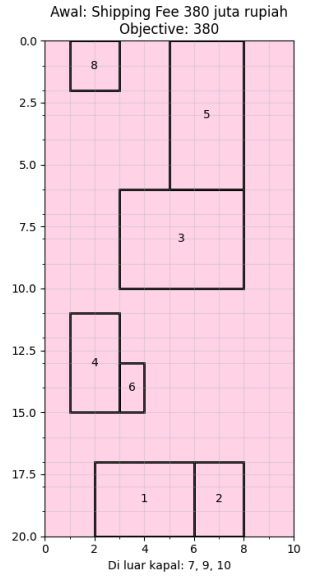
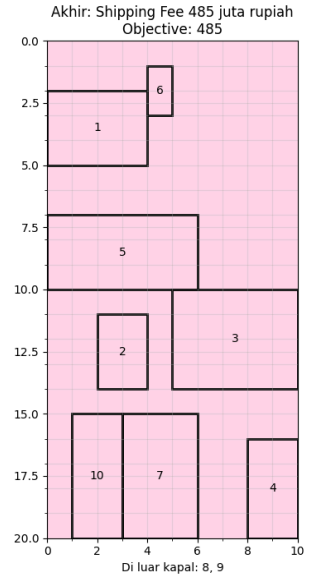
Run	Plot nilai objective function
1	

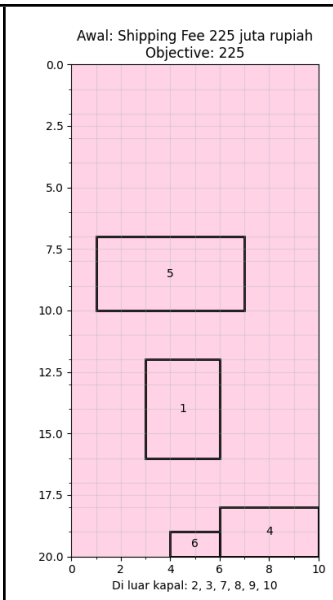
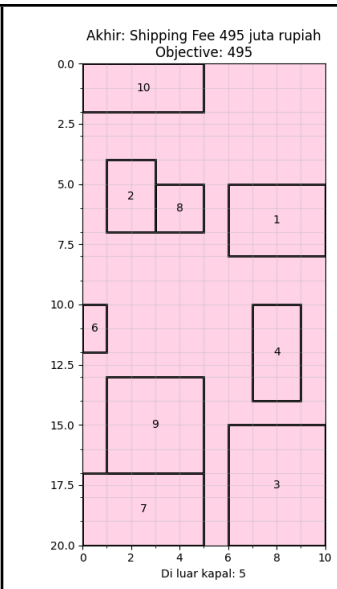
2	 Objective: Total Shipping Fee Objective Shipping Fee (juta rupiah; -1 = invalid) 150 200 250 300 350 400 450 500 0 50 100 150 200 250 300 Iterasi
3	 Objective: Total Shipping Fee Objective Shipping Fee (juta rupiah; -1 = invalid) 0 100 200 300 400 500 600 0 50 100 150 200 250 300 Iterasi

4.5 Hasil Eksperimen Algoritma Simulated Annealing

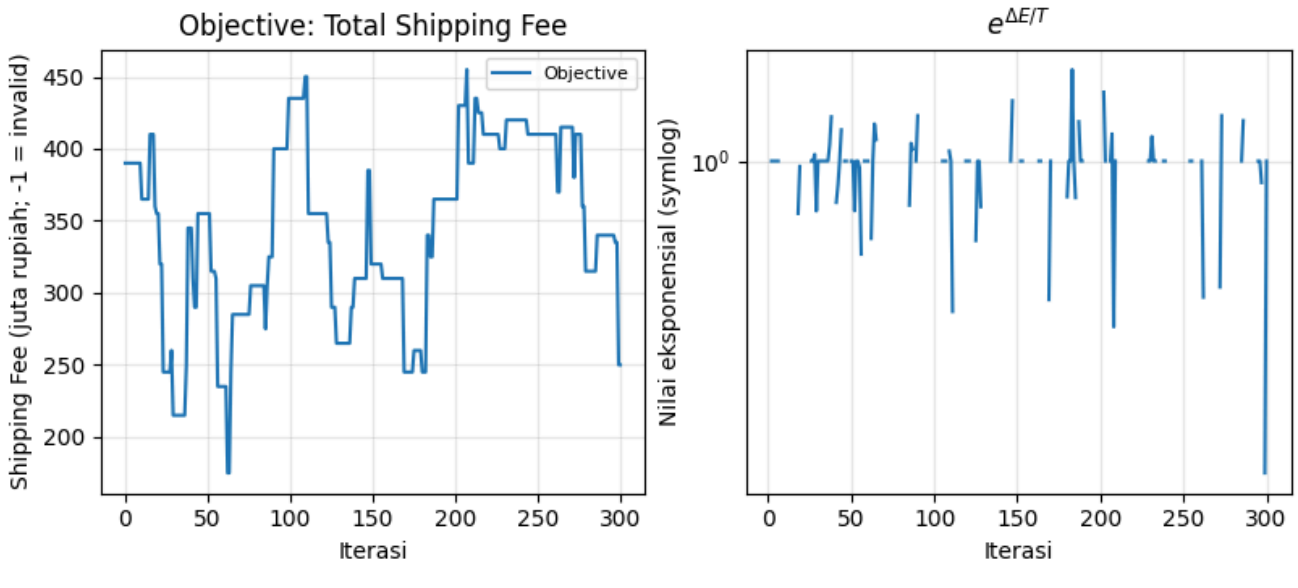
Suhu awal = 1000; Cooling rate = 0.995; Stuck threshold = 100 ; Maximum iterasi = 300

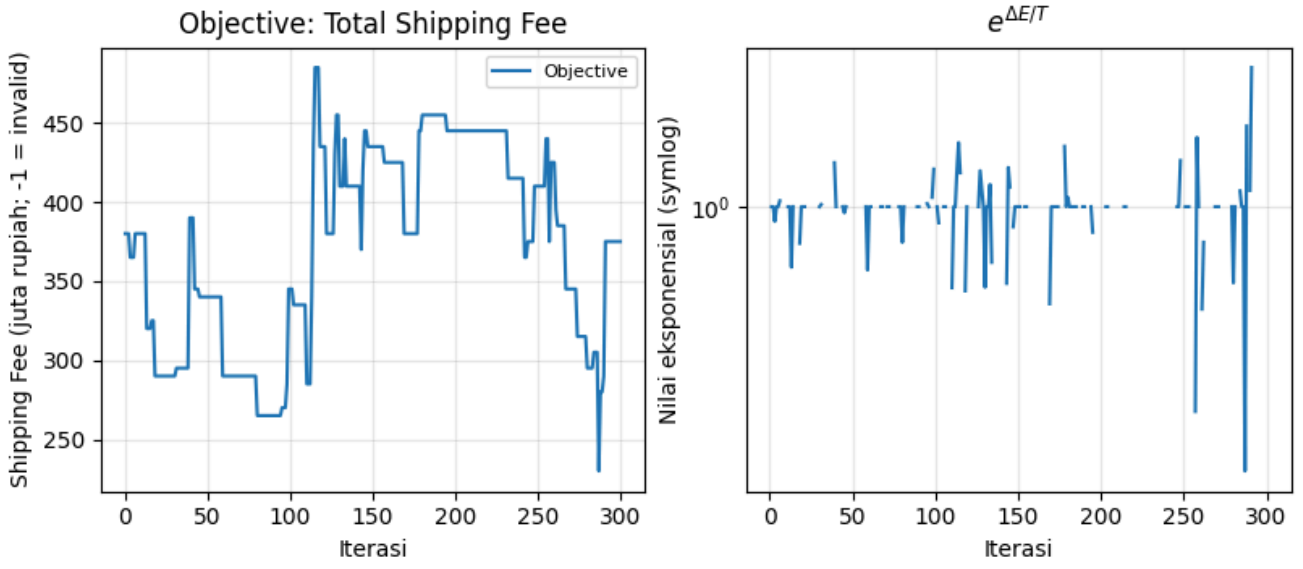
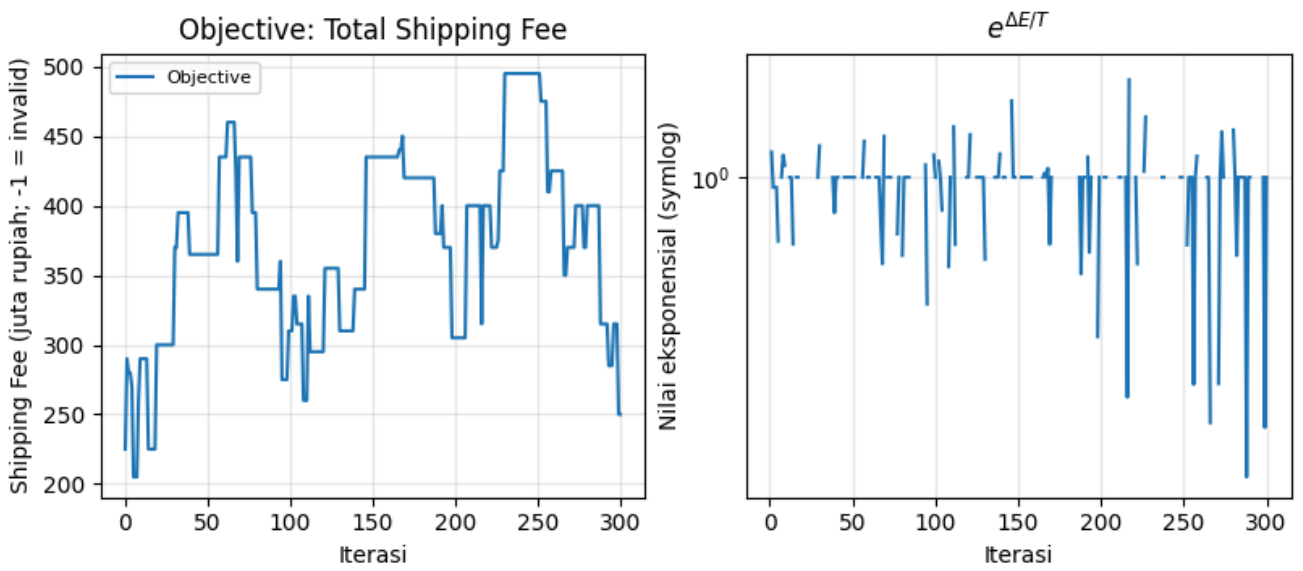
Tabel 4.5.1 Tabel Hasil Eksperimen Algoritma Simulated Annealing

Run	State awal	State akhir	Objective Akhir	Frekuensi Stuck	Durasi	Total Iterasi
1			455	0	0.03	300
2			485	1	0.03	300

3			495	1	0.03	300

Tabel 4.5.2 Plot Nilai Objective Function Algoritma Simulated Annealing

Run	**Plot nilai objective function**
1	

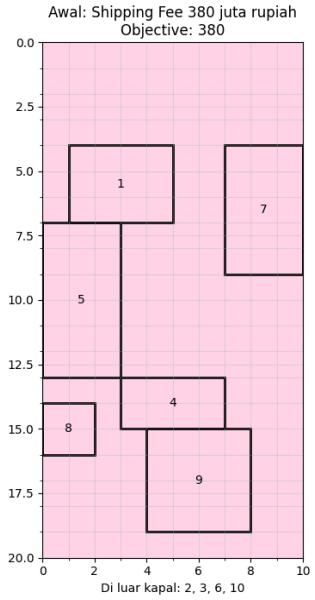
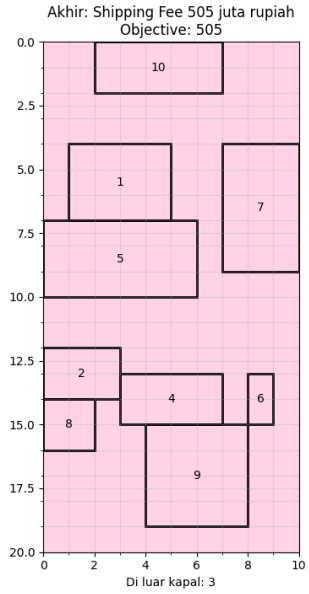
2	 Objective: Total Shipping Fee Objective Shipping Fee (juta rupiah; -1 = invalid) Iterasi $e^{\Delta E/T}$ Nilai eksponensial (symlog) 10^0 Iterasi
3	 Objective: Total Shipping Fee Objective Shipping Fee (juta rupiah; -1 = invalid) Iterasi $e^{\Delta E/T}$ Nilai eksponensial (symlog) 10^0 Iterasi

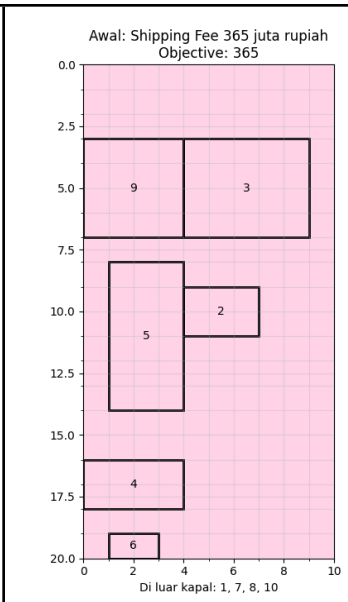
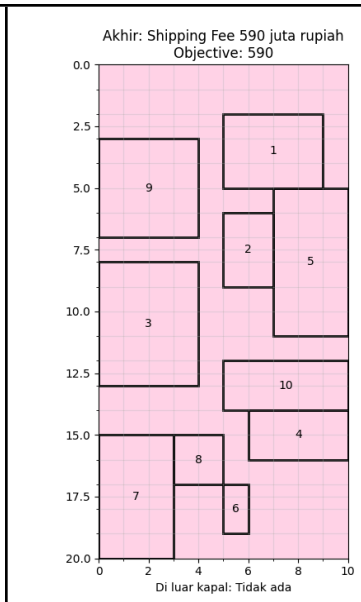
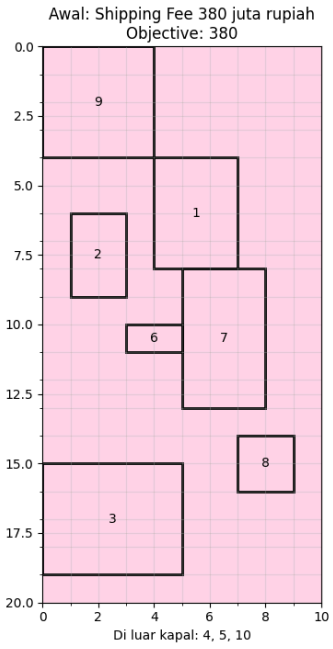
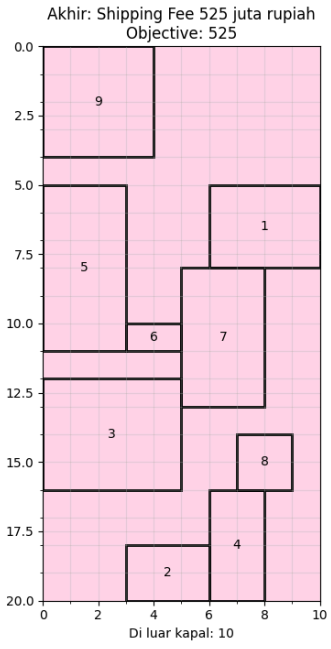
4.6 Hasil Eksperimen Genetic Algorithm

Skenario 1: Jumlah Populasi Tetap, Iterasi Bervariasi

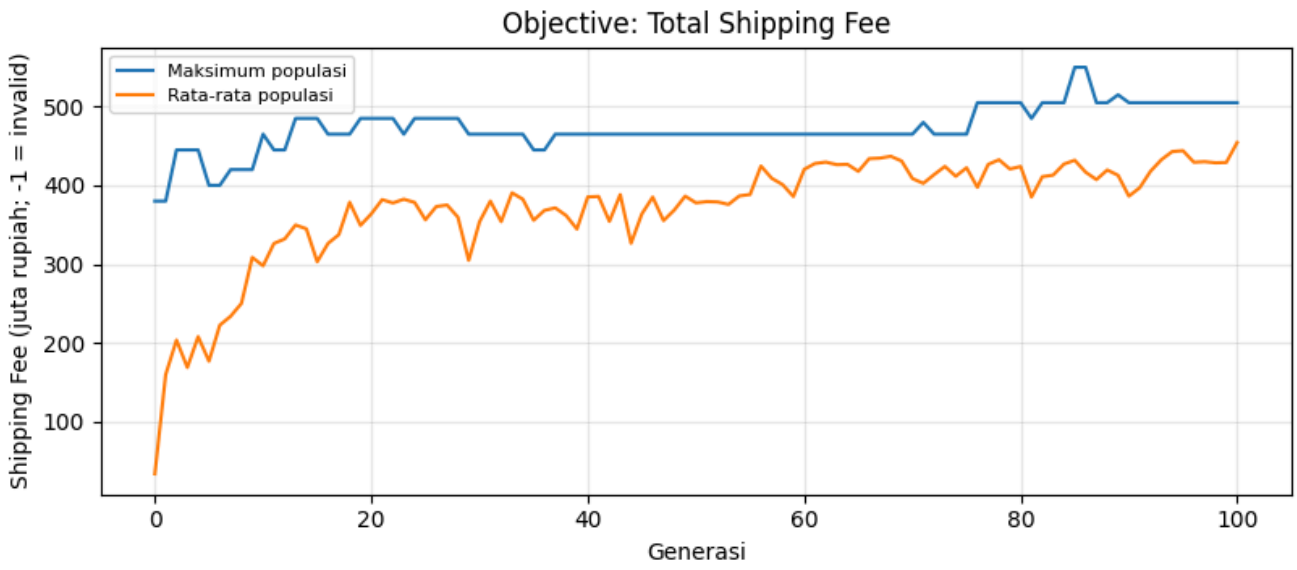
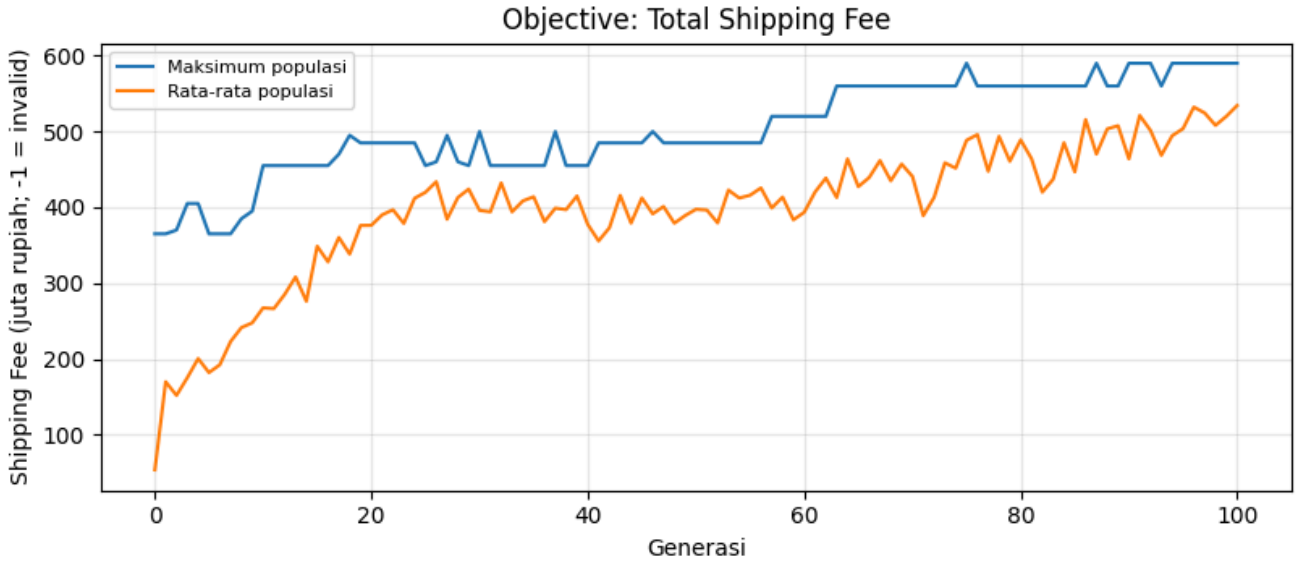
Populasi = 50, iterasi = 100

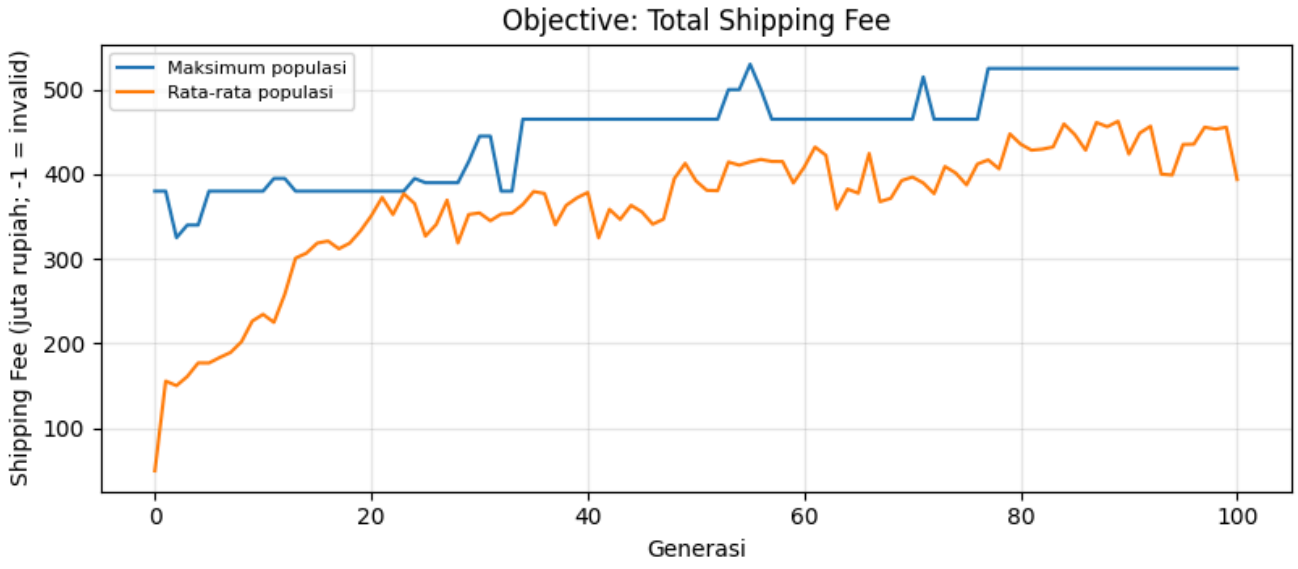
Tabel 4.6.1 Tabel Hasil Eksperimen Genetic Algorithm Populasi=50 Iterasi=100

Run	State awal	State akhir	Objective	Durasi	Total Iterasi	Banyak Populasi
1			505	0.39	100	50

2			590	0.44	100	50
3			525	0.38	100	50

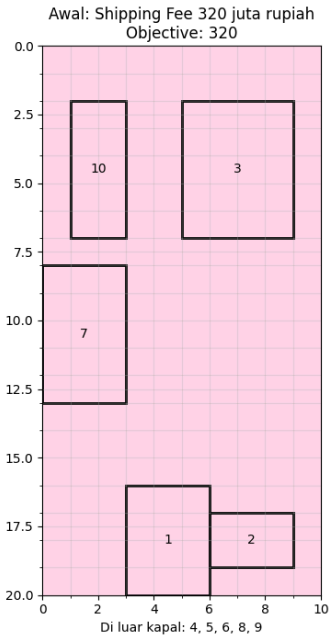
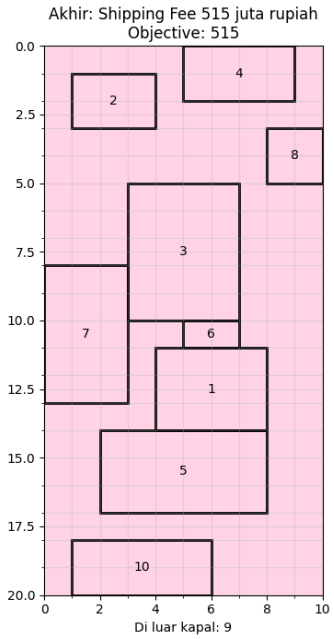
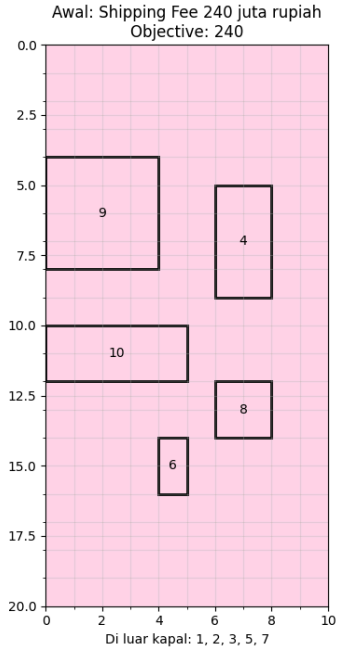
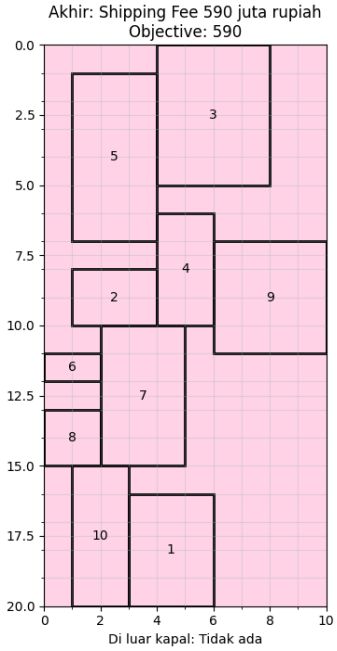
Tabel 4.6.2 Plot Nilai Objective Function Algoritma Genetic Algorithm Populasi=50 Iterasi=100

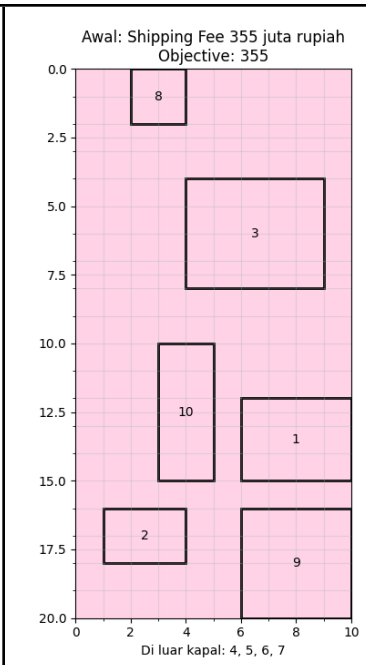
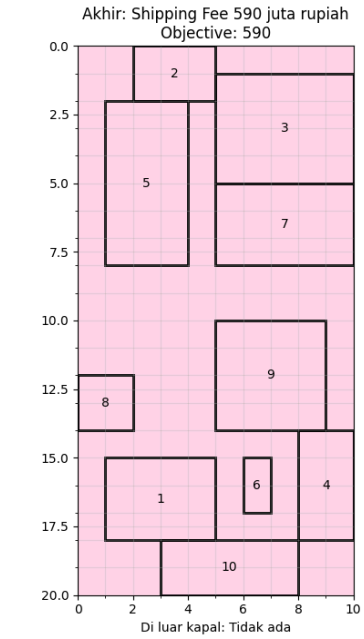
Run	Plot nilai objective function
1	
2	

3	 Objective: Total Shipping Fee Maksimum populasi Rata-rata populasi Shipping Fee (juta rupiah; -1 = invalid) 500 400 300 200 100 0 20 40 60 80 100 Generasi

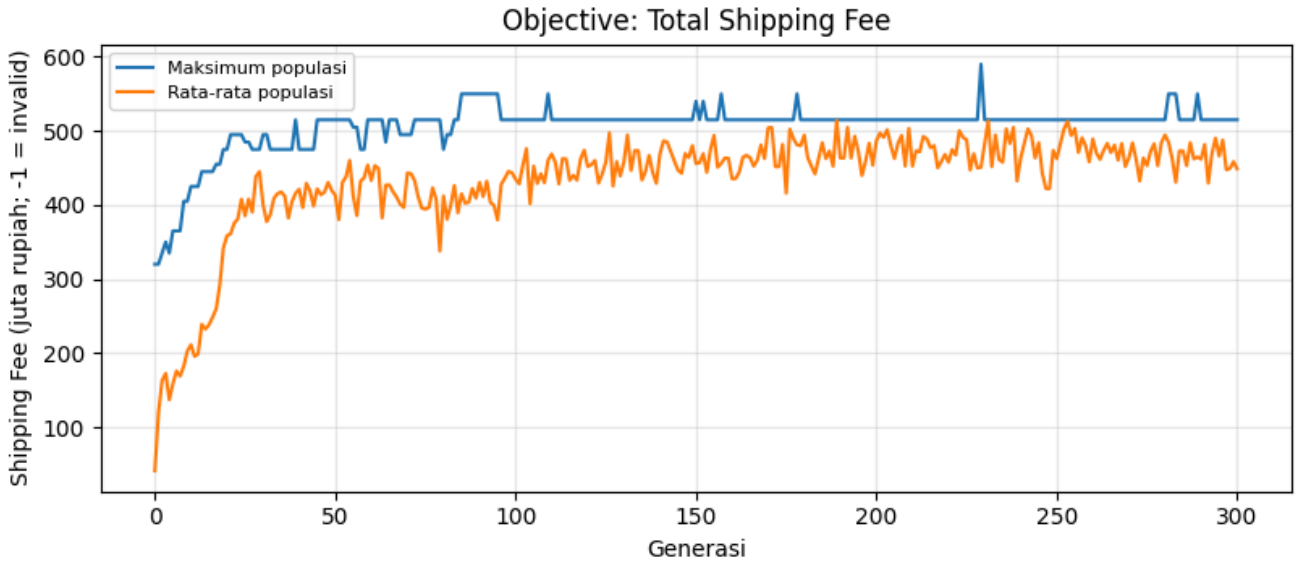
Populasi = 50, iterasi = 300

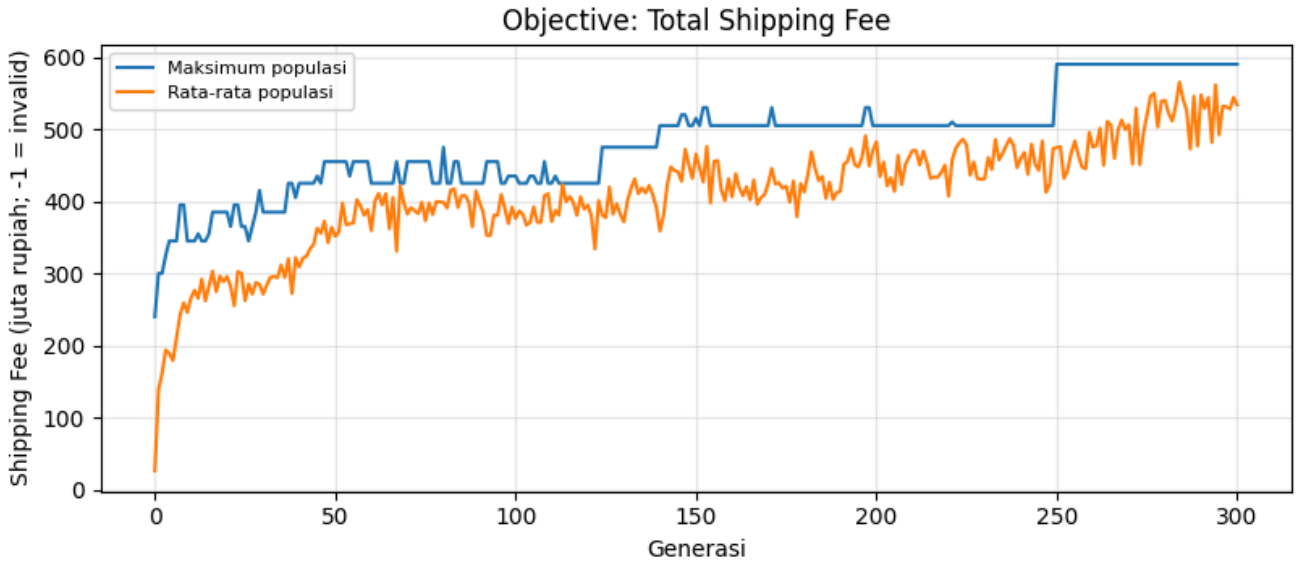
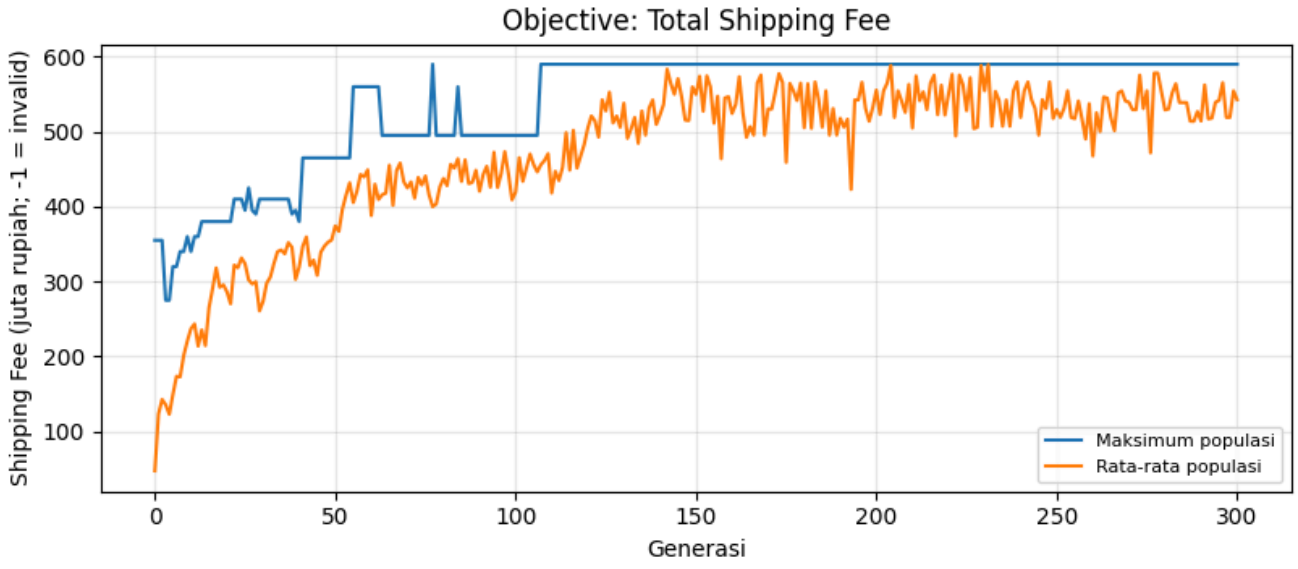
Tabel 4.6.3 Tabel Hasil Eksperimen Genetic Algorithm Populasi=50 Iterasi=300

Run	State awal	State akhir	Objective	Durasi	Total Iterasi	Banyak Populasi
1			515	1.20	300	50
2			590	1.17	300	50

3			590	1.26	300	50

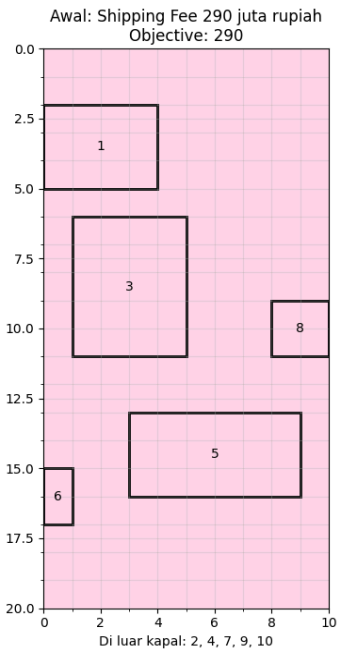
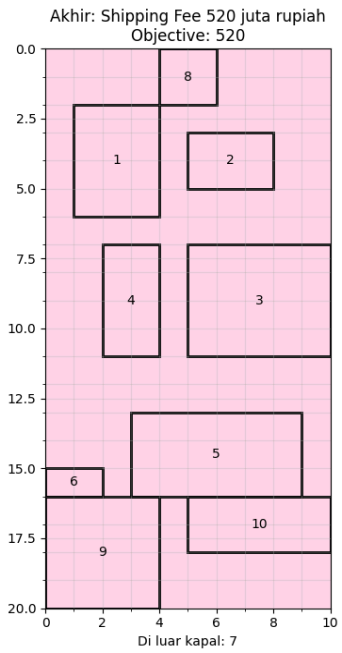
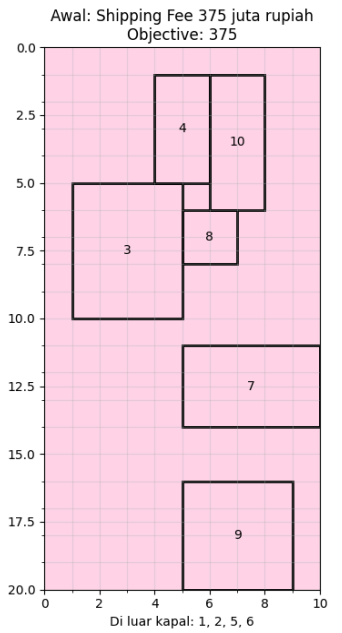
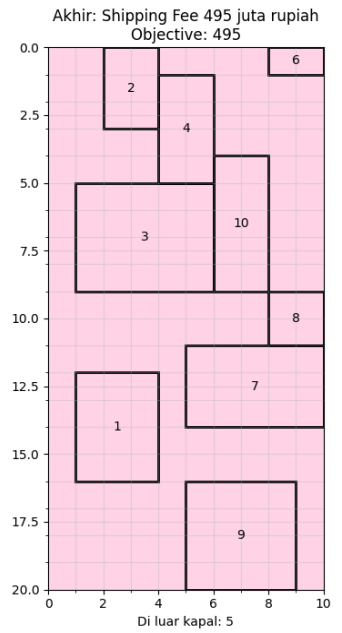
Tabel 4.6.4 Plot Nilai Objective Function Algoritma Genetic Algorithm Populasi=50 Iterasi=300

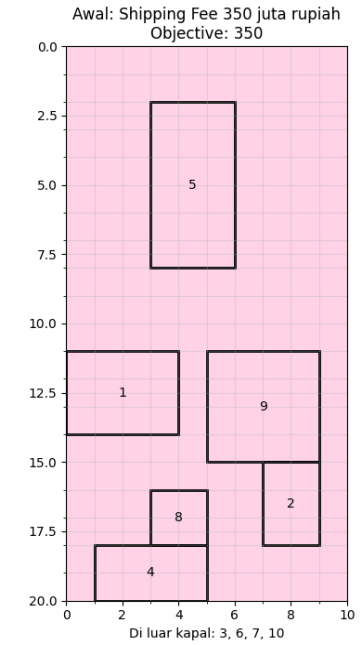
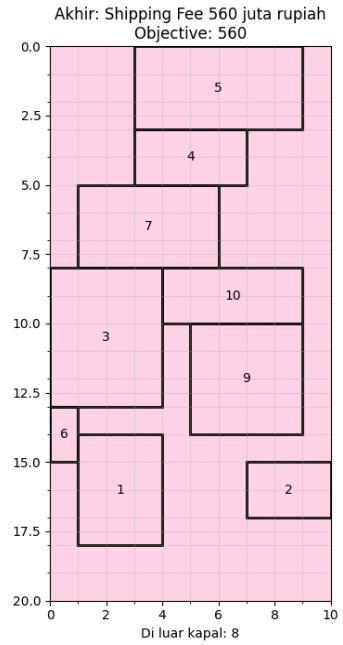
Run	Plot nilai objective function
1	

2	 Objective: Total Shipping Fee Maksimum populasi Rata-rata populasi Shipping Fee (juta rupiah; -1 = invalid) 0 100 200 300 400 500 600 0 50 100 150 200 250 300 Generasi
3	 Objective: Total Shipping Fee Shipping Fee (juta rupiah; -1 = invalid) 100 200 300 400 500 600 0 50 100 150 200 250 300 Generasi Maksimum populasi Rata-rata populasi

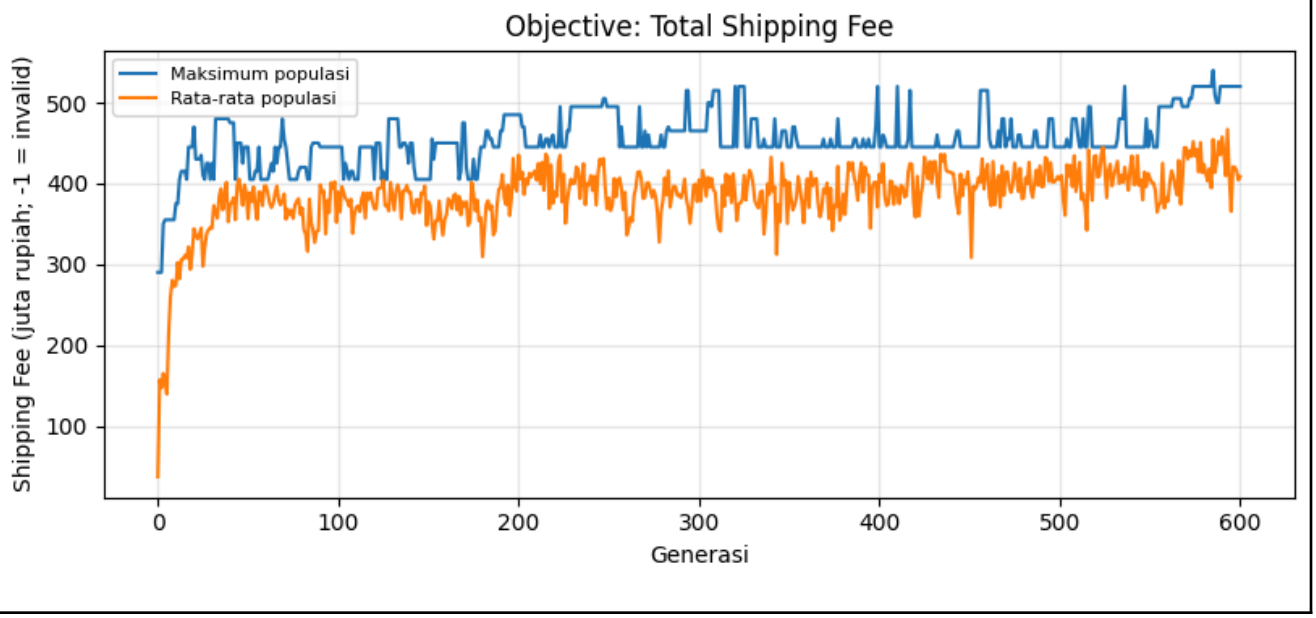
Populasi = 50, iterasi = 600

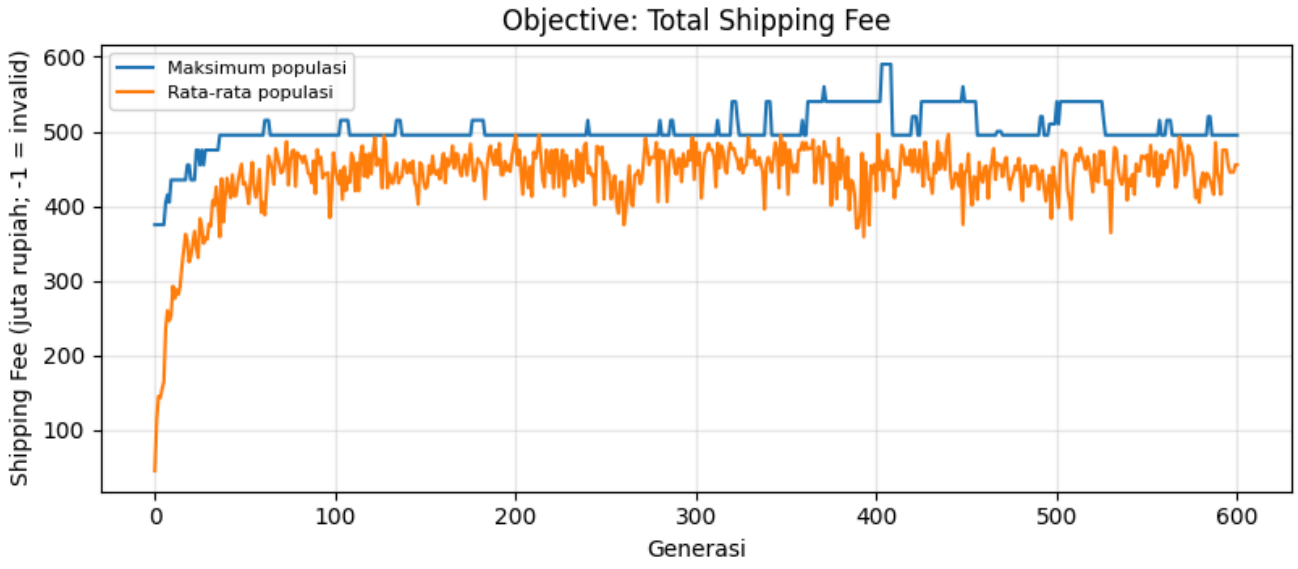
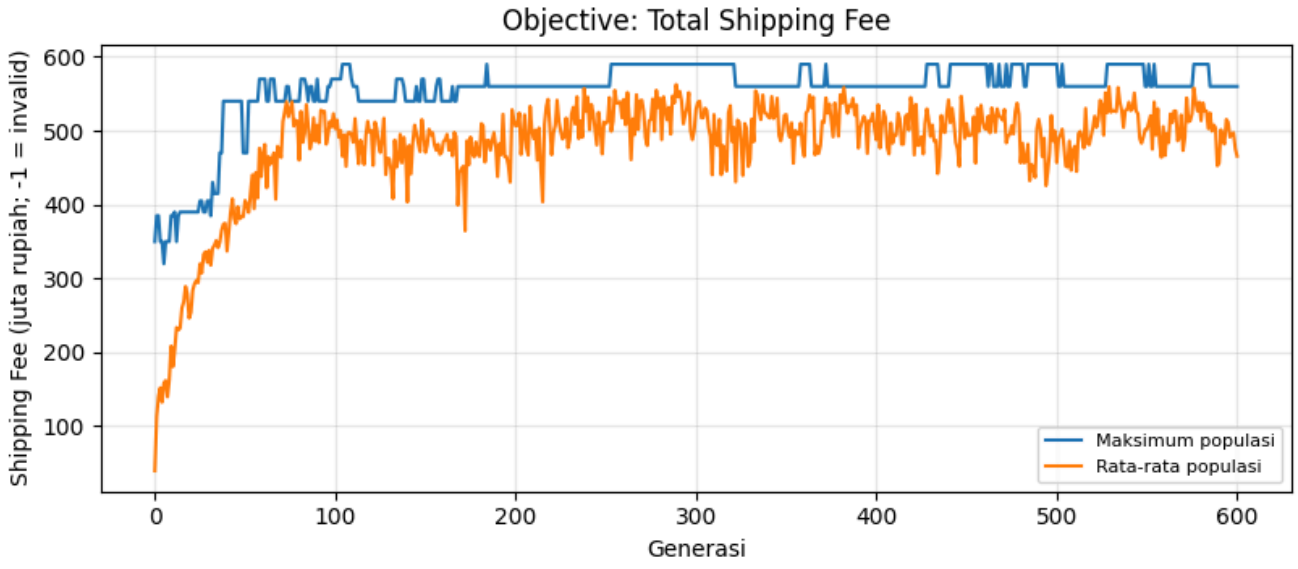
Tabel 4.6.5 Tabel Hasil Eksperimen Genetic Algorithm Populasi=50 Iterasi=600

Run	State awal	State akhir	Objective	Durasi	Total Iterasi	Banyak Populasi
1			520	5.64	600	50
2			495	6.90	600	50

3			560	4.39	600	50

Tabel 4.6.6 Plot Nilai Objective Function Algoritma Genetic Algorithm Populasi=50 Iterasi=600

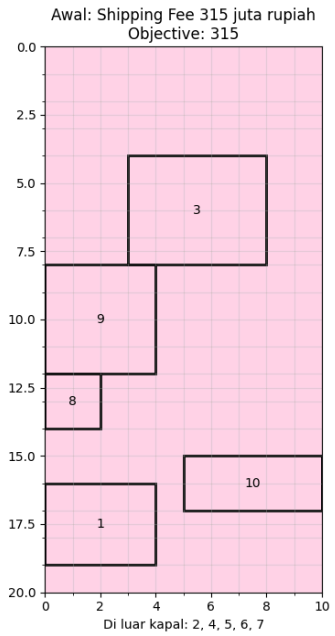
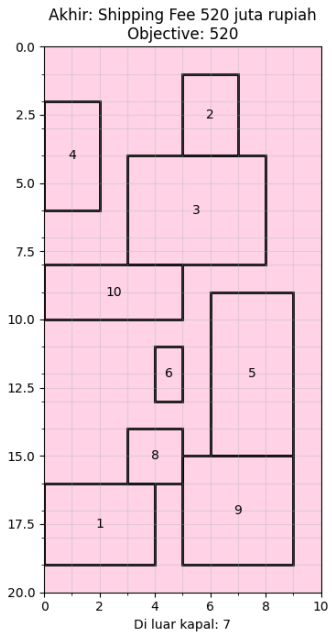
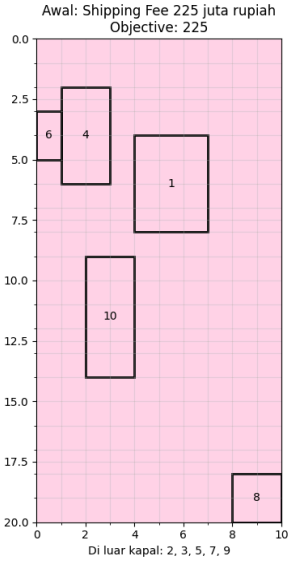
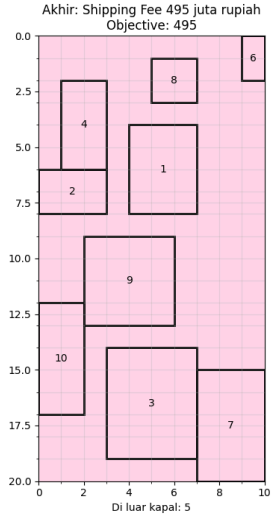
Run	**Plot nilai objective function**
1	

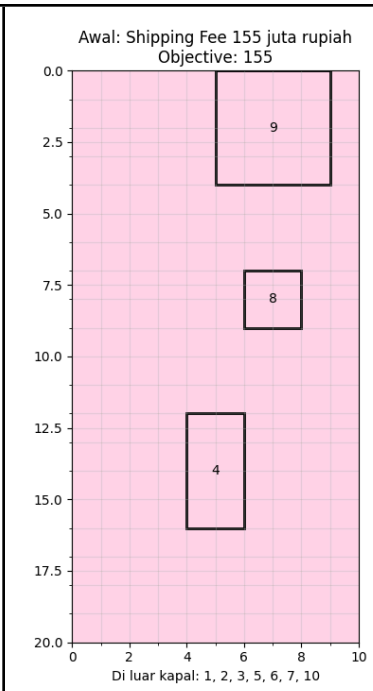
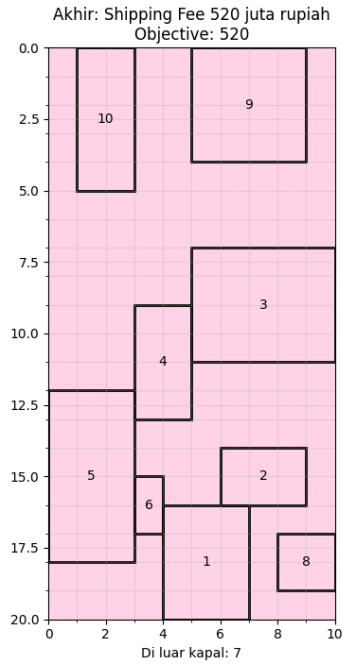
2	 Objective: Total Shipping Fee Maksimum populasi Rata-rata populasi Shipping Fee (juta rupiah; -1 = invalid) 600 500 400 300 200 100 0 100 200 300 400 500 600 Generasi
3	 Objective: Total Shipping Fee Shipping Fee (juta rupiah; -1 = invalid) 600 500 400 300 200 100 0 100 200 300 400 500 600 Generasi Maksimum populasi Rata-rata populasi

Skenario 2: Jumlah Iterasi Tetap, Populasi Bervariasi

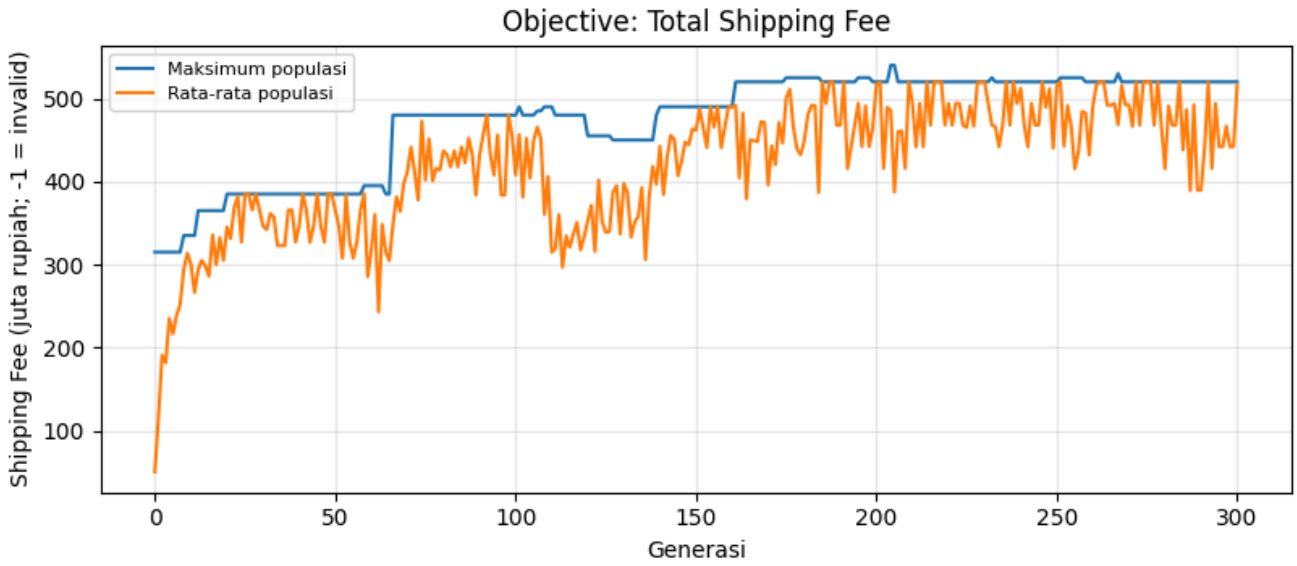
Iterasi = 300, Populasi = 20

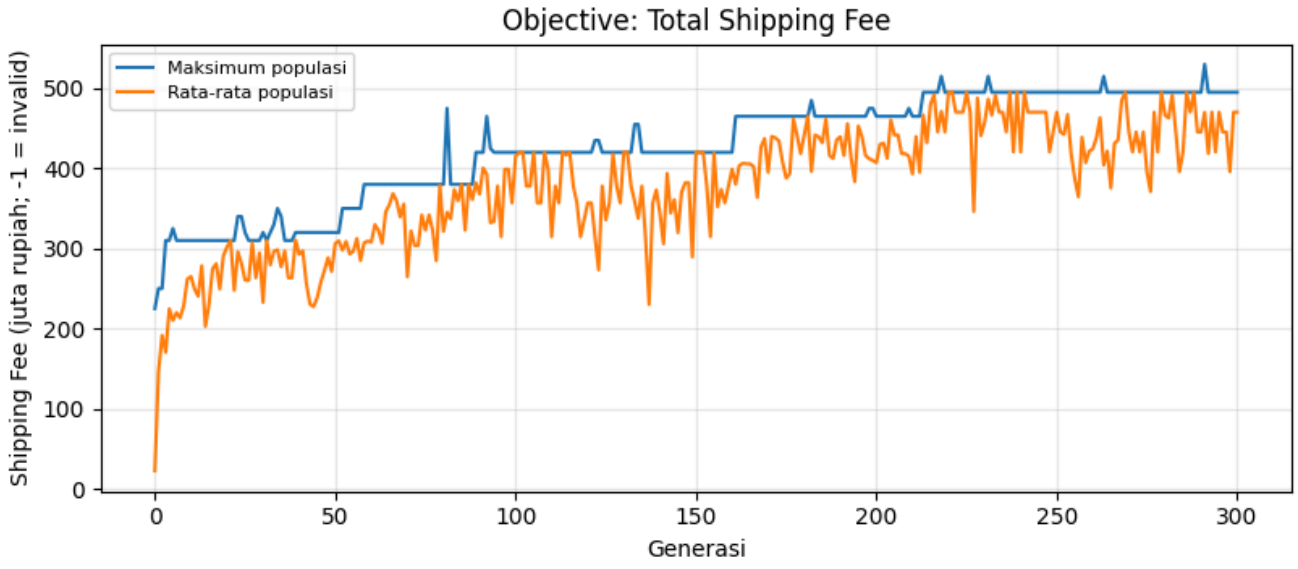
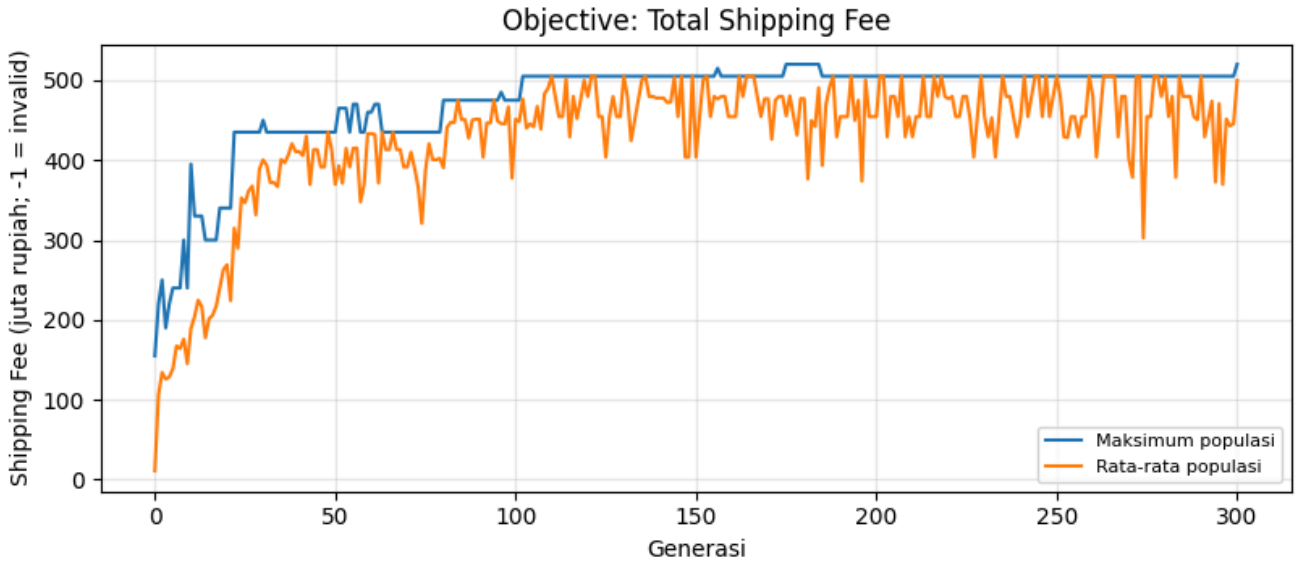
Tabel 4.6.7 Tabel Hasil Eksperimen Genetic Algorithm Iterasi=300 Populasi=20

Run	State awal	State akhir	Objective	Durasi	Total Iterasi	Banyak Populasi
1			520	0.45	300	20
2			495	0.45	300	20

3			520	0.48	300	20

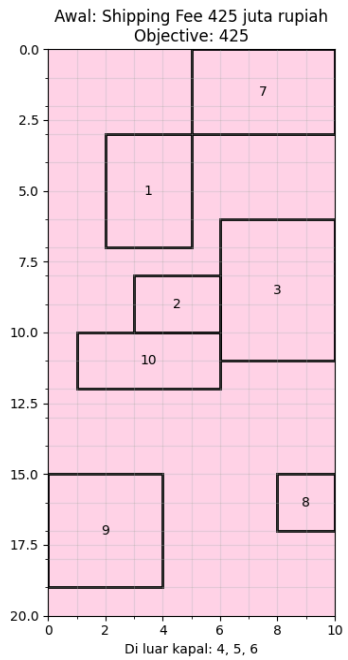
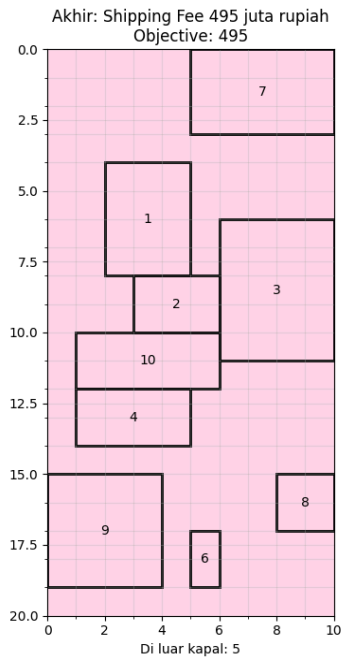
Tabel 4.6.8 Plot Nilai Objective Function Algoritma Genetic Algorithm Iterasi=300 Populasi=20

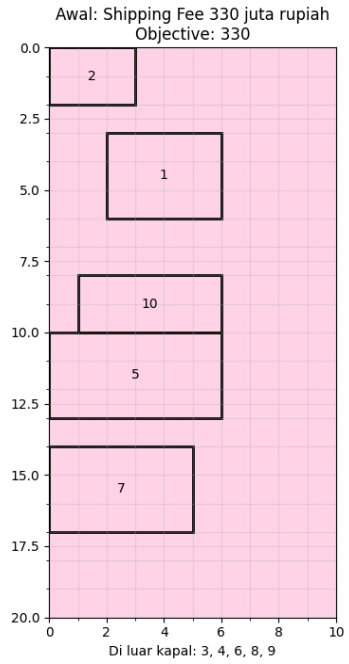
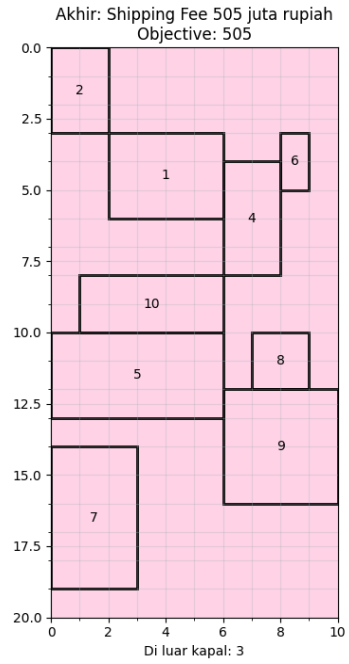
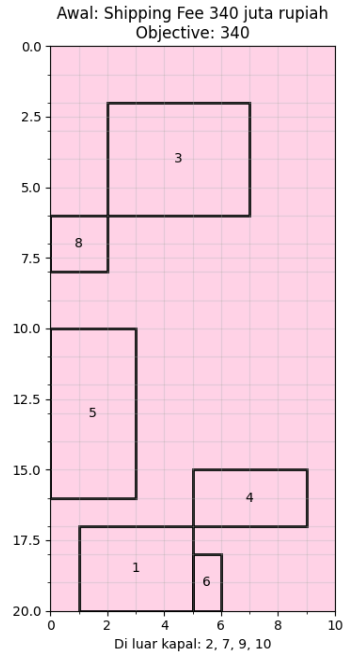
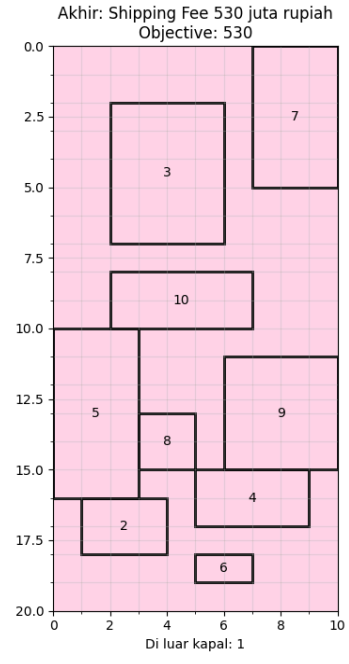
Run	Plot nilai objective function
1	

2	 Objective: Total Shipping Fee Maksimum populasi Rata-rata populasi Shipping Fee (juta rupiah; -1 = invalid) 0 100 200 300 400 500 0 50 100 150 200 250 300 Generasi
3	 Objective: Total Shipping Fee Shipping Fee (juta rupiah; -1 = invalid) 0 100 200 300 400 500 0 50 100 150 200 250 300 Generasi Maksimum populasi Rata-rata populasi

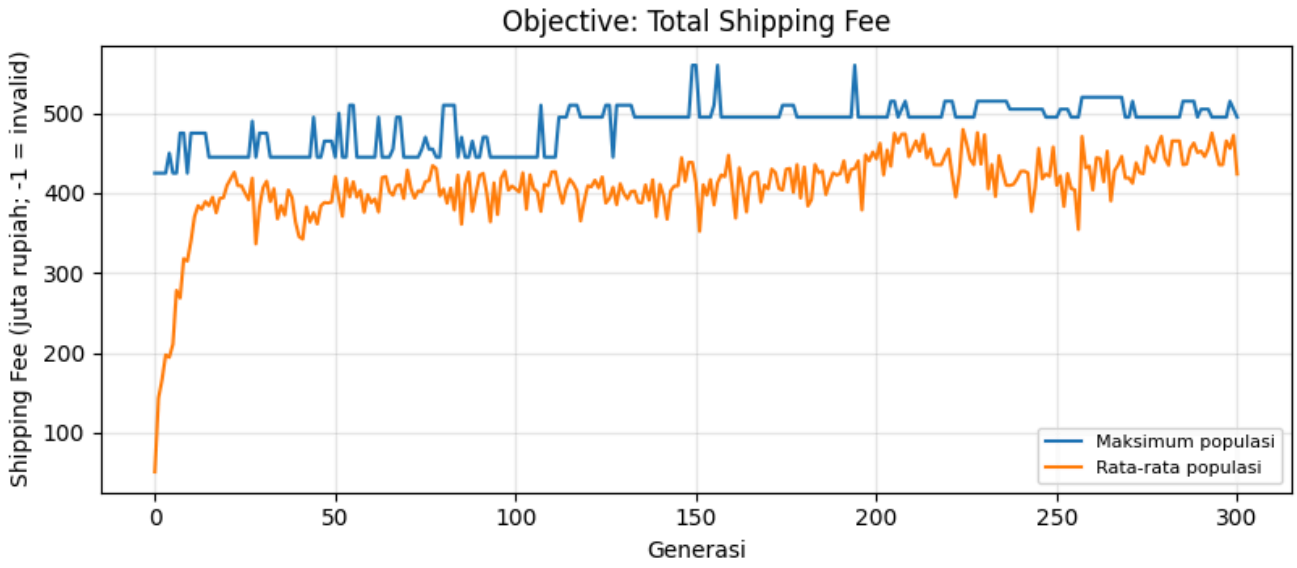
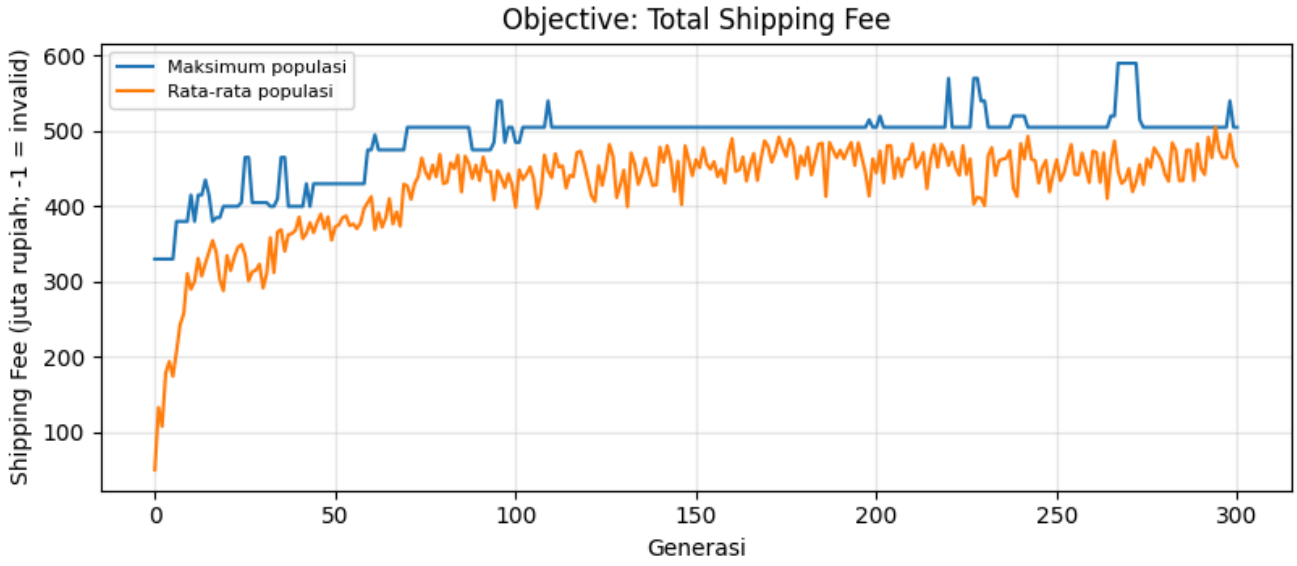
Iterasi = 300, Populasi = 50

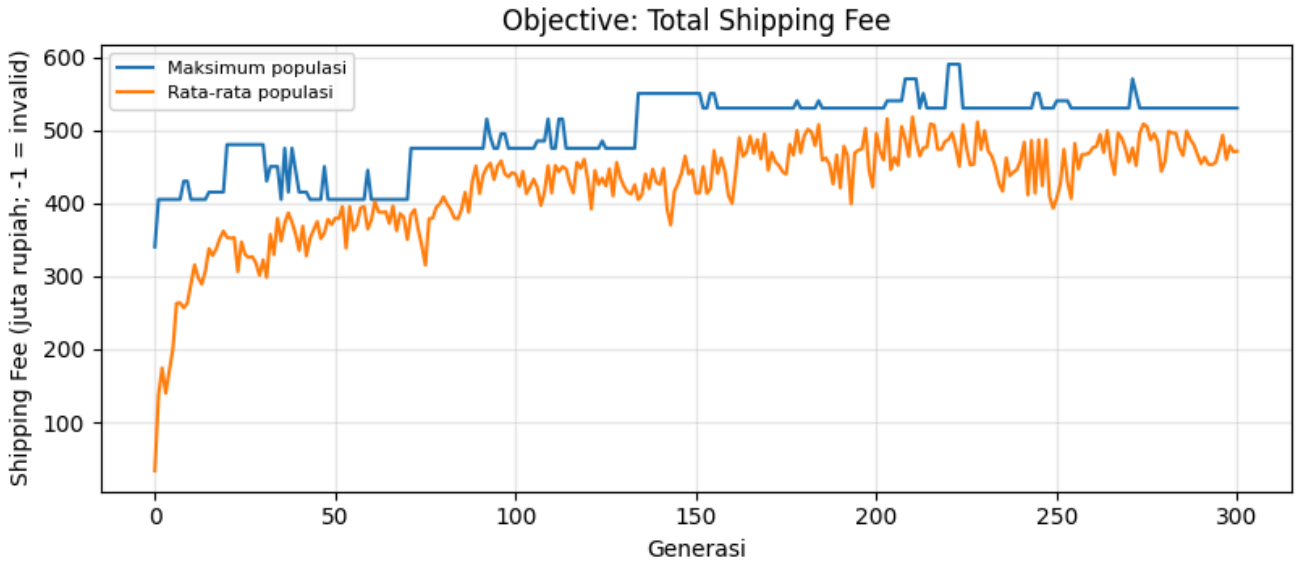
Tabel 4.6.9 Tabel Hasil Eksperimen Genetic Algorithm Iterasi=300 Populasi=50

Run	State awal	State akhir	Objective	Durasi	Total Iterasi	Banyak Populasi
1			495	1.21	300	50

2			505	1.24	300	50
3			530	1.19	300	50

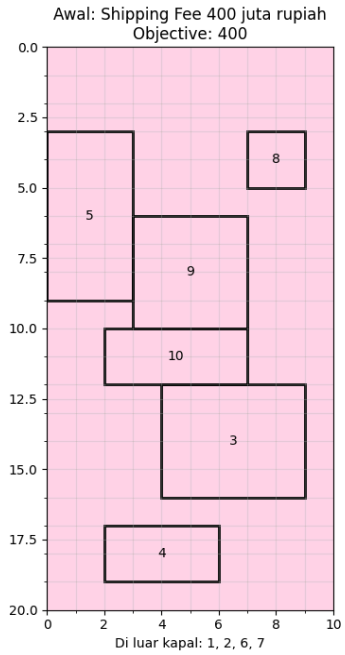
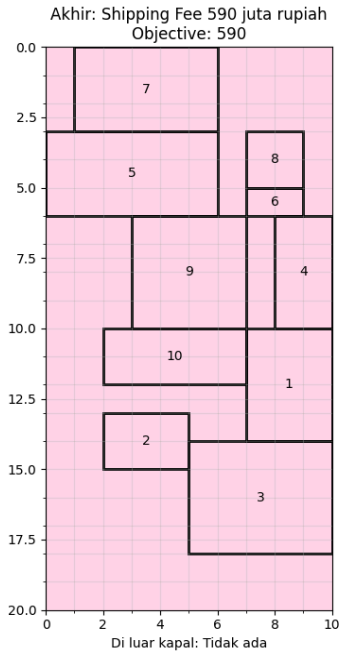
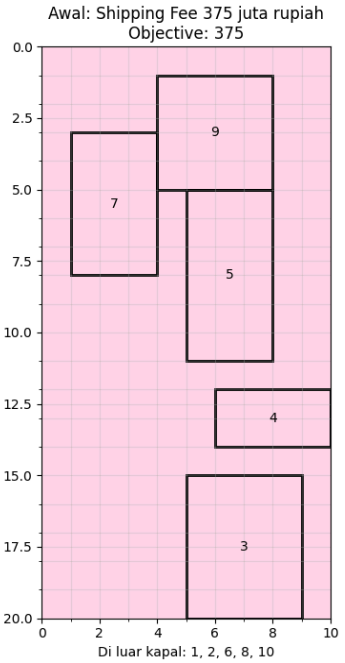
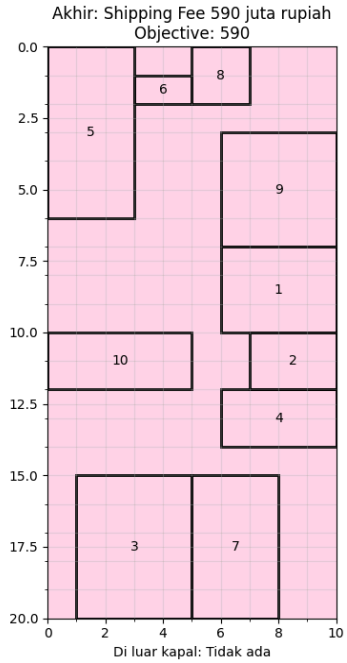
Tabel 4.6.10 Plot Nilai Objective Function Algoritma Genetic Algorithm Iterasi=300 Populasi=50

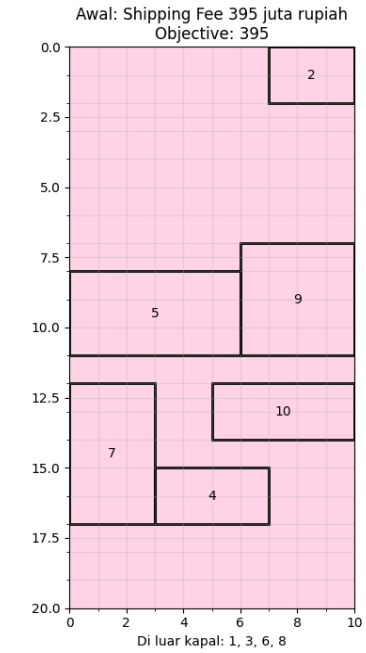
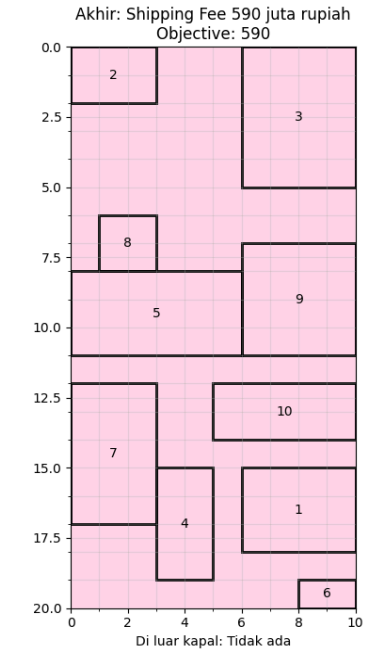
Run	Plot nilai objective function
1	
2	

3	 Objective: Total Shipping Fee Maksimum populasi Rata-rata populasi Shipping Fee (juta rupiah; -1 = invalid) 600 500 400 300 200 100 0 50 100 150 200 250 300 Generasi

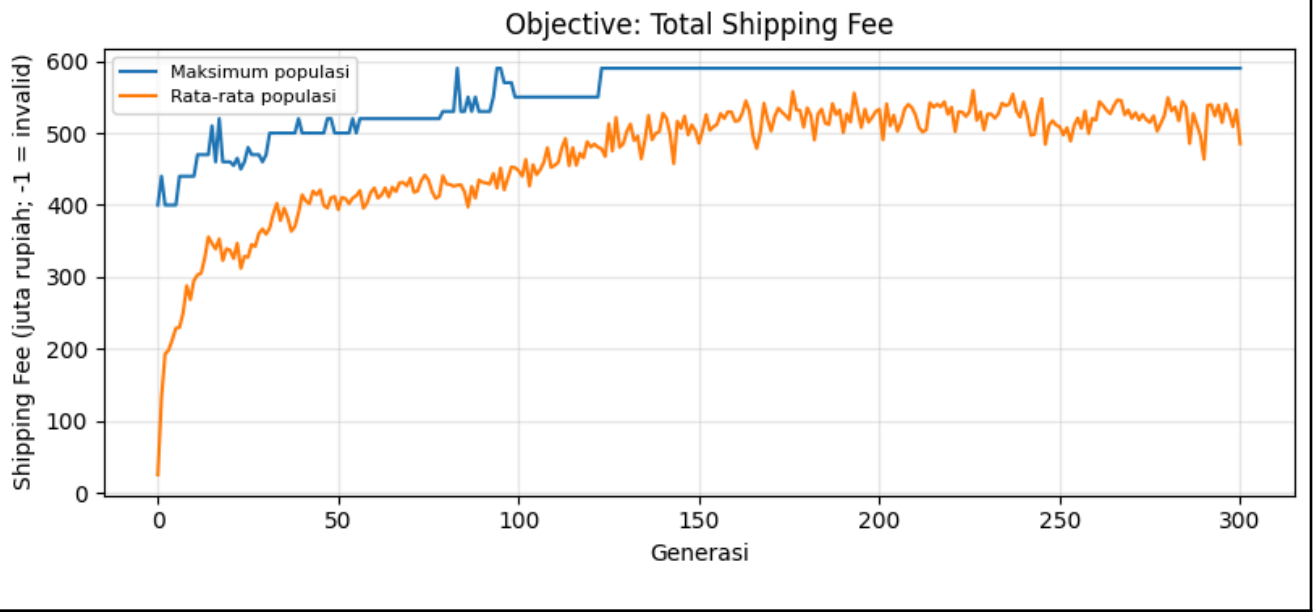
Iterasi = 300, Populasi = 100

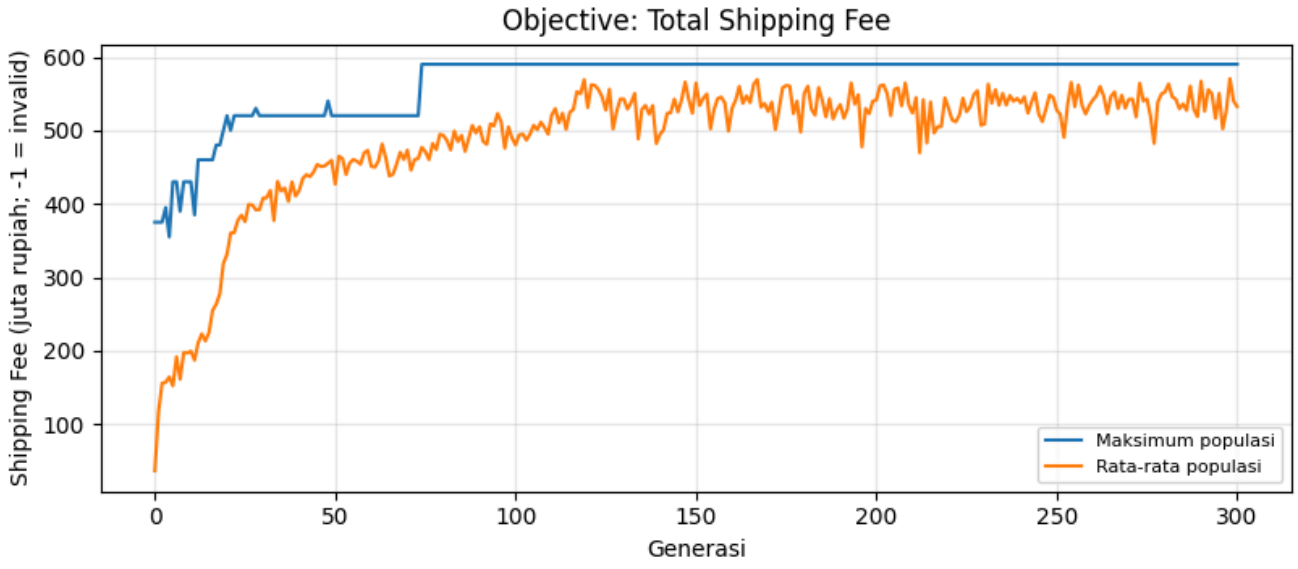
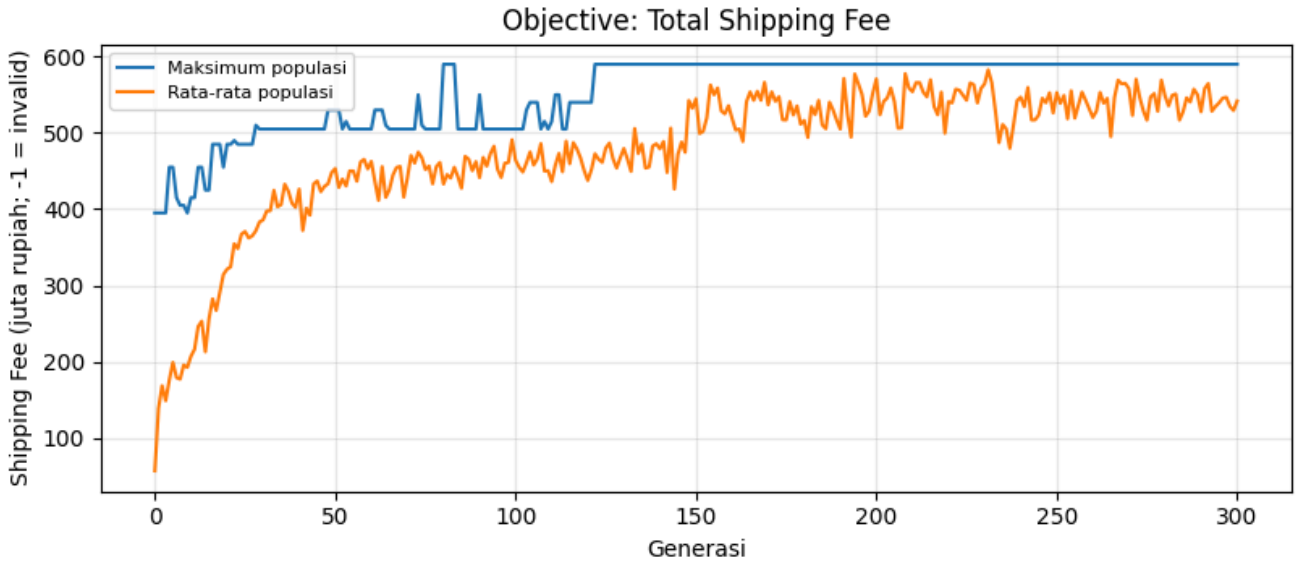
Tabel 4.6.11 Tabel Hasil Eksperimen Genetic Algorithm Iterasi=300 Populasi=100

Run	State awal	State akhir	Objective	Durasi	Total Iterasi	Banyak Populasi
1			590	7.29	300	100
2			590	5.07	300	100

3			590	2.73	300	100

Tabel 4.6.12 Plot Nilai Objective Function Algoritma Genetic Algorithm Iterasi=300 Populasi=100

Run	Plot nilai objective function
1	

2	 Objective: Total Shipping Fee Shipping Fee (juta rupiah; -1 = invalid) 600 500 400 300 200 100 0 50 100 150 200 250 300 Generasi Maksimum populasi Rata-rata populasi
3	 Objective: Total Shipping Fee Shipping Fee (juta rupiah; -1 = invalid) 600 500 400 300 200 100 0 50 100 150 200 250 300 Generasi Maksimum populasi Rata-rata populasi

BAB 5: Analisis Eksperimen

5.1 Pertanyaan Pertama

Seberapa dekat tiap-tiap algoritma bisa mendekati global optima dan mengapa hasilnya demikian?

Global optimum dari dataset data.json yang digunakan untuk eksperimen ini dapat dibuktikan sebesar 590 juta rupiah. Nilai tersebut merupakan jumlah ShippingFee seluruh 10 kendaraan sehingga tidak ada solusi yang dapat melampauinya lagi. Beberapa state hasil eksperimen memuat seluruh kendaraan tanpa overlap, keluar batas, atau kelebihan kapasitas sehingga batas atas 590 juta ini terbukti dapat dicapai.

Algoritma	Objective akhir	Rata-rata	Optimum
Steepest Ascent	-1, 590, 590	- (ada yang invalid)	2/3
Sideways Move	590, 590, 590	590,00	3/3
Stochastic	590, 515, 590	565,00	2/3
Random Restart	590, 590, 590	590,00	3/3
Simulated Annealing	455, 485, 495	478,33	0/3
Genetic Algorithm	Rentang 495 - 590	540,28	6/18

Steepest Ascent mencapai optimum pada 2 dari 3 run. Satu run ada menghasilkan -1 karena state awal yang tidak valid dan tidak dapat menemukan neighbor dengan fitness lebih baik. **Sideways Move** mencapai optimum pada seluruh run. State awalnya yang valid dan pemilihan neighbor terbaik membantu pencarian mencapai nilai global optimum. Pada percobaan ini, perpindahan sideways pun baru terjadi setelah optimum tercapai. **Stochastic** hampir mencapai angka global optimum. Pemilihan neighbor secara acak membuat peluang menemukan perpindahan terbaik berbeda pada setiap run. **Random Restart** mencapai optimum pada seluruh run. Beberapa titik awal memungkinkan

pencarian lain dilakukan dengan mengatasi kegagalan suatu restart. **Simulated Annealing** memiliki tingkat pencapaian optimum paling rendah. Dengan parameter yang digunakan, suhu masih cukup tinggi saat iterasi berakhir sehingga pencarian masih banyak melakukan eksplorasi. **Genetic Algorithm** menghasilkan 6 dari 18 run mencapai optimum. Tanpa elitisme, individu optimal yang sudah ditemukan dapat hilang sebelum generasi terakhir.

Hasil ini berlaku untuk dataset dan parameter eksperimen yang digunakan. Hasil run berikutnya dapat berbeda karena pengacakan state awal, neighbor, dan proses pencarian, serta perubahan parameter. Oleh karena itu, hasil eksperimen ini tidak menjamin kinerja yang sama pada semua percobaan atau dataset.

5.2 Pertanyaan Kedua

Bagaimana perbandingan hasil pencarian tiap-tiap algoritma dengan algoritma local search yang lain?

Perbandingan hasil pencarian:

Algoritma	Terbaik	Terburuk	Rata-rata (run valid)
HC - Steepest Ascent	590	-1 (melanggar constraint)	590
HC - Sideways Move	590	590	590
HC - Random Restart	590	590	590
HC - Stochastic	590	515	565
Simulated Annealing	495	455	478,33
Genetic Algorithm	590	495	540,28

Secara umum, varian Hill Climbing terutama Sideways Move dan Random Restart memberikan hasil yang terbaik tanpa ada yang melanggar constraint. Hal ini karena fungsi best_neighbor() mengevaluasi semua kemungkinan manipulasi kendaraan untuk memilih

successor terbaik dan menguntungkan. Di lain hal, Steepest Ascent dan Sideways Move memiliki cara pemilihan neighbor yang sebenarnya sama. Namun, kelemahan Steepest Ascent adalah ada state awal yang bernilai -1 dan tidak ada neighbor yang lebih baik sehingga pencarian langsung berhenti dalam keadaan invalid akibat ada constraint yang dilanggar. Hal ini menyebabkan algoritma stuck pada -1 karena tidak bisa bergeser ke neighbor yang bahkan sebenarnya memiliki nilai sama (tidak lebih buruk).

Simulated Annealing memberikan hasil yang paling rendah. SA hanya membangkitkan satu neighbor acak per iterasi. Hal ini disebabkan karena pengaturan parameter yang tidak optimal. Dengan T_awal = 1000 dan cooling rate 0,995, suhu pada iterasi ke-300 masih sekitar $1000 \times 0{,}995^{300} \approx 222$. Pada suhu ini, langkah turun sebesar 50 juta masih diterima dengan peluang e^(−50/222) ≈ 0,80.

Kemudian, algoritma yang memanfaatkan banyak titik awal adalah Random Restart dan Genetic Algorithm. Namun, GA dinilai kurang efektif. Random Restart melakukan eksploitasi penuh dari setiap titik awal, sedangkan GA banyak bergantung pada crossover dan mutasi acak.

Apabila disimpulkan, urutan kualitas hasil pada eksperimen kali ini dengan data kendaraan yang ada adalah Sideways Move >> Random Restart >> Steepest Ascent >> Stochastic >> Genetic Algorithm >> Simulated Annealing.

5.3 Pertanyaan Ketiga

Bagaimana perbandingan durasi proses pencarian tiap algoritma relatif terhadap algoritma lainnya?

Berdasarkan eksperimen, rata-rata durasi pencarian per run adalah sebagai berikut.

Algoritma	Rata-rata durasi (detik)
Steepest Ascent	0,6545
Sideways Move	2,0081

Stochastic	0,0294
Random Restart	6,96
Simulated Annealing	0,0321
Genetic Algorithm	0,4 - 5,64, bergantung konfigurasi

Stochastic dan SA paling cepat karena hanya mengevaluasi satu neighbor per iterasi. Steepest dan Sideways lebih lama karena memeriksa banyak neighbor. Sideways juga melanjutkan perpindahan dengan fitness sama. Durasi GA meningkat seiring jumlah populasi dan generasi. Random Restart memiliki rata-rata terlama karena menjalankan pencarian HC sebanyak 10 kali per run. Namun, rata-rata Steepest dipengaruhi satu run yang berhenti cepat dalam kondisi invalid. Waktu yang lebih singkat belum tentu menunjukkan pencarian yang lebih baik.

5.4 Pertanyaan Keempat

Seberapa konsisten hasil akhir yang didapatkan dari tiap-tiap eksperimen yang dilakukan?

Berdasarkan eksperimen yang dilakukan **Sideways Move dan Random Restart** paling konsisten karena ketiga run selalu menghasilkan global optimum 590 juta rupiah. Steepest Ascent mendapat dua run mencapai 590, tetapi satu run invalid sehingga keberhasilannya belum konsisten. Stochastic seluruh hasilnya valid, tetapi dengan value yang lebih bervariasi. Simulated annealing seluruh hasilnya valid, tetapi valuenya lebih bervariasi dan rendah dibanding algoritma lainnya. Genetic Algorithm hasilnya juga bervariasi tergantung pada konfigurasi. Populasi 100 dengan 300 generasi paling konsisten di antara konfigurasi GA yang diuji karena seluruh runnya mencapai global optimum. Perbedaan hasil tersebut dapat dipengaruhi karena adanya pengacakan dan parameter pencarian. Karena setiap konfigurasi hanya diuji tiga kali, konsistensi tersebut berlaku pada eksperimen ini dan belum menjamin hasil yang sama pada percobaan berikutnya.

5.5 Pertanyaan Kelima

Bagaimana pengaruh banyak iterasi dan jumlah populasi terhadap hasil akhir pencarian pada Genetic Algorithm?

Skenario 1: Populasi tetap (50), iterasi bervariasi

Iterasi	Objective akhir	Rata-rata	Rata-rata durasi
100	505, 590, 525	540	0.4
300	515, 590, 590	565	1.21
600	520, 495, 560	525	5.64

Jumlah iterasi tidak berpengaruh pada hasil. Rata-rata naik dari 100 ke 300 iterasi, tetapi turun lagi pada 600 iterasi. Plot menunjukkan sebagian besar kenaikan fitness terjadi pada sekitar 50–100 generasi pertama. Setelah itu, fitness rata-rata populasi hanya naik-turun di sekitar nilai yang sama. Contohnya terlihat pada run 2 dengan 600 iterasi dengan fitness maksimum sempat mendekati 590 sekitar generasi 400, tetapi hasil akhirnya hanya 495. Sementara itu, durasi naik hampir sebanding dengan jumlah iterasi.

Skenario 2: Iterasi tetap (300), populasi bervariasi

Populasi	Objective akhir	Rata-rata	Rata-rata durasi
20	520, 495, 520	511.67	0.46
50	495, 505, 530	510	1.21
100	590, 590, 590	590	5.03

Jumlah populasi cukup berpengaruh pada objective. Pada populasi 20, fitness maksimum naik pelan secara bertahap dan rata-ratanya sangat fluktuatif. Penyebabnya adalah keragaman genetik yang rendah, sehingga crossover jarang menghasilkan kombinasi baru yang lebih baik. Pada populasi 100, fitness maksimum sudah mencapai 590 pada sekitar generasi 125 dan bertahan sampai akhir. Populasi besar membawa lebih banyak variasi gen. Namun, waktu komputasi naik hampir sebanding dengan ukuran populasi.

BAB 6: Kesimpulan dan Saran

6.1 Kesimpulan

Implementasi algoritma local search berhasil diterapkan untuk menentukan penempatan kendaraan pada geladak kapal dengan tujuan memaksimalkan total Shipping Fee. Fungsi evaluasi dapat mendeteksi pelanggaran overlap, batas geladak, dan kapasitas angkut melalui penalti −1. Namun, penalti tersebut belum menjamin seluruh hasil akhir valid, sebagaimana terlihat pada satu run Steepest Ascent yang berhenti dalam kondisi invalid.

Pada dataset yang digunakan, global optimum terbukti sebesar 590 juta rupiah, yaitu total Shipping Fee seluruh kendaraan. Sideways Move dan Random Restart menunjukkan hasil paling konsisten karena mencapai nilai tersebut pada ketiga pengulangan. Steepest Ascent dan Stochastic masing-masing mencapai optimum pada dua dari tiga run. Stochastic menghasilkan rata-rata 565 juta rupiah, sedangkan Simulated Annealing menghasilkan rata-rata 478,33 juta rupiah.

Genetic Algorithm menghasilkan rata-rata 540,28 juta rupiah dari seluruh konfigurasi, dengan delapan dari 18 run berakhir optimal. Konfigurasi populasi 100 dan 300 generasi memberikan hasil paling konsisten karena seluruh pengulangannya mencapai 590. Meskipun demikian, penambahan populasi atau generasi tidak selalu meningkatkan hasil akhir. Tidak adanya elitisme menyebabkan individu optimal yang pernah ditemukan dapat hilang sebelum pencarian berakhir.

Dari sisi waktu, Stochastic dan Simulated Annealing merupakan algoritma tercepat karena hanya mengevaluasi satu neighbor per iterasi. Sebaliknya, Random Restart memiliki rata-rata durasi terlama karena mengulangi pencarian dari beberapa titik awal. Hasil ini menunjukkan adanya pertukaran antara biaya komputasi, kualitas solusi, dan konsistensi keberhasilan.

Secara keseluruhan, keberhasilan pencarian dipengaruhi oleh validitas state awal, mekanisme pemilihan neighbor, parameter, dan pengacakan. Oleh karena itu, hasil

eksperimen ini menggambarkan kinerja pada dataset dan konfigurasi yang diuji, bukan jaminan bahwa suatu algoritma selalu lebih unggul pada seluruh permasalahan.

6.2 Saran

Berikut adalah saran-saran yang dapat diikuti untuk eksperimen berikutnya.

1. Seluruh algoritma sebaiknya menggunakan pembentukan state awal acak yang memenuhi constraints. Hal ini mengurangi risiko pencarian tertahan pada kumpulan state invalid yang sama-sama bernilai −1, sekaligus membuat perbandingan antaralgoritma lebih adil.
2. Parameter Simulated Annealing perlu disesuaikan agar algoritma dapat menjelajahi berbagai solusi pada awal pencarian dan lebih selektif menerima solusi menjelang akhir pencarian.
3. Genetic Algorithm sebaiknya dilengkapi elitisme dan penyimpanan solusi terbaik agar kualitas yang telah dicapai tidak hilang pada generasi berikutnya. Pengujian populasi, generasi, dan mutation rate juga perlu diperluas untuk menilai keseimbangan kualitas dan durasi.
4. Pencarian dapat dihentikan apabila solusi valid sudah mencapai batas atas yang terbukti. Pada dataset ini, langkah tersebut berpotensi mengurangi perpindahan sideways dan restart tambahan setelah Shipping Fee mencapai 590.
5. Eksperimen selanjutnya sebaiknya menggunakan variasi jumlah kendaraan, dimensi geladak, dan kapasitas kapal. Pencatatan seed acak, penambahan jumlah pengulangan, serta perbandingan dengan anggaran evaluasi yang sebanding akan membantu menghasilkan kesimpulan yang lebih kuat dan mudah direproduksi.

Pembagian Tugas

Tabel 8.1 Pembagian Tugas Anggota Kelompok

Anggota Kelompok	Tugas
Keisha Sakura Widjaja 18224002	1. Mengerjakan algoritma Hill Climbing: Sideways Move 2. Mengerjakan algoritma Simulated Annealing 3. Inisialisasi main program 4. Mengerjakan pembahasan implementasi algoritma Hill Climbing: Sideways Move 5. Mengerjakan pembahasan implementasi algoritma Simulated Annealing 6. Mengerjakan hasil eksperimen algoritma Hill Climbing: Sideways Move 7. Mengerjakan hasil eksperimen algoritma Simulated Annealing 8. Mengerjakan analisis pada pertanyaan 9. Mengerjakan bagian kesimpulan dan saran
Claudia Melati Krid 18224032	1. Mengerjakan algoritma Genetic Algorithm 2. Mengerjakan file evaluation.py untuk fungsional evaluasi dan utilitas umum 3. Mengerjakan pembahasan algoritma Genetic Algorithm 4. Mengerjakan pembahasan bagian evaluation functions 5. Membuat deskripsi hill climbing: sideway move 6. Membuat deskripsi simulated annealing 7. Mengerjakan hasil eksperimen Genetic Algorithm

	8. Mengerjakan pertanyaan analisis kedua dan kelima
Zahra Zahidah Putri Ruchwijaya 18224058	1. Mengerjakan algoritma Hill Climbing Steepest Ascent 2. Mengerjakan algoritma Hill Climbing Stochastic 3. Mengerjakan algoritma Hill Climbing Random Restart 4. Mengerjakan Bab 1: Deskripsi Persoalan 5. Mengerjakan Bab 2: Pembahasan bagian Pemilihan Fungsi Initial State dan Pemilihan Objective Function 6. Mengerjakan penjelasan algoritma dan hasil eksperimen Hill Climbing Steepest Ascent 7. Mengerjakan penjelasan algoritma dan hasil eksperimen Hill Climbing Stochastic 8. Mengerjakan penjelasan algoritma dan hasil eksperimen Hill Climbing Random Restart 9. Melakukan formatting dan finalisasi laporan

Referensi

[1] M. L. Khodra, "Beyond Classical Search: Classical vs Local Search," YouTube, 2021. [Video]. Available: Beyond Classical Search: Classical vs Local Search [Accessed: Sep. 16, 2026].

[2] M. L. Khodra, "Beyond Classical Search: State (Value, Neighbor)," YouTube, 2021. [Video]. Available: https://www.youtube.com/watch?v=_6rIa37QorM. [Accessed: Sep. 16, 2026].

[3] M. L. Khodra, "Beyond Classical Search: Hill-climbing Search," YouTube, 2021. [Video]. Available: https://www.youtube.com/watch?v=L0b0CN3Fds8. [Accessed: Sep. 16, 2026].

[4] M. L. Khodra, "Beyond Classical Search: Simulated Annealing," YouTube, 2021. [Video]. Available: Beyond Classical Search: Simulated Annealing [Accessed: Sep. 16, 2026].

[5] M. L. Khodra, "Beyond Classical Search: Genetic Algorithm," YouTube, 2021. [Video]. Available: Beyond Classical Search: Genetic Algorithm [Accessed: Sep. 16, 2026].

FORMAT DEKLARASI PENGGUNAAN AI
SURAT PERNYATAAN PENGGUNAAN AI

Dengan ini, ~~saya~~/kami*:

Nama/NIM	: Keisha Sakura Widjaja / 18224002, Claudia Melati Krid / 18224032, Zahra Zahidah Putri Ruchwijaya / 18224058
Fakultas/Sekolah	: Sekolah Teknik Elektro dan Informatika – Komputasi
Kode Mata Kuliah	: IF3070
Nama Mata Kuliah	: Dasar Intelegensi Artifisial
Judul Tugas/Proposal	: Tugas Besar 1 IF3070 Dasar Intelegensi Artifisial – Pencarian Solusi Kapal Kargo Pengangkut dengan Local Search

Menyatakan bahwa ~~saya~~/kami* menggunakan/~~tidak menggunakan~~* AI dalam pengerjaan maupun penyusunan luaran tugas pada mata kuliah yang tertulis di atas.

Jika Ya, maka alat AI/kecerdasan buatan yang digunakan adalah:

Lingkup Pekerjaan	Digunakan?	Tingkat Penggunaan
Pemeriksaan Ejaan: menggunakan tools seperti Grammarly, Paperpal, Deepl, Quillbot, Grammarbot, LanguageTool, ProWritingAid, ChatGPT, Google Gemini, atau sejenisnya.	~~Ya~~/Tidak*	0%
Pembuatan Teks: menggunakan tools seperti ChatGPT, Gemini, Paperpal, Copilot, GrammarlyGO, WordAI, WriteSonic, Jasper, Jenni AI, atau sejenisnya.	Ya/~~Tidak~~*	40%
Bantuan Diskusi dalam Penyusunan Konten: menggunakan tools seperti ChatGPT, Gemini, Copilot, Perplexity, Jenni AI, Zoom-Companion, atau sejenisnya.	Ya/~~Tidak~~*	50%
Pembuatan Gambar, Video, dan/atau Grafik: menggunakan tools seperti Craiyon, DALL-E, Midjourney, Stable Diffusion, Microsoft Designer, Gemini, Canva AI, atau sejenisnya.	~~Ya~~/Tidak*	0%
Lainnya, Jelaskan:		

~~Saya~~/Kami* juga menyatakan bahwa setiap penggunaan AI/kecerdasan buatan yang dilakukan pada mata kuliah tersebut telah dinyatakan di dalam surat ini.

Bandung, 22 Oktober 2026



Claudia Melati Krid

18224032